Python Setup and Usage

Release 3.14.0rc3

Guido van Rossum and the Python development team

outubro 01, 2025

Python Software Foundation
Email: docs@python.org

Sumário

1 Linha de comando e ambiente **3**
- 1.1 Linha de comando . . . 3
  - 1.1.1 Opções de interface . . . 3
  - 1.1.2 Opções genéricas . . . 5
  - 1.1.3 Opções diversas . . . 6
  - 1.1.4 Controlando cores . . . 11
- 1.2 Variáveis de ambiente . . . 11
  - 1.2.1 Variáveis de modo de depuração . . . 18

2 Utilizando Python em plataformas Unix **21**
- 2.1 Obtendo e instalando a versão mais recente do Python . . . 21
  - 2.1.1 No Linux . . . 21
  - 2.1.2 No FreeBSD e OpenBSD . . . 22
- 2.2 Compilando o Python . . . 22
- 2.3 Paths e arquivos relacionados com o Python . . . 22
- 2.4 Diversos . . . 23
- 2.5 OpenSSL personalizado . . . 23

3 Configurando o Python **25**
- 3.1 Requisitos de construção . . . 25
- 3.2 Arquivos gerados . . . 26
  - 3.2.1 Script configure . . . 26
- 3.3 Opções de configuração . . . 26
  - 3.3.1 Opções gerais . . . 26
  - 3.3.2 Opções do compilador C . . . 29
  - 3.3.3 Opções da ligação . . . 30
  - 3.3.4 Opções para dependências de terceiros . . . 30
  - 3.3.5 Opções de WebAssembly . . . 31
  - 3.3.6 Opções de instalação . . . 32
  - 3.3.7 Opções de desempenho . . . 32
  - 3.3.8 Compilação de depuração do Python . . . 34
  - 3.3.9 Opções de depuração . . . 35
  - 3.3.10 Opções da ligação . . . 36
  - 3.3.11 Opções da biblioteca . . . 36
  - 3.3.12 Opções de segurança . . . 37
  - 3.3.13 Opções do macOS . . . 38
  - 3.3.14 Opções do iOS . . . 39
  - 3.3.15 Opções de compilação cruzada . . . 39
- 3.4 Sistema de Construção Python . . . 40
  - 3.4.1 Arquivos principais do sistema de construção . . . 40
  - 3.4.2 Principais etapas de construção . . . 40

3.4.3 Alvos principais do Makefile . . . 40
3.4.4 Extensões C . . . 41
3.5 Sinalizadores do compilador e do vinculador . . . 42
3.5.1 Sinalizadores do pré-processador . . . 42
3.5.2 Sinalizadores do compilador . . . 42
3.5.3 Sinalizadores do vinculador . . . 44

4 Utilizando Python no Windows **47**
4.1 Gerenciador de instalação do Python . . . 47
4.1.1 Instalação . . . 47
4.1.2 Uso básico . . . 48
4.1.3 Comando help . . . 49
4.1.4 Listando ambientes de execução . . . 49
4.1.5 Instalando ambientes de execução . . . 50
4.1.6 Instalações offline . . . 50
4.1.7 Desinstalando ambientes de execução . . . 51
4.1.8 Configuração . . . 51
4.1.9 Shebang lines . . . 53
4.1.10 Advanced Installation . . . 53
4.1.11 Administrative Configuration . . . 55
4.1.12 Instalando binários com threads livres . . . 55
4.1.13 Troubleshooting . . . 56
4.2 O pacote embutível . . . 58
4.2.1 Aplicação Python . . . 58
4.2.2 Incorporando Python . . . 58
4.3 Os pacotes nuget.org . . . 59
4.3.1 Pacotes com threads livres . . . 59
4.4 Pacotes Alternativos . . . 60
4.5 Supported Windows versions . . . 60
4.6 Removendo a Limitação do MAX_PATH . . . 60
4.7 Modo UTF-8 . . . 60
4.8 Encontrando módulos . . . 61
4.9 Módulos adicionais . . . 62
4.9.1 PyWin32 . . . 62
4.9.2 cx_Freeze . . . 63
4.10 Compilando Python no Windows . . . 63
4.11 The full installer (deprecated) . . . 63
4.11.1 Etapas de instalação . . . 63
4.11.2 Removendo a Limitação do MAX_PATH . . . 64
4.11.3 Instalando sem UI . . . 64
4.11.4 Instalando Sem Download . . . 67
4.11.5 Modificando uma instalação . . . 67
4.11.6 Instalando binários com threads livres . . . 68
4.12 Python Launcher for Windows (Deprecated) . . . 69
4.12.1 Começando . . . 69
4.12.2 Linhas Shebang . . . 71
4.12.3 Argumentos em linhas shebang . . . 72
4.12.4 Personalização . . . 72
4.12.5 Diagnósticos . . . 73
4.12.6 Teste de simulação . . . 73
4.12.7 Instalar sob demanda . . . 73
4.12.8 Códigos de retorno . . . 73

5 Usando Python em macOS **75**
5.1 Usando Python para macOS do `python.org` . . . 75
5.1.1 Etapas de instalação . . . 75
5.1.2 Como executar um script Python . . . 83
5.2 Distribuições alternativas . . . 83

5.3 Instalando pacotes adicionais ao python . . . 83
5.4 Programação de GUI . . . 84
5.5 Tópicos Avançados . . . 84
5.5.1 Instalando binários com threads livres . . . 84
5.5.2 Instalação usando a linha de comando . . . 86
5.5.3 Distribuindo aplicações Python . . . 87
5.5.4 Conformidade com a App Store . . . 87
5.6 Outros recursos . . . 88

6 Utilizando Python no Android **89**
6.1 Adicionando Python a uma aplicação Android . . . 89
6.2 Construindo um pacote Python para Android . . . 90

7 Usando Python no iOS **91**
7.1 Python em tempo de execução no iOS . . . 91
7.1.1 Compatibilidade com a versão do iOS . . . 91
7.1.2 Identificação da plataforma . . . 91
7.1.3 Disponibilidade da biblioteca padrão . . . 92
7.1.4 Módulos de extensão binária . . . 92
7.1.5 Binários stub do compilador . . . 92
7.2 Instalando Python no iOS . . . 93
7.2.1 Ferramentas para construir aplicações de iOS . . . 93
7.2.2 Adicionando Python a um projeto iOS . . . 93
7.2.3 Testando um pacote Python . . . 94
7.3 Conformidade com a App Store . . . 95
7.3.1 Incompatible code in the standard library . . . 95
7.3.2 Privacy manifests . . . 95

8 Editores e IDEs **97**
8.1 IDLE — editor e console Python . . . 97
8.2 Outros editores e IDEs . . . 97

A Glossário **99**

B Sobre esta documentação **119**
B.1 Contribuidores da documentação do Python . . . 119

C História e Licença **121**
C.1 História do software . . . 121
C.2 Termos e condições para acessar ou usar Python . . . 122
C.2.1 PYTHON SOFTWARE FOUNDATION LICENSE VERSION 2 . . . 122
C.2.2 ACORDO DE LICENCIAMENTO DA BEOPEN.COM PARA PYTHON 2.0 . . . 123
C.2.3 CONTRATO DE LICENÇA DA CNRI PARA O PYTHON 1.6.1 . . . 124
C.2.4 ACORDO DE LICENÇA DA CWI PARA PYTHON 0.9.0 A 1.2 . . . 125
C.2.5 ZERO-CLAUSE BSD LICENSE FOR CODE IN THE PYTHON DOCUMENTATION . . . 125
C.3 Licenças e Reconhecimentos para Software Incorporado . . . 125
C.3.1 Mersenne Twister . . . 125
C.3.2 Soquetes . . . 126
C.3.3 Serviços de soquete assíncrono . . . 127
C.3.4 Gerenciamento de cookies . . . 127
C.3.5 Rastreamento de execução . . . 128
C.3.6 Funções UUencode e UUdecode . . . 128
C.3.7 Chamadas de procedimento remoto XML . . . 129
C.3.8 test_epoll . . . 129
C.3.9 kqueue de seleção . . . 130
C.3.10 SipHash24 . . . 130
C.3.11 strtod e dtoa . . . 131
C.3.12 OpenSSL . . . 131
C.3.13 expat . . . 135

C.3.14 libffi . . . 135
C.3.15 zlib . . . 136
C.3.16 cfuhash . . . 136
C.3.17 libmpdec . . . 137
C.3.18 Conjunto de testes C14N do W3C . . . 137
C.3.19 mimalloc . . . 138
C.3.20 asyncio . . . 138
C.3.21 Global Unbounded Sequences (GUS) . . . 139
C.3.22 Ligações Zstandard . . . 139

D Direitos autorais **141**

Índice **143**

Esta parte da documentação é dedicada a informações gerais sobre a configuração do ambiente Python em diferentes plataformas, a chamada do interpretador e outras coisas que facilitam o trabalho com o Python.

CAPÍTULO 1

Linha de comando e ambiente

O interpretador do CPython verifica a linha de comando e o ambiente em busca de várias configurações.

Os esquemas de linha de comando de outras implementações podem ser diferentes. Consulte implementations para mais recursos.

1.1 Linha de comando

Ao invocar o Python, você pode especificar qualquer uma destas opções:

```
python [-bBdEhiIOPqRsSuvVWx?] [-c comando | -m nome-módulo | script | - ] [args]
```

O caso de uso mais comum é, obviamente, uma simples invocação de um script:

```
python meuscript.py
```

1.1.1 Opções de interface

A interface do interpretador é semelhante à do console do UNIX, mas fornece alguns métodos adicionais de chamada:

- Quando chamado com a entrada padrão conectada a um dispositivo tty, ele solicita comandos e os executa até um EOF (um caractere de fim de arquivo, você pode produzi-lo com `Ctrl-D` no UNIX ou `Ctrl-Z, Enter` no Windows) ser lido. Para mais sobre o modo interativo, veja tut-interac.
- Quando chamado com um argumento de nome de arquivo ou com um arquivo como entrada padrão, ele lê e executa um script desse arquivo.
- Quando chamado com um argumento de nome de diretório, ele lê e executa um script nomeado adequadamente desse diretório.
- Quando chamado com `-c command`, ele executa as instruções Python fornecidas como *command*. Aqui *command* pode conter várias instruções separadas por novas linhas. O espaço em branco à esquerda é significativo nas instruções do Python!
- Quando chamado com `-m module-name`, o módulo fornecido está localizado no caminho do módulo Python e é executado como um script.

No modo não interativo, toda a entrada é analisada antes de ser executada.

Uma opção de interface termina a lista de opções consumidas pelo interpretador, todos os argumentos consecutivos terminam em `sys.argv` – observe que o primeiro elemento, subscrito zero (`sys.argv[0]`) , é uma string que reflete a fonte do programa.

-c <command>

Executa o código Python em *command*. *command* pode ser uma ou mais instruções separadas por novas linhas, com espaços em branco à esquerda significativos, como no código normal do módulo.

Se esta opção for fornecida, o primeiro elemento de `sys.argv` será `"-c"` e o diretório atual será adicionado ao início de `sys.path` (permitindo módulos nesse diretório para ser importado como módulos de nível superior).

Levanta um evento de auditoria `cpython.run_command` com o argumento `command`.

Alterado na versão 3.14: *command* é automaticamente dedentado antes da execução.

-m <module-name>

Procura `sys.path` pelo módulo nomeado e executa seu conteúdo como o módulo `__main__`.

Como o argumento é um nome de *module*, você não deve fornecer uma extensão de arquivo (`.py`). O nome do módulo deve ser um nome de módulo Python absoluto válido, mas a implementação nem sempre pode impor isso (por exemplo, pode permitir que você use um nome que inclua um hífen).

Nomes de pacotes (incluindo pacotes de espaço de nomes) também são permitidos. Quando um nome de pacote é fornecido ao invés de um módulo normal, o interpretador irá executar `<pkg>.__main__` como o módulo principal. Esse comportamento é deliberadamente semelhante ao tratamento de diretórios e arquivos zip que são passados para o interpretador como o argumento do script.

Nota

Esta opção não pode ser usada com módulos embutidos e módulos de extensão escritos em C, uma vez que eles não possuem arquivos de módulo Python. No entanto, ele ainda pode ser usado para módulos pré-compilados, mesmo se o arquivo fonte original não estiver disponível.

Se esta opção for fornecida, o primeiro elemento de `sys.argv` será o caminho completo para o arquivo do módulo (enquanto o arquivo do módulo está sendo localizado, o primeiro elemento será definido como `"-m"`). Como com a opção *-c*, o diretório atual será adicionado ao início de `sys.path`.

A opção *-I* pode ser usada para executar o script em modo isolado onde `sys.path` não contém nem o diretório atual nem o diretório de pacotes de sites do usuário. Todas as variáveis de ambiente `PYTHON*` são ignoradas também.

Muitos módulos de biblioteca padrão contêm código que é chamado em sua execução como um script. Um exemplo é o módulo `timeit`:

```
python -m timeit -s "configuração inicial aqui" "código a ser analisado aqui"
python -m timeit -h # para detalhes
```

Levanta um evento de auditoria `cpython.run_module` com o argumento `module-name`.

Ver também

runpy.run_module()
Funcionalidade equivalente diretamente disponível para o código Python

PEP 338 – Executando módulos como scripts

Alterado na versão 3.1: Forneça o nome do pacote para executar um submódulo `__main__`.

Alterado na versão 3.4: pacotes de espaço de nomes também são suportados

-

Lê os comandos da entrada padrão (`sys.stdin`). Se a entrada padrão for um terminal, `-i` está implícito.

Se esta opção for fornecida, o primeiro elemento de `sys.argv` será `"-"` e o diretório atual será adicionado ao início de `sys.path`.

Levanta um evento de auditoria `cpython.run_stdin` sem argumentos.

<script>

Executa o código Python contido em *script*, que deve ser um caminho do sistema de arquivos (absoluto ou relativo) referindo-se a um arquivo Python, um diretório contendo um arquivo `__main__.py`, ou um arquivo zip contendo um arquivo `__main__.py`.

Se esta opção for fornecida, o primeiro elemento de `sys.argv` será o nome do script conforme fornecido na linha de comando.

Se o nome do script se referir diretamente a um arquivo Python, o diretório que contém esse arquivo é adicionado ao início de `sys.path`, e o arquivo é executado como o módulo `__main__`.

Se o nome do script se referir a um diretório ou arquivo zip, o nome do script será adicionado ao início de `sys.path` e o arquivo `__main__.py` nesse local será executado como o módulo `__main__`.

A opção `-I` pode ser usada para executar o script em modo isolado onde `sys.path` não contém nem o diretório do script nem o diretório de pacotes de sites do usuário. Todas as variáveis de ambiente `PYTHON*` são ignoradas também.

Levanta um evento de auditoria `cpython.run_file` com o argumento `filename`.

Ver também

runpy.run_path()
Funcionalidade equivalente diretamente disponível para o código Python

Se nenhuma opção de interface for fornecida, `-i` está implícito, `sys.argv[0]` é uma string vazia (`""`) e o diretório atual será adicionado ao início de `sys.path`. Além disso, o preenchimento por tab e a edição do histórico são habilitados automaticamente, se disponíveis em sua plataforma (veja rlcompleter-config).

Ver também

tut-invoking

Alterado na versão 3.4: Ativação automática de preenchimento com tab e edição de histórico.

1.1.2 Opções genéricas

-?

-h

--help

Imprime uma breve descrição de todas as opções de linha de comando e variáveis de ambiente correspondentes e sai.

--help-env

Imprime uma breve descrição das variáveis de ambiente específicas do Python e sai.

Adicionado na versão 3.11.

--help-xoptions

Imprime uma descrição das opções `-X` específica da implementação e sai.

Adicionado na versão 3.11.

--help-all

Imprime as informações de uso completas e sai.

Adicionado na versão 3.11.

-V

--version

Exibe o número da versão do Python e saia. O exemplo de saída poderia ser:

```
Python 3.8.0b2+
```

Quando fornecido duas vezes, exibe mais informações sobre a construção, como:

```
Python 3.8.0b2+ (3.8:0c076caaa8, Apr 20 2019, 21:55:00)
[GCC 6.2.0 20161005]
```

Adicionado na versão 3.6: A opção `-VV`.

1.1.3 Opções diversas

-b

Emite um aviso ao converter `bytes` ou `bytearray` para `str` sem especificar codificação ou comparar `bytes` ou `bytearray` com `str` ou `bytes` com `int`. Emite um erro quando a opção é fornecida duas vezes (`-bb`).

Alterado na versão 3.5: Afeta também comparações de `bytes` com `int`.

-B

Se fornecido, Python não tentará escrever arquivos `.pyc` na importação de módulos fonte. Veja também *PYTHONDONTWRITEBYTECODE*.

--check-hash-based-pycs `default|always|never`

Controla o comportamento de validação de arquivos `.pyc` baseados em hash. Veja pyc-invalidation. Quando definido como `default`, os arquivos de cache bytecode baseados em hash verificados e não verificados são validados de acordo com sua semântica padrão. Quando definido como `always`, todos os arquivos `.pyc` baseados em hash, sejam verificados ou não verificados, são validados para seus arquivos fonte correspondentes. Quando definido como `never`, os arquivos `.pyc` baseados em hash não são validados para seus arquivos fonte correspondentes.

A semântica dos arquivos `.pyc` baseados em marca de tempo não é afetada por esta opção.

-d

Ativa a saída analisador sintático de depuração (uso avançado). Consulte também a variável de ambiente *PYTHONDEBUG*.

Essa opção requer uma *construção de depuração do Python*, caso contrário, será ignorada.

-E

Ignora todas as variáveis de ambiente `PYTHON*`, por exemplo *PYTHONPATH* e *PYTHONHOME*, que pode ser definido.

Veja também as opções *-P* e *-I* (isolado).

-i

Entra no modo interativo após a execução.

Usanr a opção *-i* vai entrar no modo interativo em qualquer uma das circunstâncias:

- quando um script é passado como primeiro argumento
- quando a opção *-c* é usada
- quando a opção *-m* é usada

O modo interativo vai ser iniciado mesmo quando a `sys.stdin` não parecer ser um terminal. O arquivo *PYTHONSTARTUP* não é lido.

Isso pode ser útil para inspecionar variáveis globais ou um stack trace (situação da pilha de execução) quando um script levanta uma exceção. Veja também *PYTHONINSPECT*.

-I

Executa o Python no modo isolado. Isso também implica nas opções *-E*, *-P* e *-s*.

No modo isolado, `sys.path` não contém o diretório do script nem o diretório de pacotes do site do usuário. Todas as variáveis de ambiente `PYTHON*` são ignoradas também. Outras restrições podem ser impostas para evitar que o usuário injete código malicioso.

Adicionado na versão 3.4.

-O

Remova as instruções de asserção e qualquer código condicional ao valor de `__debug__`. Aumenta o nome do arquivo para arquivos compilados (*bytecode*) adicionando `.opt-1` antes da extensão `.pyc` (veja **PEP 488**). Veja também *PYTHONOPTIMIZE*.

Alterado na versão 3.5: Modifica nomes de arquivos `.pyc` conforme a **PEP 488**.

-OO

Faz o mesmo que *-O* e também descarta docstrings. Aumenta o nome do arquivo para arquivos compilados (*bytecode*) adicionando `.opt-2` antes da extensão `.pyc` (veja **PEP 488**).

Alterado na versão 3.5: Modifica nomes de arquivos `.pyc` conforme a **PEP 488**.

-P

Não anexa um caminho potencialmente inseguro a `sys.path`:

- A linha de comando `python -m módulo`: Não anexa o diretório atual.
- A linha de comando `python script.py`: não anexa o diretório do script. Se for um link simbólico, resolve os links simbólicos.
- Linhas de comando `python -c código` e `python` (REPL): Não anexa uma string vazia, o que significa o diretório de trabalho atual.

Veja também a variável de ambiente *PYTHONSAFEPATH* e as opções *-E* e *-I* (isolado).

Adicionado na versão 3.11.

-q

Não exibe as mensagens de direitos autorais e de versão nem mesmo no modo interativo.

Adicionado na versão 3.2.

-R

Habilita a aleatorização com hash. Esta opção só tem efeito se a variável de ambiente *PYTHONHASHSEED* estiver configurada para qualquer valor diferente de `random`, uma vez que a aleatorização com hash é habilitada por padrão.

Em versões anteriores do Python, esta opção ativa a aleatorização com hash, para que os valores `__hash__()` dos objetos str e bytes sejam "salgados" com um valor aleatório imprevisível. Embora permaneçam constantes em um processo Python individual, eles não são previsíveis entre invocações repetidas de Python.

A aleatorização com hash se destina a fornecer proteção contra uma negação de serviço causada por entradas cuidadosamente escolhidas que exploram o pior caso de desempenho de uma inserção de dicionário, complexidade $O(n^2)$. Consulte http://ocert.org/advisories/ocert-2011-003.html para obter detalhes.

PYTHONHASHSEED permite que você defina um valor fixo para o segredo da semente de hash.

Adicionado na versão 3.2.3.

Alterado na versão 3.7: A opção não é mais ignorada.

-s

Não adiciona o diretório site-packages de usuário a sys.path.

Veja também *PYTHONNOUSERSITE*.

> **Ver também**
>
> **PEP 370** – Diretório site-packages por usuário.

-S

Desabilita a importação do módulo site e as manipulações dependentes do site de sys.path que isso acarreta. Também desabilita essas manipulações se site for explicitamente importado mais tarde (chame site.main() se você quiser que eles sejam acionados).

-u

Força os fluxos stdout e stderr a serem sem buffer. Esta opção não tem efeito no fluxo stdin.

Veja também *PYTHONUNBUFFERED*.

Alterado na versão 3.7: A camada de texto dos fluxos stdout e stderr agora é sem buffer.

-v

Exibe uma mensagem cada vez que um módulo é inicializado, mostrando o local (nome do arquivo ou módulo embutido) de onde ele é carregado. Quando fornecido duas vezes (-vv), exibe uma mensagem para cada arquivo que é verificado durante a busca por um módulo. Também fornece informações sobre a limpeza do módulo na saída.

Alterado na versão 3.10: O módulo site relata os caminhos específicos do site e os arquivos .pth sendo processados.

Veja também *PYTHONVERBOSE*.

-W arg

Controle de advertência. O mecanismo de aviso do Python por padrão exibe mensagens de aviso para sys.stderr.

As configurações mais simples aplicam uma determinada ação incondicionalmente a todos os avisos emitidos por um processo (mesmo aqueles que são ignorados por padrão):

```
-Wdefault  # Avisa uma vez por local de chamada
-Werror    # Converte para exceções
-Walways   # Avisa toda vez
-Wall      # Mesmo que -Walways
-Wmodule   # Avisa uma vez por módulo chamador
-Wonce     # Avisa uma vez por processo do Python
-Wignore   # Nunca avisar
```

Os nomes das ações podem ser abreviados conforme desejado e o interpretador irá resolvê-los para o nome da ação apropriado. Por exemplo, -Wi é o mesmo que -Wignore.

A forma completa de argumento é:

```
ação:mensagem:categoria:módulo:número-da-linha
```

Os campos vazios correspondem a todos os valores; campos vazios à direita podem ser omitidos. Por exemplo, -W ignore::DeprecationWarning ignora todos os avisos de DeprecationWarning.

O campo *action* é explicado acima, mas se aplica apenas a avisos que correspondem aos campos restantes.

O campo *message* deve corresponder a toda a mensagem de aviso; essa correspondência não diferencia maiúsculas de minúsculas.

O campo *category* corresponde à categoria de aviso (ex: `DeprecationWarning`). Deve ser um nome de classe; o teste de correspondência se a categoria de aviso real da mensagem é uma subclasse da categoria de aviso especificada.

O campo *module* corresponde ao nome do módulo (totalmente qualificado); esta correspondência diferencia maiúsculas de minúsculas.

O campo *lineno* corresponde ao número da linha, onde zero corresponde a todos os números da linha e, portanto, é equivalente a um número da linha omitido.

Múltiplas opções *-W* podem ser fornecidas; quando um aviso corresponde a mais de uma opção, a ação para a última opção correspondente é executada. As opções *-W* inválidas são ignoradas (embora, uma mensagem de aviso seja exibida sobre opções inválidas quando o primeiro aviso for emitido).

Os avisos também podem ser controlados usando a variável de ambiente *PYTHONWARNINGS* e de dentro de um programa Python usando o módulo `warnings`. Por exemplo, a função `warnings.filterwarnings()` pode ser usada para usar uma expressão regular na mensagem de aviso.

Veja warning-filter e describing-warning-filters para mais detalhes.

-x

Pula a primeira linha do código-fonte, permitindo o uso de formas não-Unix de `#!cmd`. Isso se destina apenas a um hack específico do DOS.

-X

Reservado para várias opções específicas de implementação. CPython atualmente define os seguintes valores possíveis:

- `-X faulthandler` para habilitar `faulthandler`. Veja também *PYTHONFAULTHANDLER*.

  Adicionado na versão 3.3.

- `-X showrefcount` para emitir a contagem de referências total e o número de blocos de memória usados quando o programa termina ou após cada instrução no interpretador interativo. Isso só funciona em *construções de depuração*.

  Adicionado na versão 3.4.

- `-X tracemalloc` para começar a rastrear alocações de memória do Python usando o módulo `tracemalloc`. Por padrão, apenas o quadro mais recente é armazenado no traceback (situação da pilha de execução) de um rastro. Use `-X tracemalloc=NFRAME` para iniciar o rastreamento com um limite de traceback de quadros *NFRAME*. Veja o `tracemalloc.start()` e *PYTHONTRACEMALLOC* para mais informações.

  Adicionado na versão 3.4.

- `-X int_max_str_digits` configura a limitação de comprimento de string na conversão para inteiro. Veja também *PYTHONINTMAXSTRDIGITS*.

  Adicionado na versão 3.11.

- `-X importtime` para mostrar quanto tempo leva cada importação. Mostra o nome do módulo, tempo cumulativo (incluindo importações aninhadas) e tempo próprio (excluindo importações aninhadas). Observe que sua saída pode ser interrompida em aplicações multithread. O uso típico é `python -X importtime -c 'import asyncio'`.

  `-X importtime=2` habilita uma saída adicional que indica quando um módulo importado já foi carregado. Nesses casos, a string `cached` será impressa em ambas as colunas de tempo.

  Veja também *PYTHONPROFILEIMPORTTIME*.

  Adicionado na versão 3.7.

  Alterado na versão 3.14: Adicionado `-X importtime=2` para também rastrear importações de módulos carregados e valores reservados diferentes de `1` e `2` para uso futuro.

- `-X dev`: habilita Modo de Desenvolvimento do Python, introduzindo verificações adicionais de tempo de execução que são muito custosas para serem habilitadas por padrão. Veja também *PYTHONDEVMODE*.

Adicionado na versão 3.7.

- `-X utf8` habilita o Modo UTF-8 do Python. `-X utf8=0` explicitamente desabilita o Modo UTF-8 do Python (mesmo quando de outra forma seria ativado automaticamente). Veja também *PYTHONUTF8*.

  Adicionado na versão 3.7.

- `-X pycache_prefix=PATH` permite a escrita de arquivos `.pyc` em uma árvore paralela enraizada em um determinado diretório em vez de na árvore de código. Veja também *PYTHONPYCACHEPREFIX*.

  Adicionado na versão 3.8.

- `-X warn_default_encoding` emite uma `EncodingWarning` quando a codificação padrão específica da localidade é usada para abrir arquivos. Veja também *PYTHONWARNDEFAULTENCODING*.

  Adicionado na versão 3.10.

- `-X no_debug_ranges` desabilita a inclusão das tabelas que mapeiam informações de localização extra (linha final, deslocamento da coluna inicial e deslocamento da coluna final) para cada instrução em objetos código. Isso é útil quando objetos código menores e arquivos pyc são desejados, bem como suprimir os indicadores de localização visual extra quando o interpretador exibe tracebacks (situação da pilha de execução). Veja também *PYTHONNODEBUGRANGES*.

  Adicionado na versão 3.11.

- `-X frozen_modules` determina se os módulos congelados são ou não ignorados pelo maquinário de importação. Um valor de `on` significa que eles são importados e `off` significa que eles são ignorados. O padrão é `on` se este for um Python instalado (o caso normal). Se estiver em desenvolvimento (executando a partir da árvore de origem), o padrão é `off`. Observe que os módulos congelados `importlib_bootstrap` e `importlib_bootstrap_external` são sempre usados, mesmo que esse sinalizador esteja definido como `off`. Veja também *PYTHON_FROZEN_MODULES*.

  Adicionado na versão 3.11.

- `-X perf` habilita o suporte para o perfilador do Linux `perf`. Quando essa opção for fornecida, o perfilador `perf` poderá relatar chamadas Python. Essa opção está disponível somente em algumas plataformas e não fará nada se não for compatível com o sistema atual. O valor padrão é "off". Consulte também *PYTHONPERFSUPPORT* e perf_profiling.

  Adicionado na versão 3.12.

- `-X perf_jit` habilita o suporte para o perfilador do Linux `perf` com suporte a DWARF. Quando essa opção for fornecida, o perfilador `perf` poderá relatar chamadas do Python usando informações de DWARF. Essa opção está disponível somente em algumas plataformas e não fará nada se não for compatível com o sistema atual. O valor padrão é "off". Veja também *PYTHON_PERF_JIT_SUPPORT* e perf_profiling.

  Adicionado na versão 3.13.

- `-X disable_remote_debug` desabilita o suporte à depuração remota, conforme descrito em **PEP 768**. Isso inclui tanto a funcionalidade de agendar código para execução em outro processo quanto a funcionalidade de receber código para execução no processo atual.

  Esta opção está disponível apenas em algumas plataformas e não fará nada se não for compatível no sistema atual. Veja também *PYTHON_DISABLE_REMOTE_DEBUG* e **PEP 768**.

  Adicionado na versão 3.14.

- `-X cpu_count=`*n* substitui `os.cpu_count()`, `os.process_cpu_count()` e `multiprocessing.cpu_count()`. *n* deve ser maior ou igual a 1. Esta opção pode ser útil para usuários que precisam limitar os recursos da CPU de um sistema contêiner. Veja também *PYTHON_CPU_COUNT*. Se *n* for `default`, nada será substituído.

  Adicionado na versão 3.13.

- `-X presite=`*pacote.*`,`*módulo* especifica um módulo que deve ser importado antes do módulo `site` ser executado e antes do módulo `__main__` existir. Portanto, o módulo importado não é `__main__`. Isso

pode ser usado para executar código antecipadamente durante a inicialização do Python. Python precisa ser *construído no modo de depuração* para que esta opção exista. Veja também *PYTHON_PRESITE*.

Adicionado na versão 3.13.

- `-X gil=0,1` força a GIL a ser desabilitada ou habilitada, respectivamente. A definição para `0` está disponível apenas em construções configuradas com *--disable-gil*. Veja também *PYTHON_GIL* e whatsnew313-free-threaded-cpython.

  Adicionado na versão 3.13.

- `-X thread_inherit_context=0,1` faz com que `Thread` use, por padrão, uma cópia do contexto do chamador de `Thread.start()` ao iniciar. Caso contrário, as threads iniciarão com um contexto vazio. Se não definida, o valor padrão desta opção será `1` em construções com threads livres e `0` em outras. Veja também *PYTHON_THREAD_INHERIT_CONTEXT*.

  Adicionado na versão 3.14.

- `-X context_aware_warnings=0,1` faz com que o gerenciador de contexto `warnings.catch_warnings` use uma `ContextVar` para armazenar o estado do filtro de avisos. Se não definida, o valor padrão desta opção será `1` em construções com threads livres e `0` em outras. Veja também *PYTHON_CONTEXT_AWARE_WARNINGS*.

  Adicionado na versão 3.14.

- `-X tlbc=0,1` habilita (1, o padrão) ou desabilita (0) o bytecode local da thread em construções configuradas com *--disable-gil*. Quando desabilitado, também desabilita o interpretador especializado. Veja também *PYTHON_TLBC*.

  Adicionado na versão 3.14.

Também permite passar valores arbitrários e recuperá-los através do dicionário `sys._xoptions`.

Adicionado na versão 3.2.

Alterado na versão 3.9: Removida a opção `-X showalloccount`.

Alterado na versão 3.10: Removida a opção `-X oldparser`.

Removido na versão 3.14: `-J` não é mais reservado para uso por Jython e agora não tem nenhum significado especial.

1.1.4 Controlando cores

O interpretador Python é configurado por padrão para usar cores para destacar a saída em determinadas situações, como ao exibir tracebacks. Este comportamento pode ser controlado definindo diferentes variáveis de ambiente.

Definir a variável de ambiente `TERM` como `dumb` desativará cores.

Se a variável de ambiente `FORCE_COLOR` estiver definida, a cor será habilitada independentemente do valor de TERM. Isso é útil em sistemas CI que não são terminais, mas ainda podem exibir sequências de escape ANSI.

Se a variável de ambiente `NO_COLOR` estiver definida, o Python desativará todas as cores na saída. Isto tem precedência sobre `FORCE_COLOR`.

Todas essas variáveis de ambiente também são usadas por outras ferramentas para controlar a saída de cores. Para controlar a saída de cores apenas no interpretador Python, a variável de ambiente *PYTHON_COLORS* pode ser usada. Esta variável tem precedência sobre `NO_COLOR`, que por sua vez tem precedência sobre `FORCE_COLOR`.

1.2 Variáveis de ambiente

Essas variáveis de ambiente influenciam o comportamento do Python, elas são processadas antes das opções de linha de comando diferentes de -E ou -I. É comum que as opções de linha de comando substituam as variáveis ambientais onde há um conflito.

PYTHONHOME

Altera a localização das bibliotecas Python padrão. Por padrão, as bibliotecas são pesquisadas em *prefix*/lib/pythonversion e *exec_prefix*/lib/pythonversion, onde *prefix* e *exec_prefix* são diretórios dependentes da instalação, ambos padronizando para /usr/local.

Quando *PYTHONHOME* é definido como um único diretório, seu valor substitui *prefix* e *exec_prefix*. Para especificar valores diferentes para estes, defina *PYTHONHOME* para *prefix*:*exec_prefix*.

PYTHONPATH

Aumenta o caminho de pesquisa padrão para arquivos de módulo. O formato é o mesmo PATH do shell: um ou mais caminhos de diretório separados por os.pathsep (por exemplo, dois pontos no Unix ou ponto e vírgula no Windows). Os diretórios inexistentes são ignorados silenciosamente.

Além dos diretórios normais, entradas individuais *PYTHONPATH* podem referir-se a arquivos zip contendo módulos Python puros (tanto no código-fonte quanto na forma compilada). Módulos de extensão não podem ser importados de arquivos zip.

O caminho de pesquisa padrão depende da instalação, mas geralmente começa com *prefix*/lib/pythonversion (veja *PYTHONHOME* acima). É *sempre* anexado a *PYTHONPATH*.

Um diretório adicional será inserido no caminho de pesquisa antes de *PYTHONPATH* como descrito acima em *Opções de interface*. O caminho de pesquisa pode ser manipulado de dentro de um programa Python como a variável sys.path.

PYTHONSAFEPATH

Se for definido como uma string não vazia, não anexa um caminho potencialmente inseguro para sys.path: consulte a opção *-P* para obter detalhes.

Adicionado na versão 3.11.

PYTHONPLATLIBDIR

Se for definido como uma string não vazia, ela substitui o valor sys.platlibdir.

Adicionado na versão 3.9.

PYTHONSTARTUP

Se este for o nome de um arquivo legível, os comandos Python nesse arquivo serão executados antes que o primeiro prompt seja exibido no modo interativo. O arquivo é executado no mesmo espaço de nomes onde os comandos interativos são executados para que os objetos definidos ou importados nele possam ser usados sem qualificação na sessão interativa. Você também pode alterar os prompts sys.ps1 e sys.ps2 e o gancho sys.__interactivehook__ neste arquivo.

Levanta um evento de auditoria cpython.run_startup com o nome de arquivo como argumento quando chamado na inicialização.

PYTHONOPTIMIZE

Se for definido como uma string não vazia, é equivalente a especificar a opção *-O*. Se definido como um inteiro, é equivalente a especificar *-O* várias vezes.

PYTHONBREAKPOINT

Se estiver definida, ela nomeia um chamável usando a notação de caminho com pontos. O módulo que contém o chamável será importado e então o chamável será executado pela implementação padrão de sys.breakpointhook() que é chamado pelo breakpoint() embutido. Se não for definido, ou definido como uma string vazia, é equivalente ao valor "pdb.set_trace". Definir isso para a string "0" faz com que a implementação padrão de sys.breakpointhook() não faça nada além de retornar imediatamente.

Adicionado na versão 3.7.

PYTHONDEBUG

Se for definido como uma string não vazia, é equivalente a especificar a opção *-d*. Se definido como um inteiro, é equivalente a especificar *-d* várias vezes.

Essa variável de ambiente requer uma *construção de depuração do Python*, caso contrário, será ignorada.

PYTHONINSPECT

Se for definido como uma string não vazia, é equivalente a especificar a opção *-i*.

Esta variável também pode ser modificada pelo código Python usando `os.environ` para forçar o modo de inspeção no encerramento do programa.

Levanta um evento de auditoria `cpython.run_stdin` sem argumentos.

Alterado na versão 3.12.5: (também 3.11.10, 3.10.15, 3.9.20 e 3.8.20) Emite eventos de auditoria.

Alterado na versão 3.13: Usa PyrePL, se possível, nesse caso *PYTHONSTARTUP* também é executado. Emite eventos de auditoria.

PYTHONUNBUFFERED

Se for definido como uma string não vazia, é equivalente a especificar a opção *-u*.

PYTHONVERBOSE

Se for definido como uma string não vazia, é equivalente a especificar a opção *-v*. Se definido como um inteiro, é equivalente a especificar *-v* várias vezes.

PYTHONCASEOK

Se estiver definido, Python não diferencia letras maiúsculas e minúsculas nas instruções `import`. Isso só funciona no Windows e OS X.

PYTHONDONTWRITEBYTECODE

Se for definido como uma string não vazia, o Python não tentará escrever arquivos `.pyc` na importação de módulos fonte. Isso é equivalente a especificar a opção *-B*.

PYTHONPYCACHEPREFIX

Se estiver definido, o Python escreverá os arquivos `.pyc` em uma árvore de diretório espelho neste caminho, em vez de nos diretórios `__pycache__` dentro da árvore de fontes. Isso é equivalente a especificar a opção *-X* `pycache_prefix=PATH`.

Adicionado na versão 3.8.

PYTHONHASHSEED

Se esta variável não for definida ou definida como `random`, um valor aleatório é usado para semear os hashes de objetos str e bytes.

Se *PYTHONHASHSEED* for definido como um valor inteiro, ele é usado como uma semente fixa para gerar o hash() dos tipos cobertos pela aleatorização do hash.

Sua finalidade é permitir hash repetível, como autotestes do próprio interpretador, ou permitir que um cluster de processos Python compartilhe valores de hash.

O número inteiro deve ser um número decimal no intervalo [0,4294967295]. Especificar o valor 0 desabilitará a aleatorização de hash.

Adicionado na versão 3.2.3.

PYTHONINTMAXSTRDIGITS

Se esta variável estiver definida para um inteiro, é usada para configurar a limitação de comprimento de string na conversão para inteiro global do interpretador.

Adicionado na versão 3.11.

PYTHONIOENCODING

Se for definido antes de executar o interpretador, ele substitui a codificação usada para stdin/stdout/stderr, na sintaxe `encodingname:errorhandler`. Ambas as partes `encodingname` e `:errorhandler` são opcionais e têm o mesmo significado que em `str.encode()`.

Para stderr, a parte `:errorhandler` é ignorada; o tratador sempre será `'backslashreplace'`.

Alterado na versão 3.4: A parte `encodingname` é agora opcional.

Alterado na versão 3.6: No Windows, a codificação especificada por esta variável é ignorada para buffers de console interativo, a menos que *PYTHONLEGACYWINDOWSSTDIO* também seja especificado. Arquivos e canais redirecionados por meio de fluxos padrão não são afetados.

PYTHONNOUSERSITE

Se estiver definido, o Python não adicionará o `diretório site-packages do usuário` a `sys.path`.

> **Ver também**
>
> **PEP 370** – Diretório site-packages por usuário.

PYTHONUSERBASE

Define o `diretório base do usuário`, que é usado para calcular o caminho do `diretório site-packages do usuário` e caminhos de instalação para `python -m pip install --user`.

> **Ver também**
>
> **PEP 370** – Diretório site-packages por usuário.

PYTHONEXECUTABLE

Se esta variável de ambiente for definida, `sys.argv[0]` será definido com seu valor em vez do valor obtido através do tempo de execução C. Funciona apenas no macOS.

PYTHONWARNINGS

Isso é equivalente à opção *-W*. Se definido como uma string separada por vírgulas, é equivalente a especificar *-W* várias vezes, com os filtros posteriores na lista tendo precedência sobre os anteriores na lista.

As configurações mais simples aplicam uma determinada ação incondicionalmente a todos os avisos emitidos por um processo (mesmo aqueles que são ignorados por padrão):

```
PYTHONWARNINGS=default  # Avisa uma vez por local de chamada
PYTHONWARNINGS=error    # Converte para exceções
PYTHONWARNINGS=always   # Avisa toda vez
PYTHONWARNINGS=all      # Mesmo que PYTHONWARNINGS=always
PYTHONWARNINGS=module   # Avisa uma vez por módulo chamador
PYTHONWARNINGS=once     # Avisa uma vez por processo do Python
PYTHONWARNINGS=ignore   # Nunca avisar
```

Veja warning-filter e describing-warning-filters para mais detalhes.

PYTHONFAULTHANDLER

Se esta variável de ambiente for definida como uma string não vazia, `faulthandler.enable()` é chamado na inicialização: instale um tratador para os sinais `SIGSEGV`, `SIGFPE`, `SIGABRT`, `SIGBUS` e `SIGILL` para despejar o traceback (situação da pilha de execução) do Python. Isso é equivalente à opção *-X* `faulthandler`.

Adicionado na versão 3.3.

PYTHONTRACEMALLOC

Se esta variável de ambiente for definida como uma string não vazia, começa a rastrear as alocações de memória Python usando o módulo `tracemalloc`. O valor da variável é o número máximo de quadros armazenados em um traceback de um rastreamento. Por exemplo, `PYTHONTRACEMALLOC=1` armazena apenas o quadro mais recente. Veja a função `tracemalloc.start()` para mais informações. Isso é o equivalente a definir a opção *-X* `tracemalloc`.

Adicionado na versão 3.4.

PYTHONPROFILEIMPORTTIME

Se esta variável de ambiente for definida com `1`, o Python mostrará quanto tempo leva cada importação. Se

definido com `2`, o Python incluirá a saída para módulos importados que já foram carregados. Isso equivale a definir a opção *`-X importtime`*.

Adicionado na versão 3.7.

Alterado na versão 3.14: Adicionado `PYTHONPROFILEIMPORTTIME=2` para também rastrear importações de módulos carregados.

PYTHONASYNCIODEBUG

Se esta variável de ambiente for definida como uma string não vazia, habilita o modo de depuração do módulo `asyncio`.

Adicionado na versão 3.4.

PYTHONMALLOC

Define os alocadores de memória Python e/ou instale ganchos de depuração.

Define a família de alocadores de memória usados pelo Python:

- `default`: usa os alocadores padrão de memória.
- `malloc`: usa a função `malloc()` da biblioteca C para todos os domínios (`PYMEM_DOMAIN_RAW`, `PYMEM_DOMAIN_MEM`, `PYMEM_DOMAIN_OBJ`).
- `pymalloc`: usa o alocador pymalloc para domínios `PYMEM_DOMAIN_MEM` e `PYMEM_DOMAIN_OBJ` e usa a função `malloc()` para o domínio `PYMEM_DOMAIN_RAW`.
- `mimalloc`: usa o alocador mimalloc para domínios `PYMEM_DOMAIN_MEM` e `PYMEM_DOMAIN_OBJ` e usa a função `malloc()` para o domínio `PYMEM_DOMAIN_RAW`.

Instala ganchos de depuração:

- `debug`: instala os ganchos de depuração sobre os alocadores padrão de memória.
- `malloc_debug`: o mesmo que `malloc`, mas também instala ganchos de depuração.
- `pymalloc_debug`: o mesmo que `pymalloc`, mas também instala ganchos de depuração.
- `mimalloc_debug`: o mesmo que `mimalloc`, mas também instala ganchos de depuração.

Adicionado na versão 3.6.

Alterado na versão 3.7: Adicionado o alocador `"default"`.

PYTHONMALLOCSTATS

Se definido como uma string não vazia, o Python exibe estatísticas do alocador de memória pymalloc toda vez que uma nova arena de objeto pymalloc for criada e ao no desligamento.

Esta variável é ignorada se a variável de ambiente *`PYTHONMALLOC`* é usada para forçar o alocador `malloc()` da biblioteca C, ou se Python está configurado sem suporte a `pymalloc`.

Alterado na versão 3.6: Esta variável agora também pode ser usada em Python compilado no modo de lançamento. Agora não tem efeito se definido como uma string vazia.

PYTHONLEGACYWINDOWSFSENCODING

Se definido como uma string não vazia, o modo padrão do *tratador de erros e codificação do sistema de arquivos* irá reverter para seus valores pré-3.6 de “mbcs” e “replace”, respectivamente. Caso contrário, os novos padrões “utf-8” e “surrogatepass” serão usados.

Isso também pode ser habilitado em tempo de execução com `sys._enablelegacywindowsfsencoding()`.

Disponibilidade: Windows.

Adicionado na versão 3.6: Veja **PEP 529** para mais detalhes.

PYTHONLEGACYWINDOWSSTDIO

Se definido como uma string não vazia, não usa o novo leitor e escritor de console. Isso significa que os caracteres Unicode serão codificados de acordo com a página de código do console ativo, em vez de usar utf-8.

Esta variável é ignorada se os fluxos padrão forem redirecionados (para arquivos ou canais) em vez de se referir aos buffers do console.

Disponibilidade: Windows.

Adicionado na versão 3.6.

PYTHONCOERCECLOCALE

Se definido com o valor `0`, faz com que a aplicação principal de linha de comando Python ignore a coerção dos códigos de idioma legados C e POSIX baseados em ASCII para uma alternativa baseada em UTF-8 mais capaz.

Se esta variável *não* estiver definida (ou estiver definida para um valor diferente de `0`), a variável de ambiente de substituição de localidade `LC_ALL` também não será definida, e a localidade atual relatada para a categoria `LC_CTYPE` é a localidade `C` padrão, ou então a localidade `POSIX` explicitamente baseada em ASCII, então a CLI do Python tentará configurar as seguintes localidades para a categoria `LC_CTYPE` na ordem listada antes de carregar o tempo de execução do interpretador:

- `C.UTF-8`
- `C.utf8`
- `UTF-8`

Se a configuração de uma dessas categorias de local for bem-sucedida, a variável de ambiente `LC_CTYPE` também será configurada de acordo no ambiente de processo atual antes que o tempo de execução do Python seja inicializado. Isso garante que, além de ser visto pelo próprio interpretador e outros componentes com reconhecimento de localidade em execução no mesmo processo (como a biblioteca GNU `readline`), a configuração atualizada também é vista em subprocessos (independentemente de ou não esses processos estão executando um interpretador Python), bem como em operações que consultam o ambiente em vez da localidade C atual (como o `locale.getdefaultlocale()` do próprio Python).

Configurar uma dessas localidades (explicitamente ou por meio da coerção de localidade implícita acima) habilita automaticamente o tratador de erros `surrogateescape` para `sys.stdin` e `sys.stdout` (`sys.stderr` continua a usar `backslashreplace` como faz em qualquer outra localidade). Este comportamento de tratamento de fluxo pode ser substituído usando *`PYTHONIOENCODING`* como de costume.

Para fins de depuração, definir `PYTHONCOERCECLOCALE=warn` fará com que o Python emita mensagens de aviso em `stderr` se a coerção de localidade for ativada ou se uma localidade que *teria* acionado a coerção ainda estiver ativa quando o Python o tempo de execução é inicializado.

Observe também que mesmo quando a coerção de localidade está desabilitada, ou quando não consegue encontrar uma localidade de destino adequada, *`PYTHONUTF8`* ainda será ativado por padrão em localidades baseadas em ASCII legadas. Ambos os recursos devem ser desabilitados para forçar o interpretador a usar `ASCII` ao invés de `UTF-8` para interfaces de sistema.

Disponibilidade: Unix.

Adicionado na versão 3.7: Veja **PEP 538** para mais detalhes.

PYTHONDEVMODE

Se esta variável de ambiente for definida como uma string não vazia, habilita Modo de Desenvolvimento do Python, introduzindo verificações adicionais de tempo de execução que são muito caras para serem habilitadas por padrão. Isso é o equivalente a definir a opção *`-X`* `dev`.

Adicionado na versão 3.7.

PYTHONUTF8

Se definido para `1`, habilita o modo UTF-8 do Python.

Se definido para `0`, desabilita o modo UTF-8 do Python.

Definir qualquer outra string não vazia causa um erro durante a inicialização do interpretador.

Adicionado na versão 3.7.

PYTHONWARNDEFAULTENCODING

Se esta variável de ambiente for definida como uma string não vazia, emite uma `EncodingWarning` quando a codificação padrão específica da localidade é usada.

Veja io-encoding-warning para detalhes.

Adicionado na versão 3.10.

PYTHONNODEBUGRANGES

Se esta variável estiver definida, ela desabilita a inclusão das tabelas que mapeiam informações de localização extra (linha final, deslocamento da coluna inicial e deslocamento da coluna final) para cada instrução em objetos código. Isso é útil quando objetos código menores e arquivos pyc são desejados, bem como suprimir os indicadores de localização visual extra quando o interpretador exibe tracebacks.

Adicionado na versão 3.11.

PYTHONPERFSUPPORT

Se essa variável for definida como valor diferente de zero, ela habilitará suporte para o perfilador do Linux `perf` para que as chamadas Python possam ser detectadas por ele.

Se definido como `0`, desativa o suporte ao perfilador do Linux `perf`.

Consulte também a opção de linha de comando *-X perf* e perf_profiling.

Adicionado na versão 3.12.

PYTHON_PERF_JIT_SUPPORT

Se essa variável for definida como valor diferente de zero, ela habilitará suporte para o perfilador do Linux `perf` para que as chamadas Python possam ser detectadas por ele usando informações de DWARF.

Se definido como `0`, desativa o suporte ao perfilador do Linux `perf`.

Veja também a opção de linha de comando *-X perf_jit* e perf_profiling.

Adicionado na versão 3.13.

PYTHON_DISABLE_REMOTE_DEBUG

Se esta variável for definida como uma string não vazia, ela desabilitará o recurso de depuração remota descrito na **PEP 768**. Isso inclui tanto a funcionalidade de agendar código para execução em outro processo quanto a funcionalidade de receber código para execução no processo atual.

Veja também a opção de linha de comando *-X disable_remote_debug*.

Adicionado na versão 3.14.

PYTHON_CPU_COUNT

Se esta variável for definida como um número inteiro positivo, ela substitui os valores de retorno de `os.cpu_count()` e `os.process_cpu_count()`.

Veja também a opção de linha de comando *-X cpu_count*.

Adicionado na versão 3.13.

PYTHON_FROZEN_MODULES

Se esta variável for definida como `on` ou `off`, ela determina se os módulos congelados serão ou não ignorados pela maquinário de importação. Um valor `on` significa que eles serão importados e `off` significa que serão ignorados. O padrão é `on` para construções sem depuração (o caso normal) e `off` para construções com depuração. Observe que os módulos congelados `importlib_bootstrap` e `importlib_bootstrap_external` são sempre usados, mesmo se este sinalizador estiver definido como `off`.

Veja também a opção de linha de comando *-X frozen_modules*.

Adicionado na versão 3.13.

PYTHON_COLORS

Se esta variável for definida como `1`, o interpretador irá colorir vários tipos de saída. Definir como `0` desativa este comportamento. Veja também *Controlando cores*.

Adicionado na versão 3.13.

PYTHON_BASIC_REPL

Se esta variável for definida como qualquer valor, o interpretador não tentará carregar o *REPL* baseado em Python que requer `curses` e `readline`, e em vez disso usará o *REPL* com o analisador sintático tradicional.

Adicionado na versão 3.13.

PYTHON_HISTORY

Esta variável de ambiente pode ser usada para definir a localização de um arquivo `.python_history` (por padrão, é `.python_history` no diretório inicial do usuário).

Adicionado na versão 3.13.

PYTHON_GIL

Se esta variável for definida como `1`, a trava global do interpretador (GIL) será forçada. Definir como `0` força a GIL a desligar (precisa do Python configurado com a opção de construção *--disable-gil*).

Veja também a opção de linha de comando *-X gil*, que tem precedência sobre esta variável, e whatsnew313--free-threaded-cpython.

Adicionado na versão 3.13.

PYTHON_THREAD_INHERIT_CONTEXT

Se esta variável for definida como `1`, `Thread` usará, por padrão, uma cópia do contexto do chamador de `Thread.start()` ao iniciar. Caso contrário, novas threads serão iniciadas com um contexto vazio. Se não definida, esta variável assume como padrão `1` em construções com threads livres e como `0` em outras. Veja também *-X thread_inherit_context*.

Adicionado na versão 3.14.

PYTHON_CONTEXT_AWARE_WARNINGS

Se definido como `1`, o gerenciador de contexto `warnings.catch_warnings` usará uma `ContextVar` para armazenar o estado do filtro de avisos. Se não definido, o padrão desta variável será `1` em construções com threads livres e `0` em outras. Consulte *-X context_aware_warnings*.

Adicionado na versão 3.14.

PYTHON_JIT

Em construções onde a compilação experimental just-in-time está disponível, esta variável pode forçar o JIT a ser desabilitado (`0`) ou habilitado (`1`) na inicialização do interpretador.

Adicionado na versão 3.13.

PYTHON_TLBC

Se definido como `1`, habilita o bytecode local da thread. Se definido como `0`, o bytecode local da thread e o interpretador especializado são desabilitados. Aplica-se somente a construções configuradas com *--disable-gil*.

Veja também a opção de linha de comando *-X tlbc*.

Adicionado na versão 3.14.

1.2.1 Variáveis de modo de depuração

PYTHONDUMPREFS

Se definido, Python irá despejar objetos e contagens de referências ainda vivas após desligar o interpretador.

Precisa do Python configurado com a opção de construção *--with-trace-refs*.

PYTHONDUMPREFSFILE

Se definido, o Python irá despejar objetos e contagens de referências ainda ativas após desligar o interpretador em um arquivo no caminho fornecido como valor para esta variável de ambiente.

Precisa do Python configurado com a opção de construção *--with-trace-refs*.

Adicionado na versão 3.11.

PYTHON_PRESITE

Se esta variável for definida como um módulo, esse módulo será importado no início do ciclo de vida do interpretador, antes do módulo `site` ser executado, e antes do módulo __main__ ser criado. Portanto, o módulo importado não é tratado como __main__.

Isso pode ser usado para executar código antecipadamente durante a inicialização do Python.

Para importar um submódulo, use `pacote.módulo` como valor, como em uma instrução de importação.

Veja também a opção de linha de comando *-X presite*, que tem precedência sobre esta variável.

Precisa do Python configurado com a opção de construção *--with-pydebug*.

Adicionado na versão 3.13.

CAPÍTULO 2

Utilizando Python em plataformas Unix

2.1 Obtendo e instalando a versão mais recente do Python

2.1.1 No Linux

O Python vem pré-instalado na maioria das distribuições Linux e está disponível como um pacote em todas as outras. No entanto, há certos recursos que você pode querer usar que não estão disponíveis no pacote da sua distribuição. Você pode compilar a versão mais recente do Python a partir do código-fonte.

No caso de a versão mais recente do Python não vir pré-instalada e não estar nos repositórios também, você pode criar pacotes para sua própria distro. Dê uma olhada nos seguintes links:

Ver também

https://www.debian.org/doc/manuals/maint-guide/first.en.html
para usuários Debian

https://en.opensuse.org/Portal:Packaging
para usuários OpenSuse

https://docs.fedoraproject.org/en-US/package-maintainers/Packaging_Tutorial_GNU_Hello/
para usuários Fedora

https://slackbook.org/html/package-management-making-packages.html
para usuários do Slackware

Instalação do IDLE

Em alguns casos, o IDLE pode não estar incluído em sua instalação do Python.

- Para usuários do Debian e do Ubuntu:

```
sudo apt update
sudo apt install idle
```

- Para usuários do Fedora, RHEL e CentOS:

```
sudo dnf install python3-idle
```

- Para usuários do SUSE e do OpenSUSE:

```
sudo zypper install python3-idle
```

- Para usuários do Alpine Linux:

```
sudo apk add python3-idle
```

2.1.2 No FreeBSD e OpenBSD

- usuários do FreeBSD, para adicionar a utilização do pacote utilize:

```
pkg install python3
```

- Usuários do OpenBSD, para adicionar pacotes use:

```
pkg_add -r python

pkg_add ftp://ftp.openbsd.org/pub/OpenBSD/4.2/packages/<insert your␣
↪architecture here>/python-<version>.tgz
```

Por exemplo, usuários i386 podem pegar a versão 2.5.1 do Python usando o comando:

```
pkg_add ftp://ftp.openbsd.org/pub/OpenBSD/4.2/packages/i386/python-2.5.1p2.tgz
```

2.2 Compilando o Python

Se você quer compilar o CPython, a primeira coisa que você precisa fazer é baixar o código-fonte. Você pode baixar a última versão ou usar o git para fazer um clone. (Se você pretende contribuir modificações, você vai precisar um "clone".)

O processo de compilação consiste nos comandos usuais:

```
./configure
make
make install
```

Opções de configuração e advertências para plataformas específicas do Unix estão amplamente documentadas no arquivo README.rst na raiz da árvore de fontes Python.

Aviso

`make install` pode sobrescrever ou mascarar o arquivo binário `python3`. `make altinstall` é, portanto, recomendado ao invés de `make install` uma vez que o mesmo apenas instala o arquivo *`exec_prefix`*`/bin/python`*`version`*.

2.3 Paths e arquivos relacionados com o Python

Estes estão sujeitos a diferenças dependendo das convenções de instalação local; *`prefix`* e *`exec_prefix`* dependem da instalação e devem ser interpretados da mesma forma que para o software GNU; eles poderão ser os mesmos.

Por exemplo, na maioria dos sistemas Linux, o padrão para ambos é `/usr`.

Arquivo/diretório	Significado
exec_prefix/bin/python3	Localização recomendada do interpretador.
prefix/lib/python*version*, *exec_prefix*/lib/python*version*	A localização recomendada dos diretórios contendo os módulos padrão.
prefix/include/python*version*, *exec_prefix*/include/python*version*	Localizações recomendadas dos diretórios contendo os arquivos de inclusão necessários para o desenvolvimento de extensões Python e incorporação do interpretador.

2.4 Diversos

Para usar facilmente scripts Python no Unix, você precisa torná-los executáveis, por exemplo, com

```
$ chmod +x script
```

e colocar uma linha Shebang apropriada no topo do script. Uma boa escolha normalmente é

```
#!/usr/bin/env python3
```

que procura o interpretador do Python no conjunto `PATH`. No entanto, alguns sistemas Unix podem não ter o comando **env**, então você pode precisar codificar `/usr/bin/python3` como o caminho do interpretador.

Para usar comandos Shell em seus scripts Python, veja o módulo `subprocess`.

2.5 OpenSSL personalizado

1. Para usar a configuração OpenSSL do seu fornecedor e armazenamento confiável do sistema, localize o diretório com o arquivo `openssl.cnf` ou link simbólico em `/etc`. Na maioria das distribuições o arquivo está em `/etc/ssl` ou em `/etc/pki/tls`. O diretório também deve conter um arquivo `cert.pem` e/ou um diretório `certs`.

   ```
   $ find /etc/ -name openssl.cnf -printf "%h\n"
   /etc/ssl
   ```

2. Baixe, construa e instale o OpenSSL. Certifique-se de usar `install_sw` e não `install`. O destino `install_sw` não substitui o `openssl.cnf`.

   ```
   $ curl -O https://www.openssl.org/source/openssl-VERSÃO.tar.gz
   $ tar xzf openssl-VERSÃO
   $ pushd openssl-VERSÃO
   $ ./config \
       --prefix=/usr/local/custom-openssl \
       --libdir=lib \
       --openssldir=/etc/ssl
   $ make -j1 depend
   $ make -j8
   $ make install_sw
   $ popd
   ```

3. Construa o Python com o OpenSSL personalizado (veja as opções configure `--with-openssl` e `--with-openssl-rpath`)

   ```
   $ pushd python-3.x.x
   $ ./configure -C \
       --with-openssl=/usr/local/custom-openssl \
   ```

(continua na próxima página)

(continuação da página anterior)

```
    --with-openssl-rpath=auto \
    --prefix=/usr/local/python-3.x.x
$ make -j8
$ make altinstall
```

Nota

As versões de patches do OpenSSL têm uma ABI compatível com versões anteriores. Você não precisa recompilar o Python para atualizar o OpenSSL. É suficiente substituir a instalação personalizada do OpenSSL por uma versão mais recente.

CAPÍTULO 3

Configurando o Python

3.1 Requisitos de construção

Recursos e versões mínimas necessários para construir o CPython:

- Um compilador C11. Não são necessários recursos opcionais do C11.
- No Windows, é necessário o Microsoft Visual Studio 2017 ou posterior.
- Suporte para números de ponto flutuante do IEEE 754 e Not-a-Number (NaN) de ponto flutuante.
- Suporte a threads.
- OpenSSL 1.1.1 é a versão mínima e OpenSSL 3.0.16 é a versão mínima recomendada para os módulos de extensão `ssl` e `hashlib`.
- SQLite 3.15.2 para o módulo de extensão `sqlite3`.
- Tcl/Tk 8.5.12 para o módulo `tkinter`.
- libmpdec 2.5.0 para o módulo `decimal`.
- Autoconf 2.72 e aclocal 1.16.5 são necessários para regenerar o script `configure`.

Alterado na versão 3.1: Agora é necessária a versão 8.3.1 do Tcl/Tk.

Alterado na versão 3.5: No Windows, agora é necessário o Visual Studio 2015 ou posterior. Agora é necessária a versão 8.4 do Tcl/Tk.

Alterado na versão 3.6: Recursos selecionados do C99 agora são necessários, como funções `<stdint.h>` e `static inline`.

Alterado na versão 3.7: Suporte a threads e OpenSSL 1.0.2 agora são necessários.

Alterado na versão 3.10: OpenSSL 1.1.1 agora é necessário. Requer SQLite 3.7.15.

Alterado na versão 3.11: Compilador C11, suporte a IEEE 754 e NaN agora são necessários. No Windows, é necessário o Visual Studio 2017 ou posterior. A versão 8.5.12 do Tcl/Tk agora é necessária para o módulo `tkinter`.

Alterado na versão 3.13: Autoconf 2.71, aclocal 1.16.5 e SQLite 3.15.2 agora são necessários.

Alterado na versão 3.14: Autoconf 2.72 agora é necessário.

Veja também **PEP 7** “Guia de estilo para código C” e **PEP 11** “Suporte do CPython a plataformas”.

3.2 Arquivos gerados

Para reduzir as dependências de construção, o código-fonte do Python contém vários arquivos gerados. Comandos para regenerar todos os arquivos gerados:

```
make regen-all
make regen-stdlib-module-names
make regen-limited-abi
make regen-configure
```

O arquivo `Makefile.pre.in` documenta os arquivos gerados, suas entradas e ferramentas usadas para regenerá-los. Procure por alvos `regen-*` de make.

3.2.1 Script configure

O comando `make regen-configure` regera o arquivo `aclocal.m4` e o script `configure` usando o shell script `Tools/build/regen-configure.sh`, o qual usa um contêiner Ubuntu para obter as mesmas versões de ferramentas e ter uma saída reproduzível.

O contêiner é opcional, o seguinte comando pode ser executado localmente:

```
autoreconf -ivf -Werror
```

Os arquivos gerados podem mudar dependendo das versões exatas do `autoconf-archive`, `aclocal` e `pkg-config`.

3.3 Opções de configuração

Liste todas as opções do `configure` usando:

```
./configure --help
```

Veja também o `Misc/SpecialBuilds.txt` na distribuição de código-fonte do Python.

3.3.1 Opções gerais

--enable-loadable-sqlite-extensions

Suporte a extensões carregáveis no módulo de extensão `_sqlite` (o padrão é não) do módulo `sqlite3`.

Veja o método `sqlite3.Connection.enable_load_extension()` do módulo `sqlite3`.

Adicionado na versão 3.6.

--disable-ipv6

Desabilita suporte a IPv6 (habilitado por padrão se houver suporte), veja o módulo `socket`.

--enable-big-digits=`[15|30]`

Define o tamanho em bits dos dígitos de `int` do Python: 15 ou 30 bits.

Por padrão, o tamanho dos dígitos é 30.

Define o `PYLONG_BITS_IN_DIGIT` para `15` ou `30`.

Veja `sys.int_info.bits_per_digit`.

--with-suffix=`SUFFIX`

Define o sufixo do executável do Python para *SUFFIX*.

O sufixo padrão é `.exe` no Windows e macOS (executável `python.exe`), `.js` em nó Emscripten, `.html` em navegador Emscripten, `.wasm` em WASI e uma string vazia em outras plataformas (executável `python`).

Alterado na versão 3.11: O sufixo padrão na plataforma WASM é um entre `.js`, `.html` ou `.wasm`

--with-tzpath=<list of absolute paths separated by pathsep>

Seleciona o caminho de pesquisa de fuso horário padrão para `zoneinfo.TZPATH`. Veja a Configuração de tempo de compilação do módulo `zoneinfo`.

Padrão: `/usr/share/zoneinfo:/usr/lib/zoneinfo:/usr/share/lib/zoneinfo:/etc/zoneinfo`.

Veja o separador de caminhos `os.pathsep`.

Adicionado na versão 3.9.

--without-decimal-contextvar

Constrói o módulo de extensão `_decimal` usando um contexto local de thread ao invés de um contexto local de corrotina (padrão), veja o módulo `decimal`.

Veja `decimal.HAVE_CONTEXTVAR` e o módulo `contextvars`.

Adicionado na versão 3.9.

--with-dbmliborder=<list of backend names>

Substitui a ordem de verificação de backends de banco de dados para o módulo `dbm`

Um valor válido é uma string separada por dois pontos (`:`) com os nomes de backend:

- `ndbm`;
- `gdbm`;
- `bdb`.

--without-c-locale-coercion

Desabilita a coerção de localidade C para uma localidade baseada em UTF-8 (ativada por padrão).

Não define a macro `PY_COERCE_C_LOCALE`.

Consulte *`PYTHONCOERCECLOCALE`* e a **PEP 538**.

--with-platlibdir=DIRNAME

Nome do diretório da biblioteca Python (o padrão é `lib`).

Fedora e SuSE usam `lib64` em plataformas de 64 bits.

Veja `sys.platlibdir`.

Adicionado na versão 3.9.

--with-wheel-pkg-dir=PATH

Diretório de pacotes de wheel usados pelo módulo `ensurepip` (nenhum por padrão).

Algumas políticas de empacotamento de distribuição do Linux recomendam contra o empacotamento de dependências. Por exemplo, o Fedora instala pacotes wheel no diretório `/usr/share/python-wheels/` e não instala o pacote `ensurepip._bundled`.

Adicionado na versão 3.10.

--with-pkg-config=[check|yes|no]

Se o configure deve usar **pkg-config** para detectar dependências de construção.

- `check` (padrão): **pkg-config** é opcional
- `yes`: **pkg-config** é obrigatório
- `no`: configure não usa **pkg-config** mesmo quando presente

Adicionado na versão 3.11.

--enable-pystats

Ativa a coleta de estatísticas internas de desempenho do Python.

Por padrão, a coleta de estatísticas está desativada. Use o comando `python3 -X pystats` ou defina a variável de ambiente `PYTHONSTATS=1` para ativar a coleta de estatísticas na inicialização do Python.

Na saída do Python, despeja as estatísticas se a coleta de estatísticas estiver ativada e não apagada.

Efeitos:

- Adiciona a opção de linha de comando *`-X pystats`*.
- Adiciona a variável de ambiente `PYTHONSTATS`.
- Define a macro `Py_STATS`.
- Adiciona funções ao módulo `sys`:
  - `sys._stats_on()`: Ativa a coleta de estatísticas.
  - `sys._stats_off()`: Desativa a coleta de estatísticas.
  - `sys._stats_clear()`: Apaga as estatísticas.
  - `sys._stats_dump()`: Despeja as estatísticas no arquivo e apaga as estatísticas.

As estatísticas serão despejadas em um arquivo arbitrário (provavelmente único) em `/tmp/py_stats/` (Unix) ou `C:\temp\py_stats\` (Windows). Se aquele diretório não existir, os resultados serão enviados para stderr.

Use `Tools/scripts/summarize_stats.py` para ler as estatísticas.

Estatísticas:

- Código de operação:
  - Especialização: sucesso, fracasso, acerto, adiado, erro, deopt, falhas;
  - Contagem de execuções;
  - Contagem em pares.
- Chamada:
  - Chamadas Python em linha;
  - Chamadas a PyEval;
  - Quadros enviados;
  - Objetos quadro criados;
  - Chamadas de Eval: vetor, gerador, legado, função VECTORCALL, classe de construção, slot, função "ex", API, método.
- Objeto:
  - incref e decref;
  - incref e decref do interpretador;
  - alocações: tudo, 512 bytes, 4 kiB, big;
  - memória livre;
  - listas de memória livre para/de;
  - Dicionário materializado/desmaterializado;
  - cache de tipo;
  - tentativas de otimização;
  - rastros de otimização criados/executados;
  - uops executados.

- Coletor de lixo:
  - Coletas de lixo;
  - Objetos visitados;
  - Objetos coletados.

Adicionado na versão 3.11.

--disable-gil

Habilita suporte para execução de Python sem a *trava global do interpretador* (GIL): construção com threads livres.

Define a macro `Py_GIL_DISABLED` e adiciona `"t"` a `sys.abiflags`.

Veja whatsnew313-free-threaded-cpython para mais detalhes.

Adicionado na versão 3.13.

--enable-experimental-jit=[no|yes|yes-off|interpreter]

Indica como integrar o compilador just-in-time experimental.

- `no`: não constrói o JIT;
- `yes`: habilita o JIT. Para desabilitá-lo em tempo de execução, defina a variável de ambiente *PYTHON_JIT=0*.
- `yes-off`: constrói o JIT, mas desabilita-o por padrão. Para habilitá-lo em tempo de execução, defina a variável de ambiente *PYTHON_JIT=1*.
- `interpreter`: Habilita o "interpretador JIT" (útil apenas para quem está depurando o próprio JIT). Para desabilitá-lo em tempo de execução, defina a variável de ambiente *PYTHON_JIT=0*.

`--enable-experimental-jit=no` é o comportamento padrão se a opção não for fornecida, e `--enable-experimental-jit` é uma abreviação de `--enable-experimental-jit=yes`. Consulte `Tools/jit/README.md` para mais informações, incluindo como instalar as dependências necessárias em tempo de construção.

> **Nota**
>
> Ao construir o CPython com o JIT habilitado, certifique-se de que seu sistema tenha o Python 3.11 ou posterior instalado.

Adicionado na versão 3.13.

PKG_CONFIG

Caminho para o utilitário `pkg-config`.

PKG_CONFIG_LIBDIR

PKG_CONFIG_PATH

Opções do `pkg-config`.

3.3.2 Opções do compilador C

CC

Comando do compilador C.

CFLAGS

Sinalizadores do compilador C.

CPP

Comando do pré-processador C.

CPPFLAGS
Sinalizadores do pré-processador C, p.ex., `-Iinclude_dir`.

3.3.3 Opções da ligação

LDFLAGS
Sinalizadores do vinculador. p.ex., `-Llibrary_directory`.

LIBS
Bibliotecas para passar para o vinculador, p.ex. `-llibrary`.

MACHDEP
Nome para arquivos de biblioteca dependentes de máquina.

3.3.4 Opções para dependências de terceiros

Adicionado na versão 3.11.

BZIP2_CFLAGS

BZIP2_LIBS
Sinalizadores de compilador C e vinculador para vincular Python a `libbz2`, usados pelo módulo `bz2`, substituindo `pkg-config`.

CURSES_CFLAGS

CURSES_LIBS
Sinalizadores de compilador C e vinculador para `libncurses` ou `libncursesw`, usados pelo módulo `curses`, substituindo `pkg-config`.

GDBM_CFLAGS

GDBM_LIBS
Sinalizadores de compilador C e vinculador para `gdbm`.

LIBB2_CFLAGS

LIBB2_LIBS
Sinalizadores de compilador C e vinculador para `libb2` (BLAKE2), usados pelo módulo `hashlib`, substituindo `pkg-config`.

LIBEDIT_CFLAGS

LIBEDIT_LIBS
Sinalizadores de compilador C e vinculador para `libedit`, usados pelo módulo `readline`, substituindo `pkg-config`.

LIBFFI_CFLAGS

LIBFFI_LIBS
Sinalizadores de compilador C e vinculador para `libffi`, usados pelo módulo `ctypes`, substituindo `pkg-config`.

LIBMPDEC_CFLAGS

LIBMPDEC_LIBS
Sinalizadores de compilador C e vinculador para `libmpdec`, usados pelo módulo `decimal`, substituindo `pkg-config`.

> **Nota**
>
> Estas variáveis de ambiente não têm efeito a menos que `--with-system-libmpdec` seja especificado.

LIBLZMA_CFLAGS

LIBLZMA_LIBS

Sinalizadores de compilador C e vinculador para `liblzma`, usados pelo módulo `lzma`, substituindo `pkg-config`.

LIBREADLINE_CFLAGS

LIBREADLINE_LIBS

Sinalizadores de compilador C e vinculador para `libreadline`, usados pelo módulo `readline`, substituindo `pkg-config`.

LIBSQLITE3_CFLAGS

LIBSQLITE3_LIBS

Sinalizadores de compilador C e vinculador para `libsqlite3`, usados pelo módulo `sqlite3`, substituindo `pkg-config`.

LIBUUID_CFLAGS

LIBUUID_LIBS

Sinalizadores de compilador C e vinculador para `libuuid`, usados pelo módulo `uuid`, substituindo `pkg-config`.

LIBZSTD_CFLAGS

LIBZSTD_LIBS

Sinalizadores de compilador C e ligador para `libzstd`, usados pelo módulo `compression.zstd`, substituindo `pkg-config`.

Adicionado na versão 3.14.

PANEL_CFLAGS

PANEL_LIBS

Sinalizadores de compilador C e vinculador para PANEL, substituindo `pkg-config`.

Sinalizadores de compilador C e vinculador para `libpanel` ou `libpanelw`, usados pelo módulo `curses.panel`, substituindo `pkg-config`.

TCLTK_CFLAGS

TCLTK_LIBS

Sinalizadores de compilador C e vinculador para TCLTK, substituindo `pkg-config`.;

ZLIB_CFLAGS

ZLIB_LIBS

Sinalizadores de compilador C e vinculador para `libzlib`, usados pelo módulo `gzip`, substituindo `pkg-config`.

3.3.5 Opções de WebAssembly

--enable-wasm-dynamic-linking

Ativa o suporte de vinculação dinâmica para WASM.

A vinculação dinâmica permite `dlopen`. O tamanho do arquivo executável aumenta devido à eliminação limitada de código morto e recursos adicionais.

Adicionado na versão 3.11.

--enable-wasm-pthreads

Ativa o suporte a pthreads para WASM.

Adicionado na versão 3.11.

3.3.6 Opções de instalação

--prefix=PREFIX

Instala arquivos independentes de arquitetura em PREFIX. No Unix, o padrão é `/usr/local`.

Este valor pode ser recuperado em tempo de execução usando `sys.prefix`.

Como exemplo, pode-se usar `--prefix="$HOME/.local/"` para instalar um Python em seu diretório pessoal (home).

--exec-prefix=EPREFIX

Instala arquivos dependentes de arquitetura no EPREFIX, o padrão é *--prefix*.

Este valor pode ser recuperado em tempo de execução usando `sys.exec_prefix`.

--disable-test-modules

Não constrói nem instala módulos de teste, como o pacote `test` ou o módulo de extensão `_testcapi` (construído e instalado por padrão).

Adicionado na versão 3.10.

--with-ensurepip=[upgrade|install|no]

Seleciona o comando `ensurepip` executado na instalação do Python:

- `upgrade` (padrão): executa o comando `python -m ensurepip --altinstall --upgrade`.
- `install`: executa o comando `python -m ensurepip --altinstall`;
- `no`: não executa ensurepip;

Adicionado na versão 3.6.

3.3.7 Opções de desempenho

Configurar o Python usando `--enable-optimizations --with-lto` (PGO + LTO) é o recomendado para melhor desempenho. O sinalizador experimental `--enable-bolt` também pode ser usado para melhorar o desempenho.

--enable-optimizations

Habilita a otimização guiada por perfil (PGO, do inglês Profile Guided Optimization) usando *PROFILE_TASK* (desabilitado por padrão).

O compilador C Clang requer o programa `llvm-profdata` para PGO. No macOS, o GCC também exige: o GCC é apenas um apelido para o Clang no macOS.

Desabilita também a interposição semântica no libpython se `--enable-shared` e GCC forem usados: adiciona `-fno-semantic-interposition` aos sinalizadores do compilador e do vinculador.

> **Nota**
>
> Durante a construção, você poderá encontrar avisos do compilador sobre a indisponibilidade de dados de perfil para alguns arquivos fonte. Esses avisos são inofensivos, pois apenas um subconjunto do código é exercido durante a aquisição de dados de perfil. Para desativar esses avisos no Clang, suprima-os manualmente adicionando `-Wno-profile-instr-unprofiled` a *CFLAGS*.

Adicionado na versão 3.6.

Alterado na versão 3.10: Usa `-fno-semantic-interposition` no GCC.

PROFILE_TASK

Variável de ambiente usada no Makefile: argumentos de linha de comando do Python para a tarefa de geração de PGO.

Padrão: `-m test --pgo --timeout=$(TESTTIMEOUT)`.

Adicionado na versão 3.8.

Alterado na versão 3.13: A falha da tarefa não é mais ignorada silenciosamente.

--with-lto=[full|thin|no|yes]

Habilita o otimização em tempo de vinculação (LTO, do inglês Link Time Optimization) em qualquer construção (desabilitado por padrão).

O compilador C Clang requer `llvm-ar` para LTO (`ar` no macOS), bem como um vinculador compatível com LTO (`ld.gold` ou `lld`).

Adicionado na versão 3.6.

Adicionado na versão 3.11: Para usar o recurso ThinLTO, use `--with-lto=thin` no Clang.

Alterado na versão 3.12: Usa ThinLTO como política de otimização padrão no Clang se o compilador aceitar o sinalizador.

--enable-bolt

Habilita o uso do otimizador binário pós-vinculação BOLT (desabilitado por padrão).

BOLT faz parte do projeto LLVM, mas nem sempre está incluído em suas distribuições binárias. Este sinalizador requer que `llvm-bolt` e `merge-fdata` estejam disponíveis.

BOLT ainda é um projeto relativamente novo, então este sinalizador deve ser considerado experimental por enquanto. Como esta ferramenta opera em código de máquina, seu sucesso depende de uma combinação do ambiente de construção + os outros argumentos de configuração de otimização + a arquitetura da CPU, e nem todas as combinações são suportadas. Versões do BOLT anteriores ao LLVM 16 são conhecidas por travar o BOLT em alguns cenários. O uso do LLVM 16 ou mais recente para otimização do BOLT é fortemente incentivado.

As variáveis `BOLT_INSTRUMENT_FLAGS` e `BOLT_APPLY_FLAGS` do **`configure`** podem ser definidas para substituir o conjunto padrão de argumentos para **`llvm-bolt`** para instrumentar e aplicar dados BOLT aos binários , respectivamente.

Adicionado na versão 3.12.

BOLT_APPLY_FLAGS

Argumentos para `llvm-bolt` ao criar um binário otimizado com BOLT.

Adicionado na versão 3.12.

BOLT_INSTRUMENT_FLAGS

Argumentos para `llvm-bolt` ao instrumentar binários.

Adicionado na versão 3.12.

--with-computed-gotos

Habilita "gotos" computados no laço de avaliação (habilitado por padrão em compiladores suportados).

--with-tail-call-interp

Habilita interpretadores usando chamadas de cauda no CPython. Se habilitado, é altamente recomendável habilitar PGO (*`--enable-optimizations`*). Esta opção requer especificamente um compilador C com suporte adequado a chamadas de cauda e a convenção de chamada preserve_none. Por exemplo, o Clang 19 e versões mais recentes oferecem suporte a esse recurso.

Adicionado na versão 3.14.

--without-mimalloc

Desativa o alocador rápido mimalloc (habilitado por padrão).

Veja também a variável de ambiente *`PYTHONMALLOC`*.

--without-pymalloc

Desabilita o alocador de memória especializado do Python pymalloc (habilitado por padrão).

Veja também a variável de ambiente *`PYTHONMALLOC`*.

--without-doc-strings

Desabilita as strings de documentação estática para reduzir o consumo de memória (habilitado por padrão). As strings de documentação definidas em Python não são afetadas.

Não define a macro `WITH_DOC_STRINGS`.

Veja a macro `PyDoc_STRVAR()`.

--enable-profiling

Habilita o perfil de código a nível C com `gprof` (desabilitado por padrão).

--with-strict-overflow

Adiciona `-fstrict-overflow` aos sinalizadores do compilador C (por padrão adicionamos `-fno-strict-overflow`).

--without-remote-debug

Desativa o suporte à depuração remota descrito na **PEP 768** (habilitado por padrão). Quando este sinalizador é fornecido, o código que permite ao interpretador agendar a execução de um arquivo Python em um processo separado, conforme descrito na **PEP 768**, não é compilado. Isso inclui tanto a funcionalidade de agendar a execução do código quanto a funcionalidade de receber a execução do código.

Py_REMOTE_DEBUG

Esta macro é definida por padrão, a menos que o Python esteja configurado com *--without-remote-debug*.

Observe que, mesmo que a macro esteja definida, a depuração remota pode não estar disponível (por exemplo, em uma plataforma incompatível).

Adicionado na versão 3.14.

3.3.8 Compilação de depuração do Python

Uma compilação de depuração é Python compilada com a opção de configuração *--with-pydebug*.

Efeitos de uma compilação de depuração:

- Exibe todos os avisos por padrão: a lista de filtros de aviso padrão está vazia no módulo `warnings`.
- Adiciona `d` a `sys.abiflags`.
- Adiciona a função `sys.gettotalrefcount()`.
- Adiciona a opção de linha de comando *-X showrefcount*.
- Adiciona a opção de linha de comando *-d* e a variável de ambiente *PYTHONDEBUG* para depurar o analisador sintático.
- Adiciona suporte para a variável `__lltrace__`: habilita o rastreamento de baixo nível no laço de avaliação de bytecode se a variável estiver definida.
- Instala ganchos de depuração nos alocadores de memória para detectar estouro de buffer e outros erros de memória.
- Define as macros `Py_DEBUG` e `Py_REF_DEBUG`.
- Adiciona verificações de tempo de execução: código cercado por `#ifdef Py_DEBUG` e `#endif`. Habilita as asserções `assert(...)` e `_PyObject_ASSERT(...)`: não define a macro `NDEBUG` (veja também a configuração *--with-assertions* opção). Principais verificações de tempo de execução:
  - Adiciona verificações de sanidade nos argumentos da função.
  - Objetos Unicode e int são criados com sua memória preenchida com um padrão para detectar o uso de objetos não inicializados.
  - Garante que as funções que podem limpar ou substituir a exceção atual não sejam chamadas com uma exceção levantada.
  - Verifica se as funções desalocadoras não alteram a exceção atual.

- O coletor de lixo (função `gc.collect()`) executa algumas verificações básicas na consistência dos objetos.
- A macro `Py_SAFE_DOWNCAST()` verifica o underflow e o overflow de inteiros ao fazer o downcast de tipos largos para tipos estreitos.

Veja também o Modo de Desenvolvimento do Python e a opção de configuração *`--with-trace-refs`*.

Alterado na versão 3.8: Construções de lançamento e construções de depuração agora são compatíveis com ABI: definir a macro `Py_DEBUG` não implica mais na macro `Py_TRACE_REFS` (consulte a opção *`--with-trace-refs`*).

3.3.9 Opções de depuração

`--with-pydebug`

Construção de depuração do Python: define a macro `Py_DEBUG` (desabilitada por padrão).

`--with-trace-refs`

Habilita referências de rastreamento para fins de depuração (desabilitado por padrão).

Efeitos:

- Define a macro `Py_TRACE_REFS`.
- Adiciona a função `sys.getobjects()`.
- Adiciona a variável de ambiente *`PYTHONDUMPREFS`*.

A variável de ambiente *`PYTHONDUMPREFS`* pode ser usada para despejar objetos e contagens de referências ainda ativas na saída do Python.

Objetos alocados estaticamente não são rastreados.

Adicionado na versão 3.8.

Alterado na versão 3.13: Esta construção agora é compatibilidade de ABI com a construção de lançamento e *construção de depuração*.

`--with-assertions`

Constrói com asserções C habilitadas (o padrão é não): `assert(...);` e `_PyObject_ASSERT(...);`.

Se definido, a macro `NDEBUG` não é definida na variável do compilador *`OPT`*.

Veja também a opção *`--with-pydebug`* (*construção de depuração*) que também habilita asserções.

Adicionado na versão 3.6.

`--with-valgrind`

Habilita suporte ao Valgrind (o padrão é não).

`--with-dtrace`

Habilita suporte ao DTrace (o padrão é não).

Veja Instrumentando o CPython com DTrace e SystemTap.

Adicionado na versão 3.6.

`--with-address-sanitizer`

Habilita o detector de erros de memória AddressSanitizer, `asan` (o padrão é não). Para melhorar os recursos de detecção do ASan, você também pode combinar isso com *`--without-pymalloc`* para desabilitar o alocador especializado de pequenos objetos cujas alocações não são rastreadas pelo ASan.

Adicionado na versão 3.6.

`--with-memory-sanitizer`

Habilita o detector de erros de alocação do MemorySanitizer, `msan` (o padrão é não).

Adicionado na versão 3.6.

--with-undefined-behavior-sanitizer

Habilita o detector de comportamento indefinido UndefinedBehaviorSanitizer, `ubsan` (o padrão é não).

Adicionado na versão 3.6.

--with-thread-sanitizer

Habilita o detector de corrida de dados ThreadSanitizer, `tsan` (o padrão é não).

Adicionado na versão 3.13.

3.3.10 Opções da ligação

--enable-shared

Habilita a construção de uma biblioteca Python compartilhada: `libpython` (o padrão é não).

--without-static-libpython

Não constrói `libpythonMAJOR.MINOR.a` e não instala `python.o` (construído e habilitado por padrão).

Adicionado na versão 3.10.

3.3.11 Opções da biblioteca

--with-libs=`'lib1 ...'`

Vincula bibliotecas adicionais (o padrão é não).

--with-system-expat

Constrói o módulo `pyexpat` usando uma biblioteca `expat` instalada (o padrão é não).

--with-system-libmpdec

Constrói o módulo de extensão `_decimal` usando uma biblioteca `mpdecimal` instalada, veja o módulo `decimal` (o padrão é sim).

Adicionado na versão 3.3.

Alterado na versão 3.13: O padrão é usar a biblioteca `mpdecimal` instalada.

Deprecated since version 3.13, will be removed in version 3.15: Uma cópia dos fontes da biblioteca `mpdecimal` não será mais distribuída com Python 3.15.

> **Ver também**
>
> *LIBMPDEC_CFLAGS* e *LIBMPDEC_LIBS*.

--with-readline=`readline|editline`

Designa uma biblioteca backend para o módulo `readline`.

- readline: Usa readline como o backend.
- editline: Usa editline como o backend.

Adicionado na versão 3.10.

--without-readline

Não constrói o módulo `readline` (construído por padrão).

Não define a macro `HAVE_LIBREADLINE`.

Adicionado na versão 3.10.

--with-libm=`STRING`

Substitui a biblioteca matemática `libm` por *STRING* (o padrão depende do sistema).

--with-libc=STRING

Substitui a biblioteca C `libc` por *STRING* (o padrão depende do sistema).

--with-openssl=DIR

Raiz do diretório OpenSSL.

Adicionado na versão 3.7.

--with-openssl-rpath=[no|auto|DIR]

Define o diretório da biblioteca de tempo de execução (rpath) para bibliotecas OpenSSL:

- `no` (padrão): não define o rpath;
- `auto`: detecta automaticamente o rpath de *--with-openssl* e `pkg-config`;
- *DIR*: define um rpath explícito.

Adicionado na versão 3.10.

3.3.12 Opções de segurança

--with-hash-algorithm=[fnv|siphash13|siphash24]

Seleciona o algoritmo de hash para usar em `Python/pyhash.c`:

- `siphash13` (padrão);
- `siphash24`;
- `fnv`.

Adicionado na versão 3.4.

Adicionado na versão 3.11: `siphash13` é adicionado e é o novo padrão.

--with-builtin-hashlib-hashes=md5,sha1,sha256,sha512,sha3,blake2

Módulos embutidos de hash

- `md5`;
- `sha1`;
- `sha256`;
- `sha512`;
- `sha3` (com shake);
- `blake2`.

Adicionado na versão 3.9.

--with-ssl-default-suites=[python|openssl|STRING]

Substitui a string dos conjuntos de criptografia padrão do OpenSSL:

- `python` (padrão): use a seleciona preferida do Python;
- `openssl`: mantém inalterados os padrões do OpenSSL;
- *STRING*: usa uma string personalizada

Veja o módulo `ssl`.

Adicionado na versão 3.7.

Alterado na versão 3.10: As configurações `python` e *STRING* também definem TLS 1.2 como versão mínima do protocolo.

--disable-safety

Desabilita as opções do compilador recomendadas pela OpenSSF por motivos de segurança, sem causar sobrecarga de desempenho. Se esta opção não estiver habilitada, o CPython será construído com base nas opções de segurança do compilador, sem causar lentidão. Quando esta opção estiver habilitada, o CPython não será construído com as opções do compilador listadas abaixo.

As seguintes opções do compilador são desabilitadas com `--disable-safety`:

- -fstack-protector-strong: Habilita verificações em tempo de execução para estouros de buffer baseados em pilha.
- -Wtrampolines: Habilita avisos sobre trampolins que exigem pilhas executáveis.

Adicionado na versão 3.14.

--enable-slower-safety

Habilita as opções do compilador recomendadas pela OpenSSF por motivos de segurança, o que causa sobrecarga. Se esta opção não estiver habilitada, o CPython não será construído com base nas opções de segurança do compilador, o que impacta o desempenho. Quando esta opção estiver habilitada, o CPython será construído com as opções do compilador listadas abaixo.

As seguintes opções do compilador são habilitadas com `--enable-slower-safety`:

- -D_FORTIFY_SOURCE=3: Fortalece as fontes com verificações de tempo de compilação e execução para uso inseguro de libc e estouros de buffer.

Adicionado na versão 3.14.

3.3.13 Opções do macOS

Veja Mac/README.rst.

--enable-universalsdk

--enable-universalsdk=SDKDIR

Cria uma construção binária universal. *SDKDIR* especifica qual SDK do macOS deve ser usado para executar a construção (o padrão é não).

--enable-framework

--enable-framework=INSTALLDIR

Cria um Python.framework em vez de uma instalação tradicional do Unix. O *INSTALLDIR* opcional especifica o caminho de instalação (o padrão é não).

--with-universal-archs=ARCH

Especifica o tipo de binário universal que deve ser criado. Esta opção só é válida quando *--enable-universalsdk* está definido.

Opções:

- `universal2` (x86-64 e arm64);
- `32-bit` (PPC e i386);
- `64-bit` (PPC64 e x86-64);
- `3-way` (i386, PPC e x86-64);
- `intel` (i386 e x86-64);
- `intel-32` (i386);
- `intel-64` (x86-64);
- `all` (PPC, i386, PPC64 e x86-64).

Observe que os valores para este item de configuração *não* são os mesmos que os identificadores usados para rodas binárias universais no macOS. Veja o Guia de Empacotamento do Python para detalhes sobre as tags de compatibilidade de plataforma de empacotamento usadas no macOS

--with-framework-name=FRAMEWORK

Especifica o nome do framework python no macOS válido apenas quando *--enable-framework* está definido (padrão: `Python`).

--with-app-store-compliance

--with-app-store-compliance=PATCH-FILE

A biblioteca padrão do Python contém strings que são conhecidas por acionar erros de ferramentas de inspeção automatizadas quando enviadas para distribuição pelas App Stores do macOS e do iOS. Se ativada, esta opção aplicará a lista de patches que corrigem a conformidade da app store. Um arquivo de patch personalizado também pode ser especificado. Esta opção está desativada por padrão.

Adicionado na versão 3.13.

3.3.14 Opções do iOS

Veja iOS/README.rst.

--enable-framework=INSTALLDIR

Cria um Python.framework. Ao contrário do macOS, o argumento *INSTALLDIR* que especifica o caminho de instalação é obrigatório.

--with-framework-name=FRAMEWORK

Especifica o nome do framework (padrão: `Python`).

3.3.15 Opções de compilação cruzada

A compilação cruzada, também conhecida como construção cruzada, pode ser usada para construir Python para outra arquitetura ou plataforma de CPU. A compilação cruzada requer um interpretador Python para a plataforma de construção. A versão do Python para construção deve corresponder à versão do Python da compilação cruzada do host.

--build=BUILD

configura para construir em BUILD, geralmente adivinhado por **config.guess**.

--host=HOST

faz compilação cruzada para construir programas para executar no HOST (plataforma de destino)

--with-build-python=path/to/python

caminho para construir o binário `python` para compilação cruzada

Adicionado na versão 3.11.

CONFIG_SITE=file

Uma variável de ambiente que aponta para um arquivo com substituições de configuração.

Exemplo de arquivo *config.site*:

```
# config.site-aarch64
ac_cv_buggy_getaddrinfo=no
ac_cv_file__dev_ptmx=yes
ac_cv_file__dev_ptc=no
```

HOSTRUNNER

Programa para executar CPython para a plataforma host para compilação cruzada.

Adicionado na versão 3.11.

Exemplo de compilação cruzada:

```
CONFIG_SITE=config.site-aarch64 ../configure \
    --build=x86_64-pc-linux-gnu \
    --host=aarch64-unknown-linux-gnu \
    --with-build-python=../x86_64/python
```

3.4 Sistema de Construção Python

3.4.1 Arquivos principais do sistema de construção

- `configure.ac` => `configure`;
- `Makefile.pre.in` => `Makefile` (criado por `configure`);
- `pyconfig.h` (criado por `configure`);
- `Modules/Setup`: Extensões C construídas pelo Makefile usando o shell script `Module/makesetup`;

3.4.2 Principais etapas de construção

- Arquivos C (`.c`) são construídos como arquivos objeto (`.o`).
- Uma biblioteca estática `libpython` (`.a`) é criada a partir de arquivos de objetos.
- `python.o` e a biblioteca estática `libpython` estão vinculadas ao programa final `python`.
- Extensões C são construídas pelo Makefile (veja `Modules/Setup`).

3.4.3 Alvos principais do Makefile

make

Na maioria das vezes, ao reconstruir após editar algum código ou atualizar seu checkout do upstream, tudo que você precisa fazer é executar `make`, que (pela semântica do Make) constrói o alvo padrão, o primeiro definido no Makefile. Por tradição (inclusive no projeto CPython) este é geralmente o alvo `all`. O script `configure` expande uma variável `autoconf`, `@DEF_MAKE_ALL_RULE@` para descrever precisamente quais alvos `make all` serão construídos. As três opções são:

- `profile-opt` (configurado com `--enable-optimizations`)
- `build_wasm` (escolhido se a plataforma host corresponder a `wasm32-wasi*` ou `wasm32-emscripten`)
- `build_all` (configurado sem usar explicitamente nenhum dos outros)

Dependendo das alterações mais recentes no arquivo fonte, o Make irá reconstruir quaisquer alvos (arquivos objetos e executáveis) considerados desatualizados, incluindo executar `configure` novamente se necessário. As dependências de origem/alvo são muitas e mantidas manualmente, portanto, às vezes, o Make não tem todas as informações necessárias para detectar corretamente todos os alvos que precisam ser reconstruídos. Dependendo de quais destinos não são reconstruídos, você poderá enfrentar vários problemas. Se você tiver problemas de construção ou teste que você não pode explicar de outra forma, `make clean && make` deve resolver a maioria dos problemas de dependência, às custas de tempos de construção mais longos.

make platform

Constrói o programa `python`, mas não constrói os módulos de extensão da biblioteca padrão. Isto gera um arquivo chamado `platform` que contém uma única linha descrevendo os detalhes da plataforma de construção, por exemplo, `macosx-14.3-arm64-3.12` ou `linux-x86_64-3.13`.

make profile-opt

Constrói Python usando otimização guiada por perfil (PGO). Você pode usar a opção *`--enable-optimizations`* do configure para tornar este o alvo padrão do comando `make` (`make all` ou apenas `make`).

make clean

Remove arquivos construídos.

make distclean

Além do trabalho feito por `make clean`, remove os arquivos criados pelo script configure. `configure` terá que ser executado antes de construir novamente.[1]

make install

Constrói o alvo `all` e instala o Python.

make test

Constrói o alvo `all` e execute o conjunto de testes Python com a opção `--fast-ci` sem testes de GUI. Variáveis:

- `TESTOPTS`: opções adicionais de linha de comando regrtest.
- `TESTPYTHONOPTS`: opções adicionais de linha de comando do Python.
- `TESTTIMEOUT`: tempo limite em segundos (padrão: 10 minutos).

make ci

Isso é semelhante ao `make test`, mas usa `-ugui` para também executar testes de GUI.

Adicionado na versão 3.14.

make buildbottest

Isto é semelhante ao `make test`, mas usa a opção `--slow-ci` e o tempo limite padrão de 20 minutos, em vez da opção `--fast-ci`.

make regen-all

Gera novamente (quase) todos os arquivos gerados. Isso inclui (mas não está limitado a) casos de bytecode e arquivo gerador de analisador sintático. `make regen-stdlib-module-names` e `autoconf` devem ser executados separadamente para os restantes *arquivos gerados*.

3.4.4 Extensões C

Algumas extensões C são construídas como módulos embutidos, como o módulo `sys`. Eles são construídos com a macro `Py_BUILD_CORE_BUILTIN` definida. Módulos embutidos não possuem nenhum atributo `__file__`:

```
>>> import sys
>>> sys
<module 'sys' (built-in)>
>>> sys.__file__
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: module 'sys' has no attribute '__file__'
```

Outras extensões C são construídas como bibliotecas dinâmicas, como o módulo `_asyncio`. Eles são construídos com a macro `Py_BUILD_CORE_MODULE` definida. Exemplo no Linux x86-64:

[1] `git clean -fdx` é uma maneira ainda mais extrema de "limpar" seu checkout. Ele remove todos os arquivos não conhecidos pelo Git. Ao procurar bugs usando `git bisect`, isso é recomendado entre testes para garantir uma construção completamente limpa. **Use com cuidado**, pois isso excluirá todos os arquivos não registrados no Git, incluindo seu trabalho novo e não confirmado.

```
>>> import _asyncio
>>> _asyncio
<module '_asyncio' from '/usr/lib64/python3.9/lib-dynload/_asyncio.cpython-39-x86_
↪64-linux-gnu.so'>
>>> _asyncio.__file__
'/usr/lib64/python3.9/lib-dynload/_asyncio.cpython-39-x86_64-linux-gnu.so'
```

`Modules/Setup` é usado para gerar alvos Makefile para construir extensões C. No início dos arquivos, as extensões C são construídas como módulos embutidos. Extensões definidas após o marcador `*shared*` são construídas como bibliotecas dinâmicas.

As macros `PyAPI_FUNC()`, `PyAPI_DATA()` e `PyMODINIT_FUNC` de `Include/exports.h` são definidas de forma diferente dependendo se a macro `Py_BUILD_CORE_MODULE` está definida:

- Usa `Py_EXPORTED_SYMBOL` se `Py_BUILD_CORE_MODULE` estiver definido
- Do contrário, usa `Py_IMPORTED_SYMBOL`.

Se a macro `Py_BUILD_CORE_BUILTIN` for usada por engano em uma extensão C construída como uma biblioteca compartilhada, sua função `PyInit_xxx()` não será exportada, causando um `ImportError` na importação.

3.5 Sinalizadores do compilador e do vinculador

Opções definidas pelo script `./configure` e variáveis de ambiente e usadas por `Makefile`.

3.5.1 Sinalizadores do pré-processador

CONFIGURE_CPPFLAGS

Valor da variável *CPPFLAGS* passado para o script `./configure`.

Adicionado na versão 3.6.

CPPFLAGS

Sinalizadores de pré-processador C++ / (Objective) C, p.ex. `-Iinclude_dir` se você tiver cabeçalhos em um diretório não padrão *include_dir*.

Tanto *CPPFLAGS* quanto *LDFLAGS* precisam conter o valor do shell para poder construir módulos de extensão usando os diretórios especificados nas variáveis de ambiente.

BASECPPFLAGS

Adicionado na versão 3.4.

PY_CPPFLAGS

Sinalizadores extras de pré-processador adicionados para construir os arquivos de objeto do interpretador.

Padrão: `$(BASECPPFLAGS) -I. -I$(srcdir)/Include $(CONFIGURE_CPPFLAGS) $(CPPFLAGS)`.

Adicionado na versão 3.2.

3.5.2 Sinalizadores do compilador

CC

Comando do compilador C.

Exemplo: `gcc -pthread`.

CXX

Comando do compilador C++.

Exemplo: `g++ -pthread`.

CFLAGS

Sinalizadores do compilador C.

CFLAGS_NODIST

CFLAGS_NODIST é usado para construir o interpretador e extensões C da stdlib. Use-o quando um sinalizador do compilador *não* deve fazer parte de *CFLAGS* depois que o Python estiver instalado (gh-65320).

Em particular, *CFLAGS* não deve conter:

- o sinalizador do compilador `-I` (para definir o caminho de pesquisa para arquivos incluídos). Os sinalizadores `-I` são processadas da esquerda para a direita, e quaisquer sinalizadores em *CFLAGS* terão precedência sobre os sinalizadores `-I` fornecidos pelo usuário e pelo pacote.
- sinalizadores de segurança como `-Werror` porque as distribuições não podem controlar se os pacotes instalados pelos usuários estão em conformidade com esses padrões elevados.

Adicionado na versão 3.5.

COMPILEALL_OPTS

Opções passadas para a linha de comando `compileall` ao construir arquivos PYC em `make install`. Padrão: `-j0`.

Adicionado na versão 3.12.

EXTRA_CFLAGS

Sinalizadores extra do compilador C.

CONFIGURE_CFLAGS

Valor da variável *CFLAGS* passado para o script `./configure`.

Adicionado na versão 3.2.

CONFIGURE_CFLAGS_NODIST

Valor da variável *CFLAGS_NODIST* passado para o script `./configure`.

Adicionado na versão 3.5.

BASECFLAGS

Sinalizadores base do compilador.

OPT

Sinalizadores de otimização.

CFLAGS_ALIASING

Sinalizadores de alias estritos ou não estritos usados para compilar `Python/dtoa.c`.

Adicionado na versão 3.7.

CCSHARED

Sinalizadores de compilador usados para construir uma biblioteca compartilhada.

Por exemplo, `-fPIC` é usado no Linux e no BSD.

CFLAGSFORSHARED

Sinalizadores extras de C adicionados para construir os arquivos de objeto do interpretador.

Padrão: `$(CCSHARED)` quando *--enable-shared* é usado, ou uma string vazia caso contrário.

PY_CFLAGS

Padrão: `$(BASECFLAGS) $(OPT) $(CONFIGURE_CFLAGS) $(CFLAGS) $(EXTRA_CFLAGS)`.

PY_CFLAGS_NODIST

Padrão: `$(CONFIGURE_CFLAGS_NODIST) $(CFLAGS_NODIST) -I$(srcdir)/Include/internal`.

Adicionado na versão 3.5.

PY_STDMODULE_CFLAGS

Sinalizadores do C usados para construir os arquivos de objeto do interpretador.

Padrão: `$(PY_CFLAGS) $(PY_CFLAGS_NODIST) $(PY_CPPFLAGS) $(CFLAGSFORSHARED)`.

Adicionado na versão 3.7.

PY_CORE_CFLAGS

Padrão: `$(PY_STDMODULE_CFLAGS) -DPy_BUILD_CORE`.

Adicionado na versão 3.2.

PY_BUILTIN_MODULE_CFLAGS

Sinalizadores do compilador para construir um módulo de extensão de biblioteca padrão como um módulo embutido, como o módulo `posix`.

Padrão: `$(PY_STDMODULE_CFLAGS) -DPy_BUILD_CORE_BUILTIN`.

Adicionado na versão 3.8.

PURIFY

Comando de Purify. Purify é um programa depurador de memória.

Padrão: string vazia (não usada).

3.5.3 Sinalizadores do vinculador

LINKCC

Comando do vinculador usado para construir programas como `python` e `_testembed`.

Padrão: `$(PURIFY) $(CC)`.

CONFIGURE_LDFLAGS

Valor da variável *LDFLAGS* passado para o script `./configure`.

Evite atribuir *CFLAGS*, *LDFLAGS*, etc. para que os usuários possam usá-los na linha de comando para anexar a esses valores sem pisotear os valores predefinidos.

Adicionado na versão 3.2.

LDFLAGS_NODIST

LDFLAGS_NODIST é usado da mesma maneira que *CFLAGS_NODIST*. Use-o quando um sinalizador de vinculador *não* fizer parte de *LDFLAGS* depois que o Python estiver instalado (gh-65320).

Em particular, *LDFLAGS* não deve conter:

- o sinalizador do compilador `-L` (para definir o caminho de pesquisa para arquivos incluídos). Os sinalizadores `-L` são processadas da esquerda para a direita, e quaisquer sinalizadores em *LDFLAGS* terão precedência sobre os sinalizadores `-L` fornecidos pelo usuário e pelo pacote.

CONFIGURE_LDFLAGS_NODIST

Valor da variável *LDFLAGS_NODIST* passado para o script `./configure`.

Adicionado na versão 3.8.

LDFLAGS

Sinalizadores do vinculador, p.ex. `-L`*lib_dir*, se você tiver bibliotecas em um diretório não padrão *lib_dir*.

Tanto *CPPFLAGS* quanto *LDFLAGS* precisam conter o valor do shell para poder construir módulos de extensão usando os diretórios especificados nas variáveis de ambiente.

LIBS

Sinalizadores do vinculador para passar bibliotecas para o vinculador ao vincular o executável Python.

Exemplo: `-lrt`.

LDSHARED

Comando para construir uma biblioteca compartilhada.

Padrão: `@LDSHARED@ $(PY_LDFLAGS)`.

BLDSHARED

Comando para construir a biblioteca compartilhada `libpython`.

Padrão: `@BLDSHARED@ $(PY_CORE_LDFLAGS)`.

PY_LDFLAGS

Padrão: `$(CONFIGURE_LDFLAGS) $(LDFLAGS)`.

PY_LDFLAGS_NODIST

Padrão: `$(CONFIGURE_LDFLAGS_NODIST) $(LDFLAGS_NODIST)`.

Adicionado na versão 3.8.

PY_CORE_LDFLAGS

Sinalizadores de vinculador usados para construir os arquivos de objeto do interpretador.

Adicionado na versão 3.8.

CAPÍTULO 4

Utilizando Python no Windows

Este documento pretende dar uma visão geral do comportamento específico do Windows que você deve conhecer quando fores utilizar o Python no sistema operacional Microsoft Windows.

Ao contrário da maioria dos sistemas e serviços Unix, o Windows não inclui uma instalação do Python com suporte no sistema. Em vez disso, o Python pode ser obtido de diversos distribuidores, inclusive diretamente da equipe do CPython. Cada distribuição do Python terá suas próprias vantagens e desvantagens; no entanto, a consistência com outras ferramentas que você usa geralmente é um benefício valioso. Antes de se comprometer com o processo descrito aqui, recomendamos verificar suas ferramentas existentes para ver se elas podem fornecer Python diretamente.

Para obter o Python da equipe do CPython, use o Python Install Manager (gerenciador de instalação do Python). Esta é uma ferramenta autônoma que disponibiliza o Python como comandos globais na sua máquina Windows, integra-se ao sistema e suporta atualizações ao longo do tempo. Você pode baixar o Python Install Manager. em python.org/downloads ou pelo aplicativo da Microsoft Store.

Após instalar o Python Install Manager, o comando global `python` pode ser usado em qualquer terminal para iniciar a versão mais recente do Python. Essa versão pode mudar com o tempo, conforme você adiciona ou remove versões diferentes, e o comando `py list` mostrará qual é a atual.

Em geral, recomendamos que você crie um ambiente virtual para cada projeto e execute `<env>\Scripts\Activate` no seu terminal para usá-lo. Isso proporciona isolamento entre projetos, consistência ao longo do tempo e garante que comandos adicionais adicionados por pacotes também estejam disponíveis na sua sessão. Crie um ambiente virtual usando `python -m venv <caminho do ambiente>`.

Se os comandos `python` ou `py` não estiverem funcionando, consulte a seção *Solução de problemas* abaixo. Às vezes, são necessárias etapas manuais adicionais para configurar seu PC.

Além de usar o gerenciador de instalação do Python, o Python também pode ser obtido como pacotes NuGet. Veja *Os pacotes nuget.org* abaixo para mais informações sobre esses pacotes.

As distribuições incorporáveis são pacotes mínimos do Python adequados para incorporação em aplicações maiores. Elas podem ser instaladas usando o gerenciador de instalação do Python. Consulte *O pacote embutível* abaixo para obter mais informações sobre esses pacotes.

4.1 Gerenciador de instalação do Python

4.1.1 Instalação

O gerenciador de instalação do Python pode ser instalado a partir da aplicação da Microsoft Store ou baixado e instalado de python.org/downloads. As duas versões são idênticas.

Para instalar pela Store, basta clicar em "Install". Após a conclusão, abra um terminal e digite `python` para começar.

Para instalar o arquivo baixado de python.org, clique duas vezes e selecione "Install" ou execute `Add-AppxPackage <caminho para MSIX>` no Windows Powershell.

Após a instalação, os comandos `python`, `py` e `pymanager` devem estar disponíveis. Se você já possui instalações do Python ou modificou a variável `PATH`, pode ser necessário removê-las ou desfazer as modificações. Consulte *Troubleshooting* para obter mais ajuda sobre como corrigir comandos que não funcionam.

Ao instalar um ambiente de execução pela primeira vez, você provavelmente será solicitado a adicionar um diretório ao seu `PATH`. Isso é opcional se você preferir usar o comando `py`, mas é oferecido para aqueles que preferem que todos os apelidos (como `python3.14.exe`) estejam disponíveis. O diretório será `%LocalAppData%\Python\bin` por padrão, mas pode ser personalizado por um administrador. Clique em Iniciar e pesquise "Editar as variáveis de ambiente para sua conta" na página de configurações do sistema para adicionar o caminho.

Cada ambiente de execução do Python que você instalar terá seu próprio diretório para scripts. Estes também precisam ser adicionados a `PATH` se você quiser usá-los.

O gerenciador de instalação do Python será atualizado automaticamente para novas versões. Isso não afeta nenhuma instalação de ambientes de execução do Python. Desinstalar o gerenciador de instalação do Python não desinstala nenhum ambiente de execução do Python.

Se você não conseguir instalar um MSIX no seu contexto, por exemplo, se estiver usando um software de implantação automatizada que não oferece suporte a isso ou se estiver mirando o Windows Server 2019, consulte *Advanced Installation* abaixo para obter mais informações.

4.1.2 Uso básico

O comando recomendado para iniciar o Python é `python`, que iniciará a versão solicitada pelo script em execução, um ambiente virtual ativo ou a versão padrão instalada, que será a versão estável mais recente, a menos que configurado de outra forma. Se nenhuma versão for solicitada especificamente e nenhum ambiente de execução estiver instalado, a versão mais recente será instalada automaticamente.

Para todos os cenários que envolvem múltiplas versões de ambientes de execução do antigo inicializador `py.exe`. Por padrão, `py` corresponde ao comportamento de `python`, mas também permite opções de linha de comando para selecionar uma versão específica, bem como subcomandos para gerenciar instalações. Estes são detalhados abaixo.

Como o comando `py` pode já estar sendo executado pela versão anterior, há também um comando `pymanager` inequívoco. Instalações com script que pretendem usar o gerenciador de instalação Python devem considerar o uso de `pymanager`, devido à menor chance de conflito com instalações existentes. A única diferença entre os dois comandos é que, quando executados sem argumentos, `py` instalará e iniciará seu interpretador padrão, enquanto `pymanager` exibirá ajuda (`pymanager exec ...` oferece comportamento equivalente a `py ...`).

Cada um desses comandos também possui uma versão em janela que evita a criação de uma janela de console. São eles `pyw`, `pythonw` e `pymanagerw`. Um comando `python3` também está incluído, imitando o comando `python`. Ele visa detectar usos acidentais do comando POSIX típico no Windows, mas não se destina a ser amplamente utilizado ou recomendado.

Para iniciar seu ambiente de execução padrão, execute `python` ou `py` com os argumentos que você deseja passar para o ambiente de execução (como arquivos de script ou o módulo a ser iniciado):

```
$> py
...
$> python meu-script.py
...
$> py -m this
...
```

O ambiente de execução padrão pode ser substituído pela variável de ambiente *PYTHON_MANAGER_DEFAULT* ou por um arquivo de configuração. Consulte *Configuração* para obter informações sobre as configurações.

Para iniciar um ambiente de execução específico, o comando `py` aceita a opção `-V:<TAG>`. Esta opção deve ser especificada antes de quaisquer outras. A tag é parte ou todo o identificador do ambiente; para aqueles da equipe do

CPython, parece a versão, potencialmente com a plataforma. Para compatibilidade, o `V:` pode ser omitido nos casos em que a tag se refere a uma versão oficial e começa com `3`.

```
$> py -V:3.14 ...
$> py -V:3-arm64 ...
```

Ambientes de execução de outros distribuidores podem exigir a inclusão da *company* também. Esta deve ser separada da tag por uma barra e pode ser um prefixo. Especificar a empresa é opcional quando se trata de `PythonCore`, e especificar a tag é opcional (mas não a barra) quando se deseja a versão mais recente de uma empresa específica.

```
$> py -V:Distribuidor\1.0 ...
$> py -V:distrib/ ...
```

Se nenhuma versão for especificada, mas um arquivo de script for passado, o script será inspecionado em busca de uma *linha de shebang*. Este é um formato especial para a primeira linha de um arquivo que permite a substituição do comando. Consulte *Shebang lines* para obter mais informações. Quando não houver uma linha shebang ou o problema não puder ser resolvido, o script será iniciado com o ambiente de execução padrão.

Se você estiver executando em um ambiente virtual ativo, não tiver solicitado uma versão específica e não houver uma linha shebang, o ambiente de execução padrão será o ambiente virtual em questão. Nesse cenário, o comando `python` provavelmente já foi substituído e nenhuma dessas verificações ocorreu. No entanto, esse comportamento garante que o comando `py` possa ser usado de forma intercambiável.

Ao iniciar `python` ou `py`, mas não tiver nenhum ambiente de execução instalado, e a versão solicitada for a padrão, ele será instalado automaticamente e, em seguida, executado. Caso contrário, a versão solicitada será instalada se a instalação automática estiver configurada (provavelmente definindo `PYTHON_MANAGER_AUTOMATIC_INSTALL` como `true`) ou se as formas `py exec` ou `pymanager exec` do comando tiverem sido utilizadas.

4.1.3 Comando help

O comando `py help` exibirá a lista completa de comandos suportados, juntamente com suas opções. Qualquer comando pode receber a opção `-?` para exibir sua ajuda, ou seu nome pode ser passado para `py help`.

```
$> py help
$> py help install
$> py install /?
```

Todos os comandos oferecem suporte a algumas opções comuns, que serão exibidas por `py help`. Essas opções devem ser especificadas após qualquer subcomando. Especificar `-v` ou `--verbose` aumentará a quantidade de saída exibida, e `-vv` aumentará ainda mais para fins de depuração. Passar `-q` ou `--quiet` reduzirá a saída, e `-qq` a reduzirá ainda mais.

A opção `--config=<PATH>` permite especificar um arquivo de configuração para substituir várias configurações simultaneamente. Consulte *Configuração* abaixo para obter mais informações sobre esses arquivos.

4.1.4 Listando ambientes de execução

```
$> py list [-f=|--format=<FMT>] [-1|--one] [--online|-s=|--source=<URL>] [<TAG>...]
```

A lista de ambientes de execução instalados pode ser visualizada usando `py list`. Um filtro pode ser adicionado na forma de uma ou mais tags (com ou sem especificador de empresa), e cada uma pode incluir um prefixo <, <=, >= ou > para restringir a um intervalo.

Vários formatos são suportados e podem ser passados como a opção `--format=<FMT>` ou `-f <FMT>`. Os formatos incluem `table` (uma visualização de tabela amigável), `csv` (tabela separada por vírgulas), `json` (um único blob JSON), `jsonl` (um blob JSON por resultado), `exe` (apenas o caminho do executável), `prefix` (apenas o caminho do prefixo).

A opção `--one` ou `-1` exibe apenas um único resultado. Se o ambiente de execução padrão estiver incluído, será ele. Caso contrário, o "melhor" resultado será exibido ("melhor" é definido de forma deliberadamente vaga, mas ge-

ralmente será a versão mais recente). O resultado exibido por `py list --one <TAG>` corresponderá ao ambiente de execução que seria iniciado por `py -V:<TAG>`.

A opção `--only-managed` exclui resultados que não foram instalados pelo gerenciador de instalação do Python. Isso é útil para determinar quais ambientes de execução podem ser atualizados ou desinstalados por meio do comando `py`.

A opção `--online` é uma abreviação de `--source=<URL>` com a fonte padrão. Passar qualquer uma dessas opções pesquisará no índice online os ambientes de execução que podem ser instalados. O resultado exibido por `py list --online --one <TAG>` corresponderá ao ambiente de execução que seria instalado por `py install <TAG>`.

```
$> py list --online 3.14
```

Para compatibilidade com o inicializador antigo, os comandos `--list`, `--list-paths`, `-0` e `-0p` (por exemplo, `py -0p`) foram mantidos. Eles não permitem opções adicionais e produzirão uma saída com o formato antigo.

4.1.5 Instalando ambientes de execução

```
$> py install [-s=|--source=<URL>] [-f|--force] [-u|--update] [--dry-run] [<TAG>...
↪]
```

Novas versões de ambientes de execução podem ser adicionadas usando `py install`. Uma ou mais tags podem ser especificadas, e a tag especial `default` pode ser usada para selecionar o padrão. Intervalos não são suportados para instalação.

A opção `--source=<URL>` permite substituir o índice online usado para obter ambientes de execução. Isso pode ser usado com um índice offline, como mostrado em *Instalações offline*.

Passar `--force` ignorará quaisquer arquivos em cache e removerá qualquer instalação existente para substituí-la pela especificada.

Passar `--update` substituirá as instalações existentes se a nova versão for mais recente. Caso contrário, elas serão mantidas. Se nenhuma tag for fornecida com `--update`, todas as instalações gerenciadas pelo gerenciador de instalação do Python serão atualizadas se versões mais recentes estiverem disponíveis. As atualizações removerão quaisquer modificações feitas na instalação, incluindo pacotes instalados globalmente, mas os ambientes virtuais continuarão funcionando.

Passar `--dry-run` gerará saída e logs, mas não modificará nenhuma instalação.

Além das opções acima, a opção `--target` extrairá o ambiente de execução para o diretório especificado em vez de realizar uma instalação normal. Isso é útil para incorporar ambientes de aplicação em aplicações maiores.

```
$> py install ... [-t=|--target=<PATH>] <TAG>
```

4.1.6 Instalações offline

Para executar instalações offline do Python, você precisará primeiro criar um índice offline em uma máquina que tenha acesso à rede.

```
$> py install --download=<PATH> ... <TAG>...
```

A opção `--download=<PATH>` baixará os pacotes para as tags listadas e criará um diretório contendo-os e um arquivo `index.json` adequado para instalação posterior. Todo esse diretório pode ser movido para a máquina offline e usado para instalar um ou mais dos ambientes de execução incluídos:

```
$> py install --source="<PATH>\index.json" <TAG>...
```

O gerenciador de instalação do Python pode ser instalado baixando seu instalador e movendo-o para outra máquina antes da instalação.

Como alternativa, os arquivos ZIP em um diretório de índice offline podem ser simplesmente transferidos para outra máquina e extraídos. Isso não registrará a instalação de forma alguma e, portanto, ela deve ser iniciada referenciando

diretamente os executáveis no diretório extraído, mas às vezes é uma abordagem preferível em casos em que a instalação do gerenciador de instalação do Python não é possível ou conveniente.

Dessa forma, os ambientes de execução do Python podem ser instalados e gerenciados em uma máquina sem acesso à internet.

4.1.7 Desinstalando ambientes de execução

```
$> py uninstall [-y|--yes] <TAG>...
```

Ambientes de execução pode ser removidos com o uso do comando `py uninstall`. Uma ou mais tags devem ser especificadas. Não há suporte para intervalos aqui.

The `--yes` option bypasses the confirmation prompt before uninstalling.

Em vez de passar as tags individualmente, a opção `--purge` pode ser especificada. Isso removerá todos os ambientes de execução gerenciados pelo gerenciador de instalação do Python, incluindo a limpeza do menu Iniciar, do registro e de quaisquer caches de download. Ambientes de execução que não foram instalados pelo gerenciador de instalação do Python não serão afetados, nem os arquivos de configuração criados manualmente.

```
$> py uninstall [-y|--yes] --purge
```

O gerenciador de instalação do Python pode ser desinstalado por meio da página de configurações "Aplicativos instalados" do Windows. Isso não remove nenhum ambiente de execução, e eles ainda poderão ser usados, embora o `python` global e o comando `py` sejam removidos. A reinstalação do gerenciador de instalação do Python permitirá que você gerencie esses ambientes de execução novamente. Para limpar completamente todos os ambientes de execução do Python, execute o `--purge` antes de desinstalar o gerenciador de instalação do Python.

4.1.8 Configuração

Python install manager is configured with a hierarchy of configuration files, environment variables, command-line options, and registry settings. In general, configuration files have the ability to configure everything, including the location of other configuration files, while registry settings are administrator-only and will override configuration files. Command-line options override all other settings, but not every option is available.

This section will describe the defaults, but be aware that modified or overridden installs may resolve settings differently.

A global configuration file may be configured by an administrator, and would be read first. The user configuration file is stored at `%AppData%\Python\pymanager.json` (by default) and is read next, overwriting any settings from earlier files. An additional configuration file may be specified as the `PYTHON_MANAGER_CONFIG` environment variable or the `--config` command line option (but not both).

The following settings are those that are considered likely to be modified in normal use. Later sections list those that are intended for administrative customization.

Standard configuration options

Config Key	Environment Variable	Descrição
`default_tag`	**`PYTHON_MANAGER_DEFAULT`**	The preferred default version to launch or install. By default, this is interpreted as the most recent non-prerelease version from the CPython team.
`default_platform`	`PYTHON_MANAGER_DEFAULT_PLAT`	The preferred default platform to launch or install. This is treated as a suffix to the specified tag, such that `py -V:3.14` would prefer an install for `3.14-64` if it exists (and `default_platform` is `-64`), but will use `3.14` if no tagged install exists.
`logs_dir`	`PYTHON_MANAGER_LOGS`	The location where log files are written. By default, `%TEMP%`.
`automatic_install`	`PYTHON_MANAGER_AUTOMATIC_IN`	Verdadeiro para permitir instalações automáticas ao especificar um ambiente de execução específico para iniciar. Por padrão, verdadeiro.
`include_unmanaged`	`PYTHON_MANAGER_INCLUDE_UNMA`	Verdadeiro para permitir a listagem e a execução de ambientes que não foram instalados pelo gerenciador de instalação do Python, ou falso para excluí-los. Por padrão, verdadeiro.
`shebang_can_run_anything`	`PYTHON_MANAGER_SHEBANG_CAN_`	Verdadeiro para permitir que shebangs em arquivos `.py` iniciem aplicação que não sejam ambientes de execução do Python, ou falso para evitar isso. Por padrão, verdadeiro.
`log_level`	`PYMANAGER_VERBOSE`, `PYMANAGER_DEBUG`	Set the default level of output (0--50). By default, 20. Lower values produce more output. The environment variables are boolean, and may produce additional output during startup that is later suppressed by other configuration.
`confirm`	`PYTHON_MANAGER_CONFIRM`	True to confirm certain actions before taking them (such as uninstall), or false to skip the confirmation. By default, true.
`install.source`	`PYTHON_MANAGER_SOURCE_URL`	Override the index feed to obtain new installs from.
`list.format`	`PYTHON_MANAGER_LIST_FORMAT`	Specify the default format used by the `py list` command. By default, `table`.

Dotted names should be nested inside JSON objects, for example, `list.format` would be specified as `{"list": {"format": "table"}}`.

4.1.9 Shebang lines

If the first line of a script file starts with `#!`, it is known as a "shebang" line. Linux and other Unix like operating systems have native support for such lines and they are commonly used on such systems to indicate how a script should be executed. The `python` and `py` commands allow the same facilities to be used with Python scripts on Windows.

To allow shebang lines in Python scripts to be portable between Unix and Windows, a number of 'virtual' commands are supported to specify which interpreter to use. The supported virtual commands are:

- `/usr/bin/env <ALIAS>`
- `/usr/bin/env -S <ALIAS>`
- `/usr/bin/<ALIAS>`
- `/usr/local/bin/<ALIAS>`
- `<ALIAS>`

Por exemplo, se a primeira linha do seu script começa com

```
#! /usr/bin/python
```

O Python padrão ou um ambiente virtual ativo será localizado e utilizado. Como muitos scripts Python escritos para funcionar no Unix já terão essa linha, você deve perceber que esses scripts podem ser usados pelo inicializador sem modificação. Se você está escrevendo um novo script no Windows que você espera que seja útil no Unix, você deve usar uma dessas linhas shebang começando com `/usr`.

Qualquer um dos comandos virtuais acima pode ter o `<ALIAS>` substituído por um apelido de um ambiente de execução instalado. Ou seja, qualquer comando gerado no diretório global de apelidos (que você pode ter adicionado à sua variável de ambiente `PATH`) pode ser usado em uma shebang, mesmo que não esteja no seu `PATH`. Isso permite o uso de shebangs como `/usr/bin/python3.12` para selecionar um determinado ambiente de execução.

Se nenhum ambiente de execução estiver instalado ou se a instalação automática estiver ativada, o ambiente de execução solicitado será instalado, se necessário. Consulte *Configuração* para obter informações sobre as definições de configuração.

The `/usr/bin/env` form of shebang line will also search the `PATH` environment variable for unrecognized commands. This corresponds to the behaviour of the Unix `env` program, which performs the same search, but prefers launching known Python commands. A warning may be displayed when searching for arbitrary executables, and this search may be disabled by the `shebang_can_run_anything` configuration option.

Shebang lines that do not match any of patterns are treated as *Windows* executable paths that are absolute or relative to the directory containing the script file. This is a convenience for Windows-only scripts, such as those generated by an installer, since the behavior is not compatible with Unix-style shells. These paths may be quoted, and may include multiple arguments, after which the path to the script and any additional arguments will be appended. This functionality may be disabled by the `shebang_can_run_anything` configuration option.

Nota

The behaviour of shebangs in the Python install manager is subtly different from the previous `py.exe` launcher, and the old configuration options no longer apply. If you are specifically reliant on the old behaviour or configuration, we recommend keeping the legacy launcher. It may be downloaded independently and installed on its own. The legacy launcher's `py` command will override PyManager's one, and you will need to use `pymanager` commands for installing and uninstalling.

4.1.10 Advanced Installation

For situations where an MSIX cannot be installed, such as some older administrative distribution platforms, there is an MSI available from the python.org downloads page. This MSI has no user interface, and can only perform per-machine installs to its default location in Program Files. It will attempt to modify the system `PATH` environment variable to include this install location, but be sure to validate this on your configuration.

> **Nota**
>
> Windows Server 2019 is the only version of Windows that CPython supports that does not support MSIX. For Windows Server 2019, you should use the MSI.

Esteja ciente que o pacote MSI não inclui nenhum ambiente de execução e, portanto, não é adequado para instalações em ambientes off-line sem criar também um indíce para instalação off-line. Consulte *Instalações offline* e *Administrative Configuration* para obter informações sobre como lidar com esses cenários.

Ambientes de execução instalados pelo MSI são compartilhados com os instalados pelo MSIX e são todos apenas por usuário. O gerenciador de instalação do Python não permite a instalação de ambientes de execução por máquina. Para emular uma instalação por máquina, você pode usar o `py install --target=<shared location>` como administrador e adicionar suas próprias modificações em todo o sistema ao `PATH`, ao registro ou ao menu Iniciar.

When the MSIX is installed, but commands are not available in the `PATH` environment variable, they can be found under `%LocalAppData%\Microsoft\WindowsApps\PythonSoftwareFoundation.PythonManager_3847v3x7pw1km` or `%LocalAppData%\Microsoft\WindowsApps\PythonSoftwareFoundation.PythonManager_qbz5n2kfra8p0`, depending on whether it was installed from python.org or through the Windows Store. Attempting to run the executable directly from Program Files is not recommended.

To programmatically install the Python install manager, it is easiest to use WinGet, which is included with all supported versions of Windows:

```
$> winget install 9NQ7512CXL7T -e --accept-package-agreements --disable-
↪interactivity

# Optionally run the configuration checker and accept all changes
$> py install --configure -y
```

To download the Python install manager and install on another machine, the following WinGet command will download the required files from the Store to your Downloads directory (add `-d <location>` to customize the output location). This also generates a YAML file that appears to be unnecessary, as the downloaded MSIX can be installed by launching or using the commands below.

```
$> winget download 9NQ7512CXL7T -e --skip-license --accept-package-agreements --
↪accept-source-agreements
```

To programmatically install or uninstall an MSIX using only PowerShell, the Add-AppxPackage and Remove--AppxPackage PowerShell cmdlets are recommended:

```
$> Add-AppxPackage C:\Downloads\python-manager-25.0.msix
...
$> Get-AppxPackage PythonSoftwareFoundation.PythonManager | Remove-AppxPackage
```

The latest release can be downloaded and installed by Windows by passing the AppInstaller file to the Add--AppxPackage command. This installs using the MSIX on python.org, and is only recommended for cases where installing via the Store (interactively or using WinGet) is not possible.

```
$> Add-AppxPackage -AppInstallerFile https://www.python.org/ftp/python/pymanager/
↪pymanager.appinstaller
```

Other tools and APIs may also be used to provision an MSIX package for all users on a machine, but Python does not consider this a supported scenario. We suggest looking into the PowerShell Add-AppxProvisionedPackage cmdlet, the native Windows PackageManager class, or the documentation and support for your deployment tool.

Regardless of the install method, users will still need to install their own copies of Python itself, as there is no way to trigger those installs without being a logged in user. When using the MSIX, the latest version of Python will be available for all users to install without network access.

Note that the MSIX downloadable from the Store and from the Python website are subtly different and cannot be installed at the same time. Wherever possible, we suggest using the above WinGet commands to download the package from the Store to reduce the risk of setting up conflicting installs. There are no licensing restrictions on the Python install manager that would prevent using the Store package in this way.

4.1.11 Administrative Configuration

There are a number of options that may be useful for administrators to override configuration of the Python install manager. These can be used to provide local caching, disable certain shortcut types, override bundled content. All of the above configuration options may be set, as well as those below.

Configuration options may be overridden in the registry by setting values under `HKEY_LOCAL_MACHINE\Software\Policies\Python\PyManager`, where the value name matches the configuration key and the value type is `REG_SZ`. Note that this key can itself be customized, but only by modifying the core config file distributed with the Python install manager. We recommend, however, that registry values are used only to set `base_config` to a JSON file containing the full set of overrides. Registry key overrides will replace any other configured setting, while `base_config` allows users to further modify settings they may need.

Note that most settings with environment variables support those variables because their default setting specifies the variable. If you override them, the environment variable will no longer work, unless you override it with another one. For example, the default value of `confirm` is literally `%PYTHON_MANAGER_CONFIRM%`, which will resolve the variable at load time. If you override the value to `yes`, then the environment variable will no longer be used. If you override the value to `%CONFIRM%`, then that environment variable will be used instead.

Configuration settings that are paths are interpreted as relative to the directory containing the configuration file that specified them.

Administrative configuration options

Config Key	Descrição
`base_config`	The highest priority configuration file to read. Note that only the built-in configuration file and the registry can modify this setting.
`user_config`	The second configuration file to read.
`additional_config`	The third configuration file to read.
`registry_override_ke`	Registry location to check for overrides. Note that only the built-in configuration file can modify this setting.
`bundled_dir`	Read-only directory containing locally cached files.
`install.fallback_source`	Path or URL to an index to consult when the main index cannot be accessed.
`install.enable_shortcut_kinc`	Comma-separated list of shortcut kinds to allow (e.g. `"pep514,start"`). Enabled shortcuts may still be disabled by `disable_shortcut_kinds`.
`install.disable_shortcut_kir`	Comma-separated list of shortcut kinds to exclude (e.g. `"pep514,start"`). Disabled shortcuts are not reactivated by `enable_shortcut_kinds`.
`pep514_root`	Registry location to read and write PEP 514 entries into. By default, `HKEY_CURRENT_USER\Software\Python`.
`start_folder`	Start menu folder to write shortcuts into. By default, `Python`. This path is relative to the user's Programs folder.
`virtual_env`	Path to the active virtual environment. By default, this is `%VIRTUAL_ENV%`, but may be set empty to disable venv detection.
`shebang_can_run_anyt`	Verdadeiro para suprimir avisos visíveis quando um shebang executa uma aplicação que não seja um ambiente de execução Python.

4.1.12 Instalando binários com threads livres

Adicionado na versão 3.13: (Experimental)

Nota

Tudo descrito nesta seção é considerado experimental e espera-se que mude em versões futuras.

Pre-built distributions of the experimental free-threaded build are available by installing tags with the `t` suffix.

```
$> py install 3.14t
$> py install 3.14t-arm64
$> py install 3.14t-32
```

Ele será instalado e registrado normalmente. Se você não tiver outro ambiente de execução instalado, `python` iniciará este. Caso contrário, você precisará usar `py -V:3.14t ...` ou, se tiver adicionado o diretório global de apelidos à sua variável de ambiente `PATH`, o comando `python3.14t.exe`.

4.1.13 Troubleshooting

If your Python install manager does not seem to be working correctly, please work through these tests and fixes to see if it helps. If not, please report an issue at our bug tracker, including any relevant log files (written to your `%TEMP%` directory by default).

Troubleshooting

Symptom	Things to try
`python` gives me a "command not found" error or opens the Store app when I type it in my terminal.	Did you *install the Python install manager*?
	Click Start, open "Manage app execution aliases", and check that the aliases for "Python (default)" are enabled. If they already are, try disabling and re-enabling to refresh the command. The "Python (default windowed)" and "Python install manager" commands may also need refreshing.
	Check that the `py` and `pymanager` commands work.
`py` gives me a "command not found" error when I type it in my terminal.	Did you *install the Python install manager*?
	Click Start, open "Manage app execution aliases", and check that the aliases for "Python (default)" are enabled. If they already are, try disabling and re-enabling to refresh the command. The "Python (default windowed)" and "Python install manager" commands may also need refreshing.
`py` gives me a "can't open file" error when I type commands in my terminal.	This usually means you have the legacy launcher installed and it has priority over the Python install manager. To remove, click Start, open "Installed apps", search for "Python launcher" and uninstall it.
`python` não inicia o mesmo ambiente de execução que `py`	Clique em Iniciar, abra "Aplicativos instalados", procure por qualquer ambiente de execução do Python existente e remova-o ou modifique e desative as opções de `PATH`.
	Click Start, open "Manage app execution aliases", and check that your `python.exe` alias is set to "Python (default)"
`python` e `py` não iniciam o ambiente de execução que eu esperava	Check your *`PYTHON_MANAGER_DEFAULT`* environment variable or `default_tag` configuration. The `py list` command will show your default based on these settings.
	As instalações gerenciadas pelo gerenciador de instalação do Python serão escolhidas antes das instalações não gerenciadas. Use `py install` para instalar o ambiente de execução que você espera ou configure sua tag padrão.
	As instalações de pré-lançamento e experimentais que não são gerenciadas pelo gerenciador de instalação Python podem ser escolhidas antes das instalações estáveis. Configure sua tag padrão ou desinstale a versão do ambiente de execução de pré-lançamento e reinstale usando `py install`.
`pythonw` ou `pyw` não iniciam o mesmo ambiente de execução que `python` ou `py`	Click Start, open "Manage app execution aliases", and check that your `pythonw.exe` and `pyw.exe` aliases are consistent with your others.
`pip` gives me a "command not found" error when I type it in my terminal.	Have you activated a virtual environment? Run the `.venv\Scripts\activate` script in your terminal to activate.
	O pacote pode estar disponível, mas sem o executável gerado. Recomendamos usar o comando `python -m pip` ou, alternativamente, `python -m pip install --force pip` recriará os executáveis e mostrará o caminho a ser adicionado ao `PATH`. Esses scripts são separados para cada ambiente de execução e, portanto, talvez seja necessário adicionar vários caminhos.

4.2 O pacote embutível

Adicionado na versão 3.5.

A distribuição embutida é um arquivo ZIP contendo um ambiente Python mínimo. Ela foi criada para atuar como parte de outra aplicação, ao invés de ser diretamente acessada por usuários finais.

To install an embedded distribution, we recommend using `py install` with the `--target` option:

```
$> py install 3.14-embed --target=runtime
```

When extracted, the embedded distribution is (almost) fully isolated from the user's system, including environment variables, system registry settings, and installed packages. The standard library is included as pre-compiled and optimized `.pyc` files in a ZIP, and `python3.dll`, `python313.dll`, `python.exe` and `pythonw.exe` are all provided. Tcl/tk (including all dependents, such as Idle), pip and the Python documentation are not included.

A default `._pth` file is included, which further restricts the default search paths (as described below in *Encontrando módulos*). This file is intended for embedders to modify as necessary.

Pacotes de terceiros devem ser instalados pelo instalador da aplicação juntamente com a distribuição embutida. Usar o pip para gerenciar as dependências como em uma instalação regular do Python não é suportado nessa distribuição, apesar de que com algum cuidado pode ser possível incluir e usar o pip para atualizações automáticas. Em geral, pacotes de terceiros devem ser tratados como parte da aplicação ("vendoring") para que o desenvolvedor consiga garantir compatibilidade com versões mais recentes antes de fornecer atualizações para os usuários.

Os dois casos de uso recomendados para essa distribuição são descritos abaixo.

4.2.1 Aplicação Python

Uma aplicação escrita em Python não requer necessariamente que os usuários estejam cientes deste fato. A distribuição embutida pode ser usada neste caso para incluir uma versão privada do pacote de instalação do Python. Dependendo de quão transparente deve ser (ou pelo contrário, quão profissional deve parecer), existem duas opções.

Usar um executável especializado como inicializador requer um pouco de código, mas fornece a experiência mais transparente para os usuários. Com um inicializador personalizado, não existem indicações óbvias que o programa está rodando em Python: ícones podem ser personalizados, informações da empresa e versão podem ser especificadas, e associações de arquivo se comportam de forma apropriada. Na maioria dos casos, um inicializador personalizado deve simplesmente ser capaz de chamar `Py_Main` com uma linha de comando predefinida (hard-coded).

A abordagem mais simples é fornecer um arquivo batch ou um atalho gerado que chama diretamente o `python.exe` ou `pythonw.exe` com os argumentos de linha de comando necessários. Neste caso, a aplicação irá aparecer como Python e não seu nome real, e os usuários podem ter problemas em distinguir ela de outros processos ou associações de arquivo em Python.

Com a abordagem anterior, pacotes devem ser instalados como diretórios juntamente do executável do Python para garantir que estarão disponíveis no caminho do ambiente. Com o instalador especializado, pacotes podem ser armazenados em outras localidades já que há uma oportunidade de especificar o caminho de busca antes de executar a aplicação.

4.2.2 Incorporando Python

Aplicações escritas em código nativo frequentemente precisam de alguma forma de linguagem de script, e a distribuição embutida do Python pode ser usada para esse propósito. Em geral, a maior parte da aplicação é em código nativo, e uma parte irá ou invocar `python.exe` ou usar diretamente `python3.dll`. Para ambos os casos, extrair a distribuição embutida em um subdiretório da instalação da aplicação é suficiente para providenciar um interpretador Python carregável.

Para uso da aplicação, pacotes podem ser instalados em qualquer local já que há uma oportunidade de especificar caminhos de busca antes de inicializar o interpretador. De outra forma, não existem diferenças fundamentais entre usar a distribuição embutida ou a instalação regular.

4.3 Os pacotes nuget.org

Adicionado na versão 3.5.2.

O pacote nuget.org é um ambiente Python de tamanho reduzido criado para uso em integração contínua e construção de sistemas que não precisam de uma instalação de Python por todo o sistema da máquina. Enquanto nuget é o "gerenciador de pacotes para .NET", ele também funciona perfeitamente bem para pacotes contendo ferramentas em tempo de construção.

Visite nuget.org para informações mais atualizadas sobre utilização do nuget. A seguir está um sumário que é suficiente para desenvolvedores Python.

A ferramenta de linha de comando `nuget.exe` pode ser baixada diretamente de `https://aka.ms/nugetclidl`, por exemplo, usando o curl ou PowerShell. Com a ferramenta, a versão mais recente do Python para máquinas 64-bit ou 32-bit é instalada usando:

```
nuget.exe install python -ExcludeVersion -OutputDirectory .
nuget.exe install pythonx86 -ExcludeVersion -OutputDirectory .
```

Para selecionar uma versão específica, adicione `-Version 3.x.y`. O diretório de saída pode ser mudado de `.`, e o pacote será instalado em um subdiretório. Por padrão, o subdiretório é nomeado igual ao pacote, e sem a opção `-ExcludeVersion` esse nome irá incluir a versão específica instalada. Dentro do subdiretório está um diretório `tools` que contém a instalação do Python:

```
# Sem -ExcludeVersion
> .\python.3.5.2\tools\python.exe -V
Python 3.5.2

# Com -ExcludeVersion
> .\python\tools\python.exe -V
Python 3.5.2
```

Em geral, pacotes nuget não são atualizáveis, e novas versões devem ser instaladas lado-a-lado e referenciadas usando o caminho completo. Alternativamente, delete o diretório do pacote manualmente e instale novamente. Muitos sistemas CI irão fazer isso automaticamente se eles não preservam arquivos entre construções de projetos.

Juntamente com o diretório `tools` está o diretório `build\native`. Ele contém um arquivo de propriedades MSBuild `python.props` que pode ser usado em um projeto C++ para referenciar a instalação do Python. Incluir as configurações irá automaticamente usar os cabeçalhos e importar as bibliotecas na sua construção de projeto.

As páginas de informação dos pacotes em nuget.org são www.nuget.org/packages/python para a versão 64 bits, www.nuget.org/packages/pythonx86 para a versão 32 bits e www.nuget.org/packages/pythonarm64 para a versão ARM64

4.3.1 Pacotes com threads livres

Adicionado na versão 3.13: (Experimental)

> **Nota**
>
> Tudo descrito nesta seção é considerado experimental e espera-se que mude em versões futuras.

Pacotes contendo binários com threads livres são chamados python-freethreaded para a versão de 64 bits, pythonx86--freethreaded para a versão de 32 bits e pythonarm64-freethreaded para a versão ARM64. Esses dois pacotes contêm os pontos de entrada `python3.13t.exe` e `python.exe`, os quais são executados com threads livres.

4.4 Pacotes Alternativos

À parte da distribuição padrão CPython, existem pacotes modificados incluindo funcionalidades adicionais. A seguir está uma lista de versões populares e seus recursos chave:

ActivePython
: Instalador com compatibilidade multi-plataforma, documentação, PyWin32

Anaconda
: Módulos científicos populares (como o numpy, scipy e pandas) e o gerenciador de pacotes `conda`.

Enthought Deployment Manager
: "A próxima geração de ambiente Python e gerenciador de pacotes".

  Anteriormente, a Enthought fornecia Canopy, mas este chegou ao fim de vida em 2016.

WinPython
: Distribuição específica do Windows com pacotes científicos pré-construídos e ferramentas para construir pacotes.

Note que esses pacotes podem não incluir as últimas versões do Python ou outras bibliotecas, e não são mantidos ou suportados pelo time do núcleo do Python.

4.5 Supported Windows versions

As specified in **PEP 11**, a Python release only supports a Windows platform while Microsoft considers the platform under extended support. This means that Python 3.14 supports Windows 10 and newer. If you require Windows 7 support, please install Python 3.8. If you require Windows 8.1 support, please install Python 3.12.

4.6 Removendo a Limitação do MAX_PATH

O Windows historicamente tem limitado os comprimentos dos caminhos de arquivos em 260 caracteres. Isso significava que caminhos maiores que isso não seriam resolvidos e resultariam em erros.

In the latest versions of Windows, this limitation can be expanded to over 32,000 characters. Your administrator will need to activate the "Enable Win32 long paths" group policy, or set `LongPathsEnabled` to `1` in the registry key `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\FileSystem`.

Isso permite que a função `open()`, o módulo `os` e a maior parte das outras funcionalidades de caminho aceitem e retornem caminhos maiores que 260 caracteres quando usando strings.

After changing the above option and rebooting, no further configuration is required.

4.7 Modo UTF-8

Adicionado na versão 3.7.

O Windows ainda usa codificações legadas para a codificação do sistema (a Página de Código ANSI). O Python usa-o para a codificação padrão de arquivos de texto (por exemplo, `locale.getencoding()`).

Isso pode causar problemas, porque o UTF-8 é amplamente usado na Internet e na maioria dos sistemas Unix, incluindo o WSL (Subsistema Windows para Linux).

Você pode usar o Modo UTF-8 do Python para alterar a codificação de texto padrão para UTF-8. Você pode habilitar Modo UTF-8 do Python através da opção de linha de comando `-X utf8` ou da variável de ambiente `PYTHONUTF8=1`. Veja *`PYTHONUTF8`* para habilitar o modo UTF-8, e *Gerenciador de instalação do Python* para saber como modificar variáveis de ambiente.

Quando o Modo UTF-8 do Python está habilitado, você ainda pode usar a codificação do sistema (a página de código ANSI) através do codec "mbcs".

Observe que adicionar `PYTHONUTF8=1` às variáveis de ambiente padrão afetará todas as aplicações Python 3.7+ em seu sistema. Se você tiver aplicações Python 3.7+ que dependem da codificação do sistema legado, é recomendável definir a variável de ambiente temporariamente ou usar a opção de linha de comando `-X utf8`.

Nota

Mesmo quando o modo UTF-8 está desativado, o Python usa o UTF-8 por padrão no Windows para:

- E/S do console, incluindo E/S padrão (consulte **PEP 528** para detalhes).
- A *codificação do sistema de arquivos* (veja a **PEP 529** para detalhes).

4.8 Encontrando módulos

Estas notas complementam a descrição em sys-path-init com notas detalhadas do Windows.

Quando nenhum arquivo `._pth` é encontrado, assim é como `sys.path` é populado no Windows:

- Uma entrada em branco é adicionada ao início, que corresponde ao diretório atual.
- Se a variável de ambiente *PYTHONPATH* existe, como descrito em *Variáveis de ambiente*, suas entradas são adicionadas em seguida. Note que no Windows, caminhos nessa variável devem ser separados por ponto e vírgula, para distinguir eles dos dois pontos usados nos identificadores de drivers (`C:\` etc.).
- "Caminhos da aplicação" adicionais podem ser adicionados ao registro como subchaves de `\SOFTWARE\Python\PythonCore{version}\PythonPath` sob ambas `HKEY_CURRENT_USER` e `HKEY_LOCAL_MACHINE`. Subchaves que possuem string de caminhos delimitados por ponto e vírgula como seu valor padrão farão com que cada caminho seja adicionado ao `sys.path`. (Note que todos os instaladores conhecidos usam apenas HKLM, portanto HKCU está tipicamente vazio.)
- Se a variável de ambiente *PYTHONHOME* está definida, ela é presumida como "Python Home". Caso contrário, o caminho do principal executável do Python é usado para localizar um "arquivo de referência" (ou `Lib\os.py` ou `pythonXY.zip`) para deduzir o "Python Home". Se um Python Home é encontrado, os subdiretórios relevantes adicionados ao `sys.path` (`Lib`, `plat-win`, etc) são baseados naquela pasta. Se não, o caminho núcleo do Python é construído a partir do PythonPath armazenado no registro.
- Se o Python Home não puder ser localizado, nenhum *PYTHONPATH* está especificado no ambiente, e nenhuma entrada de registro pôde ser encontrada, um caminho padrão com entradas relativas é usado (por exemplo, `.\Lib;.\plat-win`, etc).

Se um arquivo `pyvenv.cfg` for encontrado juntamente com o principal executável ou no diretório um nível acima do executável, as seguintes variações se aplicam:

- Se `home` é um caminho absoluto e *PYTHONHOME* não está definido, o caminho é usado ao invés do caminho ao principal executável quando deduzindo a localização do Home.

O resultado final de tudo isso é:

- Quando executando `python.exe`, ou qualquer outro .exe no diretório principal do Python (ou uma versão instalada, ou diretamente do diretório PCbuild), o caminho núcleo é deduzido, e os caminhos núcleo no registro são ignorados. Outros "caminhos da aplicação" no registro são sempre lidos.
- Quando Python é hospedado em outro .exe (diretório diferente, embutido via COM, etc), o "Python Home" não será deduzido, então o caminho núcleo do registro é usado. Outros "caminhos da aplicação" no registro sempre são lidos.
- Se o Python não puder encontrar seu Home e não existem valores no registro (.exe imutáveis, algumas configurações de instalação bem estranhas) você recebe um caminho com alguns caminhos padrão, porém relativos.

Para aqueles que querem empacotar o Python em suas aplicações ou distribuições, o seguinte conselho irá prevenir conflitos com outras instalações:

- Inclua um arquivo `._pth` juntamente do executável contendo os diretórios a serem incluídos. Isso irá ignorar caminhos listados no registro e variáveis de ambiente, e também ignorar `sites` a não ser que `import site` seja listado.
- Se você estiver carregando `python3.dll` ou `python37.dll` em seu próprio executável, defina explicitamente `PyConfig.module_search_paths` antes de `Py_InitializeFromConfig()`.
- Limpe e/ou sobrescreva *`PYTHONPATH`* e defina *`PYTHONHOME`* antes de iniciar o `python.exe` a partir da sua aplicação.
- Se você não puder usar as sugestões anteriores (por exemplo, você é uma distribuição que permite que as pessoas executem o arquivo `python.exe` diretamente), certifique-se de que o arquivo de referência (`Lib\os.py`) existe no seu diretório de instalação. (Note que ele não será detectado dentro de um arquivo ZIP, mas um arquivo ZIP corretamente nomeado será detectado ao invés disso.)

Isso irá garantir que seus arquivos em uma instalação global no sistema não terão precedência sobre uma cópia da biblioteca padrão empacotada com a sua aplicação. Caso contrário, seus usuários podem experienciar problemas usando sua aplicação. Note que a primeira sugestão é a melhor, pois as outras podem ainda estar suscetíveis a caminhos não-padrão no registro e no site-packages do usuário.

Alterado na versão 3.6: Adiciona suporte a arquivos `._pth` e remove a opção `applocal` do `pyvenv.cfg`.

Alterado na versão 3.6: Adiciona `pythonXX.zip` como uma possível referência quando diretamente adjacente ao executável.

Descontinuado desde a versão 3.6: Módulos especificados no registro sob `Modules` (não `PythonPath`) podem ser importados por `importlib.machinery.WindowsRegistryFinder`. Este localizador está ativo no Windows no 3.6.0 e anteriores, mas pode precisar ser explicitamente adicionado ao `sys.meta_path` no futuro.

4.9 Módulos adicionais

Mesmo que o Python tenha como objetivo ser portável através de todas as plataformas, existem recursos que são únicos para o Windows. Alguns módulos, em ambas as bibliotecas padrão e externa, e trechos de código existem para usar esses recursos.

Os módulos padrão específicos para Windows estão documentados em mswin-specific-services.

4.9.1 PyWin32

O módulo PyWin32 de Mark Hammond é uma coleção de módulos para suporte avançado específico para Windows. Isso inclui utilitários para:

- Component Object Model (COM)
- Chamadas à API Win32
- Registro
- Log de Eventos
- Interfaces de usuário Microsoft Foundation Classes (MFC)

PythonWin é uma aplicação MFC de exemplo enviada com o PyWin32. É uma IDE embutível com um depurador embutido.

Ver também

Win32 How Do I…?
por Tim Golden

Python e COM
by David and Paul Boddie

4.9.2 cx_Freeze

cx_Freeze encapsula scripts Python em programas executáveis do Windows (arquivos `*.exe`). Quando você tiver feito isso, você pode distribuir sua aplicação sem solicitar que os seus usuários instalem o Python.

4.10 Compilando Python no Windows

Se você quer compilar o CPython por conta própria, a primeira coisa que você precisa ter é a fonte. Você pode fazer o download tanto da fonte da última versão quanto pegar um novo checkout.

A árvore de fontes contém uma solução de construção e arquivos de projeto para o Microsoft Visual Studio, que é o compilador usado para construir as versões do Python oficiais. Esses arquivos estão no diretório `PCbuild`.

Confira o `PCbuild/readme.txt` para informações gerais sobre o processo de construção.

Para módulos de extensão, consulte building-on-windows.

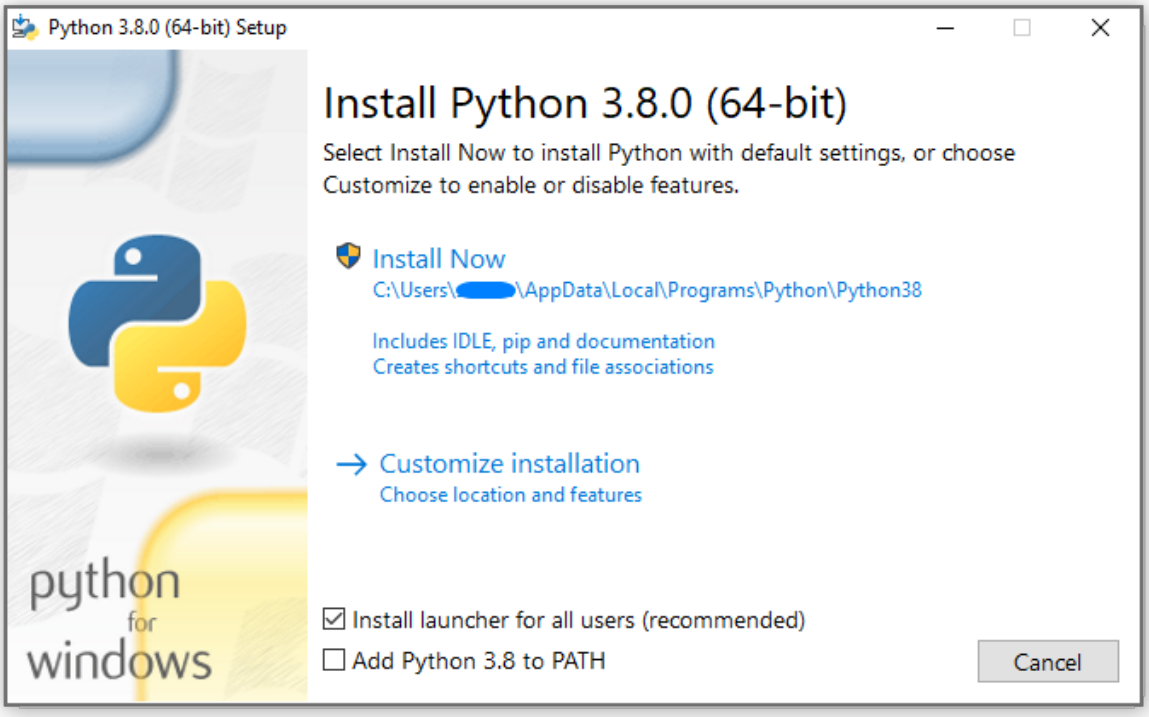
4.11 The full installer (deprecated)

Descontinuado desde a versão 3.14: This installer is deprecated since 3.14 and will not be produced for Python 3.16 or later. See *Gerenciador de instalação do Python* for the modern installer.

4.11.1 Etapas de instalação

Quatro instaladores Python 3.14 estão disponíveis para download - dois de cada para as versões 32-bit e 64-bit do interpretador. O *instalador web* é um download inicial menor, e ele irá automaticamente fazer o download dos componentes solicitados na medida do necessário. O *instalador offline* inclui os componentes necessários para uma instalação padrão e requer apenas uma conexão de internet para recursos opcionais. Veja *Instalando Sem Download* para outras formas de evitar o download durante a instalação.

Após iniciar o instalador, uma de duas opções deve ser selecionada:



Se você selecionar “Install Now”:

- Você *não* precisará ser um administrador (a menos que uma atualização de sistema para a Biblioteca em Tempo de Execução C seja necessária ou que você instale o *Gerenciador de instalação do Python* para todos os usuários)
- Python será instalado dentro do seu diretório de usuário
- O *Gerenciador de instalação do Python* será instalado de acordo com a opção ao final da primeira página
- A biblioteca padrão, a suíte de testes, o inicializador e o pip serão instalados
- Se selecionado, o diretório de instalação será adicionado no seu `PATH`
- Atalhos serão visíveis apenas para o usuário atual

Selecionando "Customize installation" irá permitir que você selecione os recursos a serem instalados, o local da instalação e outras opções ou ações pós-instalação. Para instalar símbolos de depuração ou binários, você precisará usar essa opção.

Para realizar uma instalação para todos os usuários, você deve selecionar "Customize installation". Neste caso:

- Você pode ser solicitado a providenciar credenciais administrativas ou aprovação
- Python será instalado dentro do diretório Program Files (Arquivos de Programa)
- O *Gerenciador de instalação do Python* será instalado dentro do diretório Windows
- Recursos opcionais podem ser selecionados durante a instalação
- A biblioteca padrão pode ser pré-compilada em bytecode
- Se selecionado, o diretório de instalação será adicionado ao `PATH` do sistema
- Atalhos estão disponíveis para todos os usuários

4.11.2 Removendo a Limitação do MAX_PATH

O Windows historicamente tem limitado os comprimentos dos caminhos de arquivos em 260 caracteres. Isso significava que caminhos maiores que isso não seriam resolvidos e resultariam em erros.

Nas últimas versões do Windows, essa limitação pode ser expandida para aproximadamente 32.000 caracteres. Seu administrador irá precisar ativar a política de grupo "Enable Win32 long paths", ou definir `LongPathsEnabled` para `1` na chave de registro `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\FileSystem`.

Isso permite que a função `open()`, o módulo `os` e a maior parte das outras funcionalidades de caminho aceitem e retornem caminhos maiores que 260 caracteres quando usando strings.

Após alterar a opção acima, nenhuma configuração adicional é necessária.

Alterado na versão 3.6: Suporte para caminhos longos foi ativado no Python.

4.11.3 Instalando sem UI

Todas as opções disponíveis na IU do instalador também podem ser especificadas a partir da linha de comando, permitindo que instaladores por script repliquem uma instalação em várias máquinas sem interação do usuário. Essas opções também podem ser definidas sem suprimir a IU para alterar alguns dos padrões.

As seguintes opções (encontradas executando o instalador com `/?`) podem ser passadas para o instalador:

Nome	Descrição
/passive	para exibir o progresso sem exigir interação do usuário
/quiet	para instalar/desinstalar sem exibir nenhuma UI
/simple	para evitar a personalização do usuário
/uninstall	para remover o Python (sem exigir confirmação)
/layout [diretório]	para pré-baixar todos os componentes
/log [nome-de-arquivo]	para especificar a localização dos arquivos de log

Todas as outras opções são passadas como `name=value`, onde o valor é usualmente `0` para desabilitar o recurso, `1` para ativar o recurso, ou um caminho. A lista completa de opções disponíveis é mostrada abaixo.

Nome	Descrição	Default (padrão)
InstallAllUsers (Instalar para Todos usuários)	Execute uma instalação em todo o sistema.	0
TargetDir (Diretório Alvo)	O diretório de instalação	Selecionado com base em "Instalar para Todos os Usuários"
DefaultAllUsersTargetDir (Diretório de Destino Padrão para Todos os usuários)	O diretório de instalação padrão para instalações de todos os usuários	`%ProgramFiles%\Python X.Y` or `%ProgramFiles(x86)%\Python X.Y`
DefaultJustForMeTargetDir (Diretório Alvo Padrão Apenas Para Mim)	O diretório de instalação padrão para instalações just-for-me	`%LocalAppData%\Programs\Python\PythonXY` ou `%LocalAppData%\Programs\Python\PythonXY-32` ou `%LocalAppData%\Programs\Python\PythonXY-64`
DefaultCustomTargetDir (Diretório de destino personalizado padrão)	Diretório de instalação personalizado padrão exibido na interface do usuário	(vazio)
AssociateFiles (Arquivos Associados)	Criar associações de arquivos se o launcher também estiver instalado.	1
CompileAll (Compilar Tudo)	Compile todos os arquivos `.py` para `.pyc`.	0
PrependPath (path a ser percorrido)	Anexa os diretórios de instalação e Scripts ao início de `PATH` e adiciona `.PY` a `PATHEXT`	0
AppendPath	Anexa os diretórios de instalação e Scripts ao final de `PATH` e adiciona `.PY` a `PATHEXT`	0
Shortcuts (atalhos)	Crie atalhos para o interpretador, documentação e IDLE se instalado.	1
Include_doc	Install Python manual (Instalação Manual do Python)	1
Include_debug (Incluir o Modo de Depuração)	Instalar binários de Depuração	0
Include_dev	Instala cabeçalhos e bibliotecas do desenvolvedor. Omitir isso pode levar a uma instalação inutilizável.	1
Include_exe	Instala `python.exe` e arquivos relacionados. Omitir isso pode levar a uma instalação inutilizável.	1
Include_launcher	Instalar *Gerenciador de instalação do Python*.	1
InstallLauncherAllUsers	Instala o iniciador para todos os usuários. Também requer `Include_launcher` para ser definido como 1	1
Include_lib	Instala a biblioteca padrão e os módulos de extensão. Omitir isso pode levar a uma instalação inutilizável.	1
Include_pip	Instale o pacote pip e setuptools	1
Include_symbols	Instala símbolos de depuração (`*.pdb`)	0
Include_tcltk	Instale o suporte Tcl/Tk e o IDLE	1
Include_test	Instalar o conjunto de testes da biblioteca padrão	1
Include_tools	Instalar scripts com utilitários	1
LauncherOnly	Instala apenas o launcher. Isso	0

Por exemplo, para instalar silenciosamente uma instalação de Python padrão e em todo o sistema, você pode usar o seguinte comando (a partir de um terminal de comando autorizado):

```
python-3.9.0.exe /quiet InstallAllUsers=1 PrependPath=1 Include_test=0
```

Para permitir que usuários instalem facilmente uma cópia do Python sem a suíte de testes, você pode fornecer um atalho com o seguinte comando. Isso irá exibir uma página inicial simplificado e bloquear a personalização:

```
python-3.9.0.exe InstallAllUsers=0 Include_launcher=0 Include_test=0
    SimpleInstall=1 SimpleInstallDescription="Just for me, no test suite."
```

(Observe que omitir o inicializador também omite associações de arquivos, e só é recomendado para instalações por usuários quando também existe uma instalação por todo o sistema que inclui o inicializador.)

As opções listadas acima também podem ser fornecidas em um arquivo chamado `unattend.xml` juntamente com o executável. Esse arquivo especifica uma lista de opções e valores. Quando um valor é fornecido como um atributo, ele será convertido para um número se possível. Valores fornecidos como elementos de texto são sempre deixados como strings. Esse arquivo de exemplo define as mesmas opções que o exemplo anterior:

```
<Options>
    <Option Name="InstallAllUsers" Value="no" />
    <Option Name="Include_launcher" Value="0" />
    <Option Name="Include_test" Value="no" />
    <Option Name="SimpleInstall" Value="yes" />
    <Option Name="SimpleInstallDescription">Just for me, no test suite</Option>
</Options>
```

4.11.4 Instalando Sem Download

Como alguns recursos do Python não estão inclusos no download inicial do instalador, selecionar esses recursos pode exigir uma conexão com a internet. Para evitar isso, todos os possíveis componentes podem ser baixados sob demanda para criar um *layout* completo que não irá precisar de uma conexão de internet independentemente dos recursos selecionados. Note que este download pode ser maior que o necessário, mas onde um grande número de instalações serão realizadas é bem útil ter uma cópia em cache local.

Execute o seguinte comando a partir do Prompt de Comando para fazer o download de todos os possíveis arquivos necessários. Lembre-se de substituir `python-3.9.0.exe` pelo nome real do seu instalador, e de criar layouts nos respectivos diretórios para evitar colisão entre arquivos com o mesmo nome.

```
python-3.9.0.exe /layout [diretório alvo opcional]
```

Você também pode especificar a opção `/quiet` para esconder o acompanhamento de progresso.

4.11.5 Modificando uma instalação

Uma vez que o Python foi instalado, você pode adicionar ou remover recursos através da ferramenta Programas e Recursos que é parte do Windows. Selecione o registro do Python e escolha "Uninstall/Change" para abrir o instalador no modo de manutenção.

"Modify" permite que você adicione ou remova recursos modificando as caixas de entrada - caixas de entrada não modificadas não irão instalar ou remover nada. Algumas opções não podem ser modificadas dessa forma, como o diretório de instalação; para modificá-las, você precisará remover e então reinstalar o Python completamente.

"Repair" irá verificar todos os arquivos que devem ser instalados usando as configurações atuais e substituir qualquer um que tiver sido removido ou modificado.

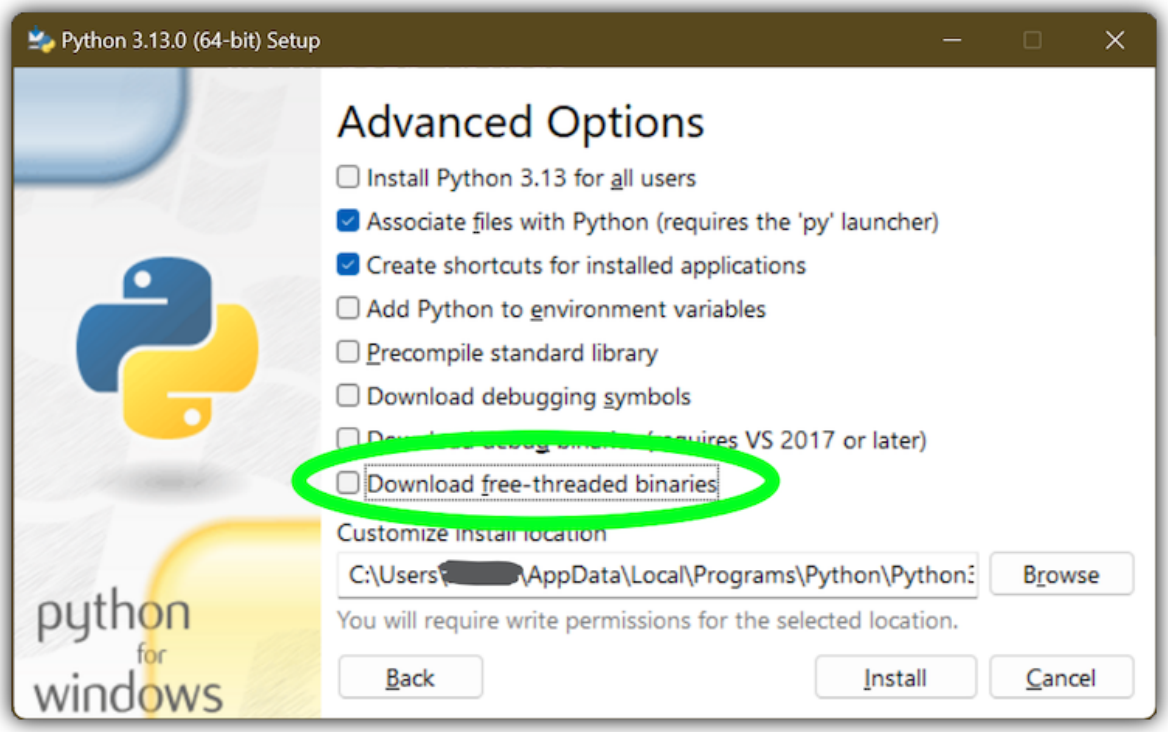
"Uninstall" irá remover o Python completamente, com a exceção do *Gerenciador de instalação do Python*, que tem seu próprio registro nos Programas e Recursos.

4.11.6 Instalando binários com threads livres

Adicionado na versão 3.13: (Experimental)

> **Nota**
>
> Tudo descrito nesta seção é considerado experimental e espera-se que mude em versões futuras.

Para instalar binários pré-construídos com recurso de threads livres habilitado (veja **PEP 703**), você deve selecionar "Customize installation" (Instalação personalizada). A segunda página de opções inclui a caixa de seleção "Download free-threaded binaries" (Baixar binários de threads livres).



Selecionar esta opção fará o download e instalará binários adicionais no mesmo local da instalação principal do Python. O executável principal é chamado `python3.13t.exe`, e outros binários recebem um sufixo `t` ou um sufixo ABI completo. Arquivos de origem do Python e dependências de terceiros empacotadas são compartilhadas com a instalação principal.

A versão com threads livres é registrada como uma instalação regular do Python com a tag `3.13t` (com um sufixo `-32` ou `-arm64` como normal para essas plataformas). Isso permite que as ferramentas o descubram e que o *Gerenciador de instalação do Python* ofereça suporte a `py.exe -3.13t`. Observe que o inicializador interpretará `py.exe -3` (ou um shebang `python3`) como "a instalação 3.x mais recente", que preferirá os binários com threads livres aos normais, enquanto `py.exe -3.13` não. Se você usar o estilo curto da opção, talvez prefira não instalar os binários com threads livres neste momento.

Para especificar a opção de instalação na linha de comando, use `Include_freethreaded=1`. Veja *Instalando Sem Download* para instruções sobre como baixar preventivamente os binários adicionais para instalação offline. As opções para incluir símbolos de depuração e binários também se aplicam às construções com threads livres.

Binários com threads livres também estão disponíveis *em nuget.org*.

4.12 Python Launcher for Windows (Deprecated)

Descontinuado desde a versão 3.14: The launcher and this documentation have been superseded by the Python Install Manager described above. This is preserved temporarily for historical interest.

Adicionado na versão 3.3.

O inicializador Python para Windows é um utilitário que auxilia na localização e execução de diferentes versões do Python. Ele permite que scripts (ou a linha de comando) indiquem uma preferência por uma versão do Python específica, e irá localizar e executar essa versão.

Ao contrário da variável `PATH`, o inicializador irá corretamente selecionar a versão mais apropriada do Python. Ele irá preferir instalações por usuário ao invés de instalações globais no sistema, e ordenará por versão da linguagem ao invés de usar a versão instalada mais recentemente.

O inicializador foi originalmente especificado na **PEP 397**.

4.12.1 Começando

Pela linha de comando

Alterado na versão 3.6.

Instalações globais no sistema do Python 3.3 ou posterior irão colocar o inicializador no seu `PATH`. O inicializador é compatível com todas as versões disponíveis do Python, então não importa qual versão está instalada. Para verificar se o inicializador está disponível, execute o seguinte comando no Prompt de Comando:

```
py
```

Você deve perceber que a última versão do Python que você tem é iniciada - ela pode ser fechada normalmente, e qualquer argumento da linha de comando adicional especificado será enviado diretamente para o Python.

Se você tem múltiplas versões do Python instaladas (por exemplo, 3.7 e 3.14) você deve ter notado que o Python 3.14 foi iniciado – para iniciar o Python 3.7, use o comando:

```
py -3.7
```

Se você quer a versão mais recente do Python 2 que você tem instalada, tente o comando:

```
py -2
```

Se você ver o seguinte erro, você não tem o inicializador instalado:

```
'py' is not recognized as an internal or external command,
operable program or batch file.
```

O comando:

```
py --list
```

exibe a(s) versão(ões) atualmente instalada(s) do Python.

O argumento `-x.y` é a forma abreviada do argumento `-V:Empresa/Tag`, que permite selecionar um ambiente de execução Python específico, incluindo aqueles que podem ter vindo de algum lugar diferente de python.org. Qualquer ambiente de execução registrado seguindo a **PEP 514** será detectável. O comando `--list` lista todos os ambientes de execução disponíveis usando o formato `-V:`.

Ao usar o argumento `-V:`, especificar a Empresa limitará a seleção aos ambientes de execução desse provedor, enquanto especificar apenas a Tag selecionará de todos os provedores. Observe que omitir a barra implica uma tag:

```
# Seleciona qualquer ambiente de execução sinalizado como '3.*'
py -V:3
```

(continua na próxima página)

(continuação da página anterior)

```
# Seleciona qualquer ambiente de execução lançado como 'PythonCore'
py -V:PythonCore/

# Seleciona o ambiente de execução Python 3 mais recente de PythonCore
py -V:PythonCore/3
```

A forma abreviada do argumento (`-3`) só seleciona a partir de versões principais do Python, e não de outras distribuições. No entanto, a forma mais longa (`-V:3`) selecionará qualquer um.

A Empresa é correspondida na string completa, sem distinção entre maiúsculas e minúsculas. A Tag corresponde a uma string completa ou a um prefixo, desde que o próximo caractere seja um ponto ou um hífen. Isso permite que `-V:3.1` corresponda a `3.1-32`, mas não a `3.10`. As tags são classificadas usando ordenação numérica (`3.10` é mais recente que `3.1`), mas são comparadas usando texto (`-V:3.01` não corresponde a `3.1`).

Ambientes virtuais

Adicionado na versão 3.5.

Se o inicializador é executado sem versão explícita do Python especificada, e um ambiente virtual (criado com o módulo da biblioteca padrão `venv` ou da ferramenta externa `virtualenv`) está ativo, o inicializador irá executar o interpretador do ambiente virtual ao invés do global. Para executar o interpretador global, ou desative o ambiente virtual, ou explicitamente especifique a versão global do Python.

Por um script

Vamos criar um script teste de Python - crie um arquivo chamado `hello.py` com os seguintes conteúdos:

```
#! python
import sys
sys.stdout.write("olá do Python %s\n" % (sys.version,))
```

A partir do diretório onde hello.py está, execute o comando:

```
py hello.py
```

Você deve notar que o número da versão da sua instalação mais recente do Python 2.x é exibido. Agora tente mudar a primeira linha para ser:

```
#! python3
```

Re-executar o comando agora deve exibir informações da última versão do Python 3.x. Como nos exemplos da linha de comando acima, você pode especificar um qualificador de versão mais explícito. Presumindo que você tem o Python 3.7 instalado, tente mudar a primeira linha para `#! python3.7` e você deve ver as informações da versão 3.7 sendo exibidas.

Note que diferentemente do uso interativo, um simples "python" irá usar a última versão do Python 2.x que você tem instalada. Isso é para retrocompatibilidade e para compatibilidade com Unix, onde o comando `python` tipicamente se refere ao Python 2.

Por associação de arquivos

O inicializador deve ter sido associado com arquivos Python (isto é, arquivos `.py`, `.pyw`, `.pyc`) quando foi instalado. Isso significa que quando você clica duas vezes em um desses arquivos a partir do Explorador de Arquivos do Windows o inicializador será usado, e assim você pode usar as mesmas facilidades descritas acima para que o script especifique qual versão deve ser usada.

O benefício chave disso é que um único inicializador pode suportar múltiplas versões do Python ao mesmo tempo dependendo dos conteúdos da primeira linha.

4.12.2 Linhas Shebang

Se a primeira linha de um arquivo de script começa com `#!`, ela é conhecida como linha "shebang". Linux e outros tipos de sistemas operacionais Unix têm suporte nativo para essas linhas e elas são comumente usadas nesses sistemas para indicar como um script deve ser executado. Esse inicializador permite que as mesmas facilidades sejam usadas com scripts Python no Windows e os exemplos acima demonstram seu uso.

Para permitir que linhas shebang em scripts Python sejam portáveis entre Unix e Windows, este inicializador suporta um número de comandos 'virtuais' para especificar qual interpretador deve ser usado. Os comandos virtuais suportados são:

- `/usr/bin/env`
- `/usr/bin/python`
- `/usr/local/bin/python`
- `python`

Por exemplo, se a primeira linha do seu script começa com

```
#! /usr/bin/python
```

O Python padrão ou um ambiente virtual ativo será localizado e utilizado. Como muitos scripts Python escritos para funcionar no Unix já terão essa linha, você deve perceber que esses scripts podem ser usados pelo inicializador sem modificação. Se você está escrevendo um novo script no Windows que você espera que seja útil no Unix, você deve usar uma dessas linhas shebang começando com `/usr`.

Qualquer um dos comandos virtuais acima pode ser sufixado com uma versão explícita (ou apenas a maior versão, ou a maior e a menor versão). Além disso, a versão de 32 bits pode ser solicitada adicionando "-32" após a menor versão. Isto é, `/usr/bin/python3.7-32` irá solicitar o uso do python 3.7 32-bit. Se um ambiente virtual estiver ativo, a versão será ignorada e o ambiente será usado.

Adicionado na versão 3.7: A partir do inicializador do python 3.7 é possível solicitar a versão 64-bit adicionando o sufixo "-64". Além disso é possível especificar uma versão maior e arquitetura sem a menor (isto é, `/usr/bin/python3-64`).

Alterado na versão 3.11: O sufixo "-64" foi descontinuado e agora implica "qualquer arquitetura que não seja comprovadamente i386/32 bits". Para solicitar um ambiente específico, use o novo argumento `-V:`*`TAG`* com a tag completa.

Alterado na versão 3.13: Comandos virtuais que referenciam `python` agora preferem um ambiente virtual ativo em vez de pesquisar `PATH`. Isso lida com casos em que o shebang especifica `/usr/bin/env python3`, mas `python3.exe` não está presente no ambiente ativo.

The `/usr/bin/env` form of shebang line has one further special property. Before looking for installed Python interpreters, this form will search the executable `PATH` for a Python executable matching the name provided as the first argument. This corresponds to the behaviour of the Unix `env` program, which performs a `PATH` search. If an executable matching the first argument after the `env` command cannot be found, but the argument starts with `python`, it will be handled as described for the other virtual commands. The environment variable `PYLAUNCHER_NO_SEARCH_PATH` may be set (to any value) to skip this search of `PATH`.

As linhas Shebang que não correspondem a nenhum desses padrões são procuradas na seção `[commands]` do arquivo *.INI* do lançador. Isso pode ser usado para lidar com certos comandos de uma maneira que faça sentido para o seu sistema. O nome do comando deve ser um único argumento (sem espaços no executável shebang) e o valor substituído é o caminho completo para o executável (argumentos adicionais especificados no .INI serão citados como parte do nome do arquivo).

```
[commands]
/bin/xpython=C:\Program Files\XPython\python.exe
```

Quaisquer comandos não encontrados no arquivo .INI são tratados como caminhos executáveis do **Windows** que são absolutos ou relativos ao diretório que contém o arquivo de script. Isso é uma conveniência para scripts somente do Windows, como aqueles gerados por um instalador, pois o comportamento não é compatível com shells de estilo Unix. Esses caminhos podem ser citados e podem incluir vários argumentos, após os quais o caminho para o script e quaisquer argumentos adicionais serão anexados.

4.12.3 Argumentos em linhas shebang

As linhas shebang também podem especificar opções adicionais a serem passadas ao interpretador Python. Por exemplo, se você tem uma linha shebang:

```
#! /usr/bin/python -v
```

O Python será iniciado com a opção `-v`

4.12.4 Personalização

Personalização via arquivos INI

Dois arquivos .ini serão pesquisados pelo inicializador - `py.ini` no diretório de dados de aplicações do usuário atual (`%LOCALAPPDATA%` ou `$env:LocalAppData`) e `py.ini` no mesmo diretório que o inicializador. Os mesmos arquivos .ini são usados para ambas a versão 'console' do inicializador (isto é, py.exe) e a versão 'windows' (isto é, pyw.exe).

Personalização especificada no "diretório da aplicação" terão precedência sobre àquela especificada junto do executável, portanto um usuário, que pode não ter acesso de escrita ao arquivo .ini junto do inicializador, pode sobrescrever comandos naquele arquivo .ini global.

Personalizando versões padrão do Python

Em alguns casos, um qualificador de versão pode ser incluído em um comando para ditar qual versão do Python deve ser usada pelo comando. Um qualificador de versão começa com um número maior de versão e pode opcionalmente ser seguido por um ponto ('.') e uma especificação de versão menor. Além disso, é possível especificar se uma implementação de 32 ou 64 bit deve ser solicitada adicionando "-32" ou "-64".

Por exemplo, uma linha shebang `#!python` não tem qualificador de versão, enquanto `#!python3` tem um qualificador de versão que especifica apenas uma versão maior.

If no version qualifiers are found in a command, the environment variable `PY_PYTHON` can be set to specify the default version qualifier. If it is not set, the default is "3". The variable can specify any value that may be passed on the command line, such as "3", "3.7", "3.7-32" or "3.7-64". (Note that the "-64" option is only available with the launcher included with Python 3.7 or newer.)

Se nenhum qualificador de versão menor é encontrado, a variável de ambiente `PY_PYTHON{major}` (onde `{major}` é o qualificador da versão maior atual, como determinado acima) pode ser definida para especificar a versão completa. Se nenhuma opção é encontrada, o inicializador irá enumerar as versões do Python instaladas e usar a última versão menor encontrada como versão maior, o que é provavelmente, ainda que não garantido, a versão instalada mais recentemente naquela família.

Em um Windows 64-bit com ambas as implementações de 32-bit e 64-bit da mesma (maior.menor) versão do Python instaladas, a versão 64-bit sempre será preferida. Isso será verdadeiro para ambas as implementações de 32-bit e 64-bit do inicializador - um inicializador 32-bit irá preferir executar uma instalação 64-bit do Python da versão especificada, se disponível. Isso é para que o comportamento do inicializador possa ser previsto sabendo apenas quais versões estão instaladas no PC e sem considerar a ordem com que elas foram instaladas (isto é, sem saber se a última a ser instalada foi a versão 32 ou 64-bit do Python e do instalador correspondente). Como observado acima, um sufixo opcional "-32" ou "-64" pode ser usado como especificador de versão para mudar esse comportamento.

Exemplos:

- Se nenhuma opção relevante for definida, os comandos `python` e `python2` irão usar a última versão instalada do Python 2.x e o comando `python3` irá usar o último Python 3.x instalado.
- O comando `python3.7` não vai consultar nenhuma opção já que as versões estão completamente especificadas.
- Se `PY_PYTHON=3`, os comandos `python` e `python3` irão ambos usar a última versão do Python 3 instalada.
- Se `PY_PYTHON=3.7-32`, o comando `python` irá usar a implementação 32 bits do 3.7 enquanto o comando `python3` irá usar o último Python instalado (PY_PYTHON não foi considerado porque uma versão maior foi especificada).

- Se `PY_PYTHON=3` e `PY_PYTHON3=3.7`, os comandos `python` e `python3` irão ambos usar especificamente 3.7

Em adição às variáveis de ambiente, as mesmas configurações podem ser feitas no arquivo .INI usado pelo inicializador. A seção nos arquivos INI é chamada `[defaults]` e o nome da chave será o mesmo que as variáveis de ambiente sem o prefixo `PY_` (e observe que os nomes das chaves no arquivo INI não diferem maiúsculas e minúsculas). Os conteúdos de uma variável de ambiente irão sobrescrever coisas especificadas em um arquivo INI.

Por exemplo:

- Configurar `PY_PYTHON=3.7` é o equivalente ao arquivo INI contendo:

```
[defaults]
python=3.7
```

- Configurar `PY_PYTHON=3` e `PY_PYTHON3=3.7` é o equivalente ao arquivo INI contendo:

```
[defaults]
python=3
python3=3.7
```

4.12.5 Diagnósticos

If an environment variable `PYLAUNCHER_DEBUG` is set (to any value), the launcher will print diagnostic information to stderr (i.e. to the console). While this information manages to be simultaneously verbose *and* terse, it should allow you to see what versions of Python were located, why a particular version was chosen and the exact command-line used to execute the target Python. It is primarily intended for testing and debugging.

4.12.6 Teste de simulação

If an environment variable `PYLAUNCHER_DRYRUN` is set (to any value), the launcher will output the command it would have run, but will not actually launch Python. This may be useful for tools that want to use the launcher to detect and then launch Python directly. Note that the command written to standard output is always encoded using UTF-8, and may not render correctly in the console.

4.12.7 Instalar sob demanda

If an environment variable `PYLAUNCHER_ALLOW_INSTALL` is set (to any value), and the requested Python version is not installed but is available on the Microsoft Store, the launcher will attempt to install it. This may require user interaction to complete, and you may need to run the command again.

An additional `PYLAUNCHER_ALWAYS_INSTALL` variable causes the launcher to always try to install Python, even if it is detected. This is mainly intended for testing (and should be used with `PYLAUNCHER_DRYRUN`).

4.12.8 Códigos de retorno

Os seguintes códigos de saída podem ser retornados pelo inicializador do Python. Infelizmente, não há como distingui-los do código de saída do próprio Python.

Os nomes dos códigos são como usados nas fontes e são apenas para referência. Não há como acessá-los ou resolvê-los além de ler esta página. As entradas são listadas em ordem alfabética de nomes.

Nome	Valor	Descrição
RC_BAD_VENV_CFG	107	Um `pyvenv.cfg` foi encontrado, mas está corrompido.
RC_CREATE_PROCES	101	Falha ao iniciar o Python.
RC_INSTALLING	111	Uma instalação foi iniciada, mas o comando precisará ser executado novamente após sua conclusão.
RC_INTERNAL_ERRO	109	Erro inesperado. Por favor, relate um bug.
RC_NO_COMMANDL	108	Não é possível obter a linha de comando do sistema operacional.
RC_NO_PYTHON	103	Não foi possível localizar a versão solicitada.
RC_NO_VENV_CFG	106	Um `pyvenv.cfg` era necessário, mas não foi encontrado.

CAPÍTULO 5

Usando Python em macOS

Este documento tem o objetivo de fornecer uma visão geral dos comportamentos específicos do macOS que você deve conhecer para começar a usar o Python em computadores Mac. O Python em um Mac com macOS é muito semelhante ao Python em outras plataformas derivadas do Unix, mas há algumas diferenças na instalação e alguns recursos.

Há várias maneiras de obter e instalar o Python para macOS. Versões pré-construídas das versões mais recentes do Python estão disponíveis em vários distribuidores. Grande parte deste documento descreve o uso dos Pythons fornecidos pela equipe de lançamento do CPython, disponíveis para download no site python.org. Consulte *Distribuições alternativas* para conhecer outras opções.

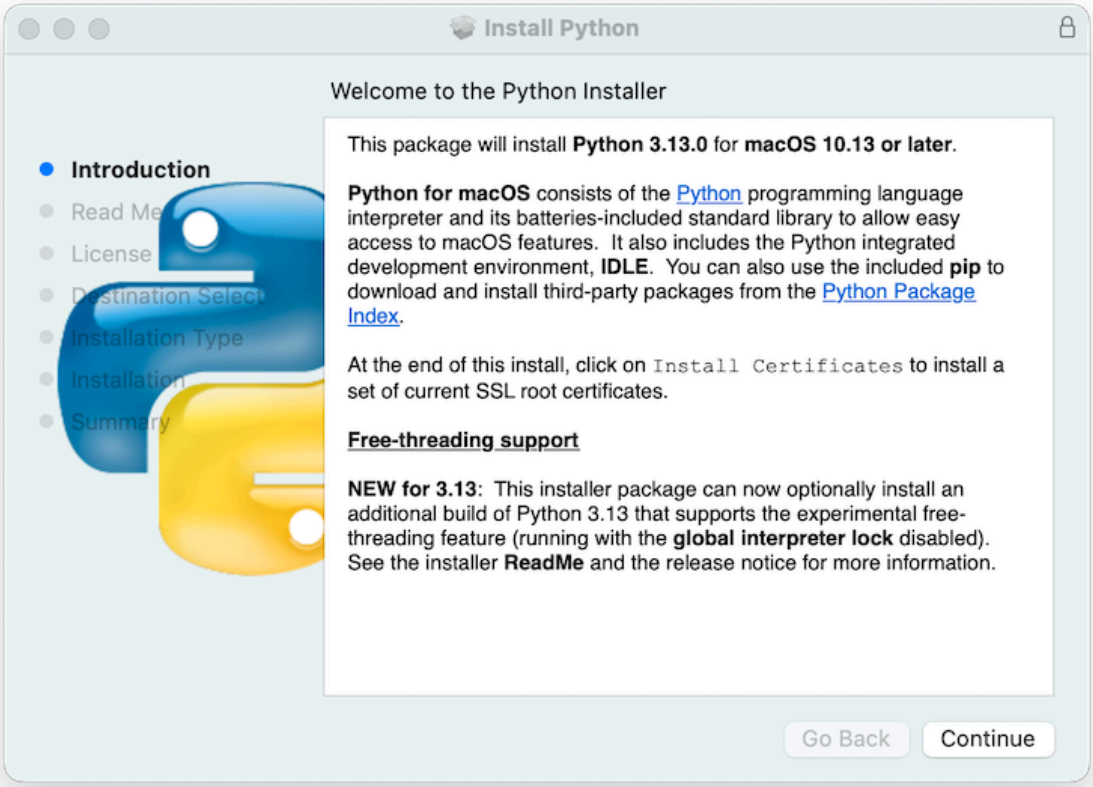
5.1 Usando Python para macOS do `python.org`

5.1.1 Etapas de instalação

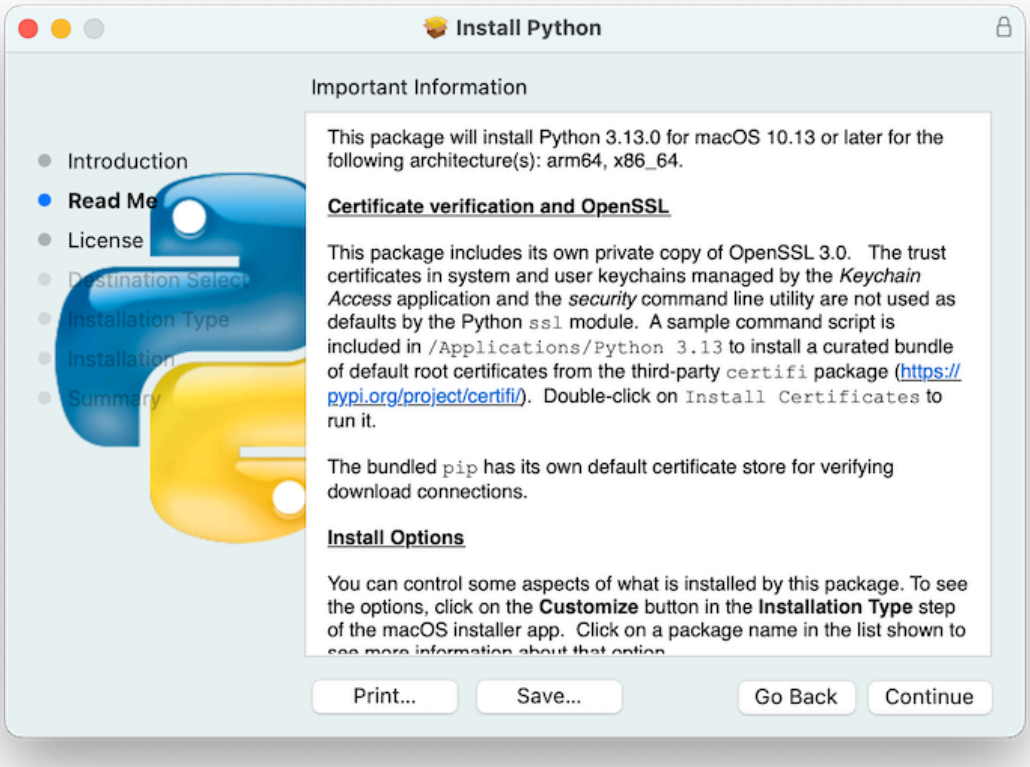
Para as versões atuais de Python (exceto as com status `security`), a equipe de lançamento produz um pacote de instalação **Python para macOS** em cada novo lançamento. Uma lista dos instaladores disponíveis está disponível aqui. Recomendamos usar a versão suportada mais recente de Python sempre que possível. Os instaladores atuais fornecem uma versão de Python como binário universal 2 que é executado nativamente em todos os Macs (Apple Silicon e Intel) compatíveis com uma ampla gama de versões do macOS, atualmente do **macOS 10.15 Catalina** em diante.

O arquivo baixado é um arquivo padrão do instalador de pacotes do macOS (`.pkg`). As informações de integridade do arquivo (soma de verificação, tamanho, assinatura de Sigstore, etc.) de cada arquivo estão inclusas na página de download dos lançamentos. Os pacotes de instalação e seus conteúdos são assinados e autenticados com certificados Apple Developer ID `Python Software Foundation` para atender aos requisitos do macOS Gatekeeper.

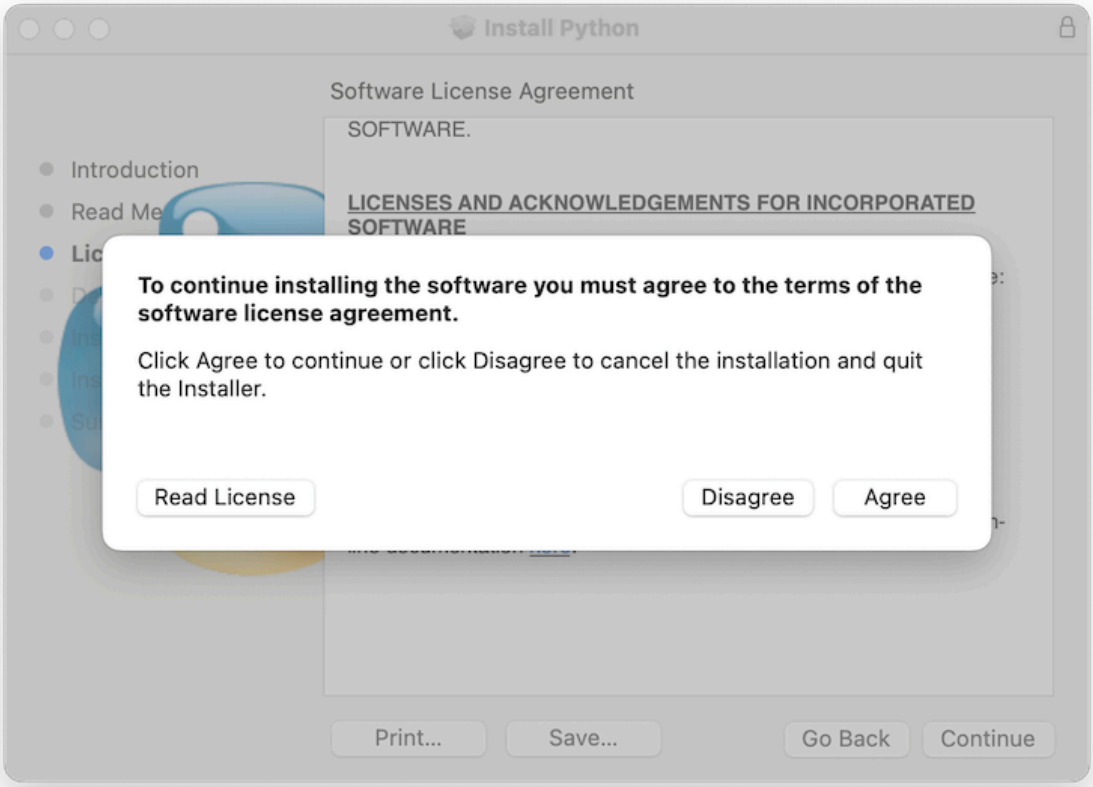
Para uma instalação padrão, clique duas vezes no arquivo instalador de pacotes baixado. Isso deverá abrir o aplicativo padrão de instalação do macOS e exibir o primeiro de vários passos de instalação.



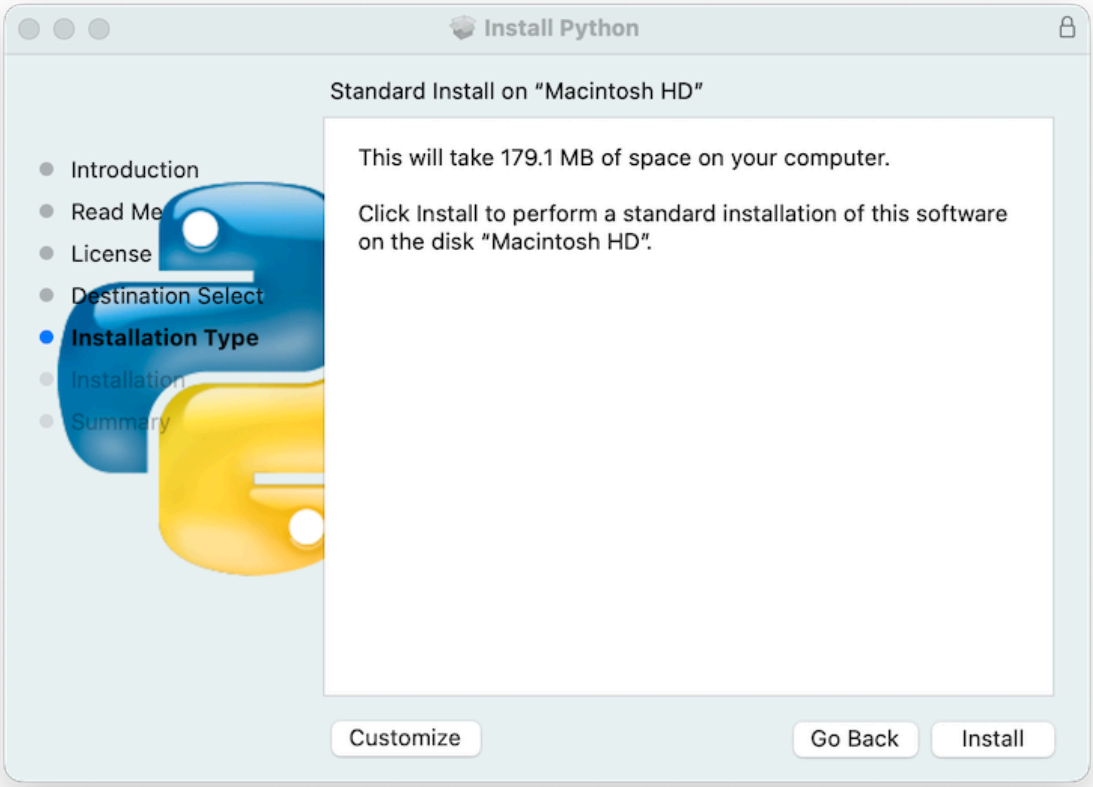
Ao clicar no botão **Continuar**, é exibido o **Read Me** desse instalador. Além de outras informações importantes, o **Read Me** documenta qual versão de Python será instalada e em quais versões do macOS ele é compatível. Talvez você precise rolar a tela para ler o arquivo inteiro. Por padrão, esse **Read Me** também será instalado em `/Applications/Python 3.14/` e estará disponível para leitura a qualquer momento.



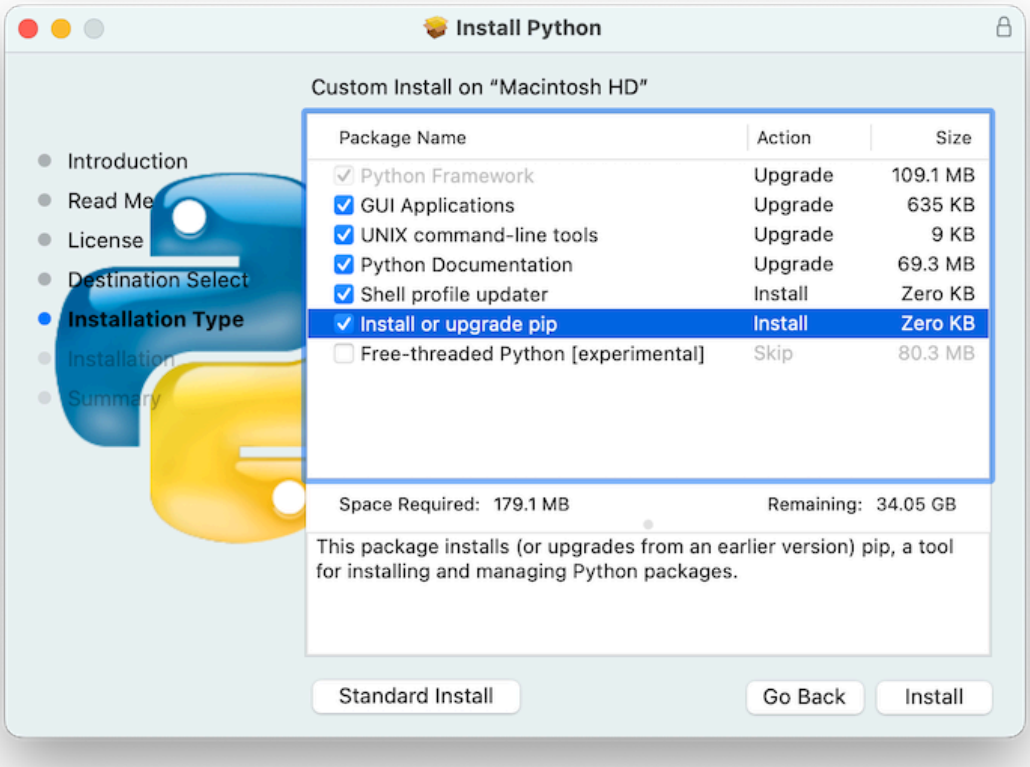
Ao clicar em **Continuar**, o instalador exibe a licença do Python e de outros softwares incluídos. Você precisará **Concordar** com os termos da licença antes de prosseguir para a próxima etapa. Esse arquivo de licença também será instalado e estará disponível para leitura a qualquer momento.



Após aceitar os termos da licença, a próxima etapa será a tela **Tipo de instalação**. Para a maioria dos usos, o conjunto padrão de operações de instalação é adequado.

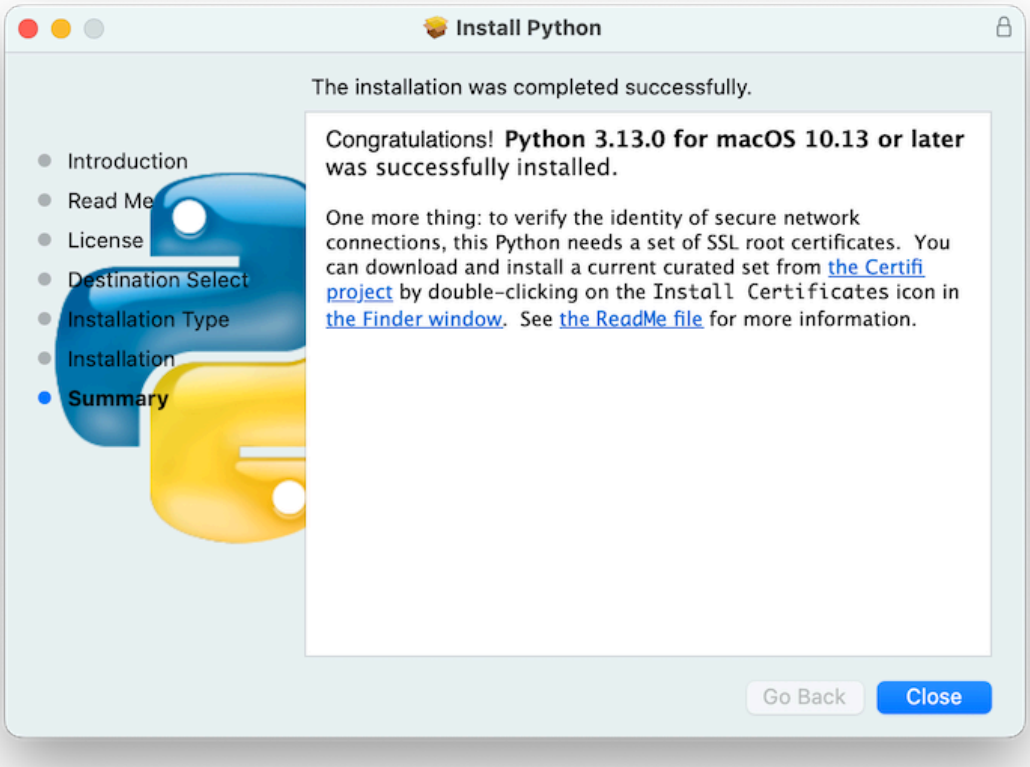


Ao pressionar o botão **Personalizar**, você pode optar por omitir ou selecionar determinados componentes de pacote do instalador. Clique em cada nome de pacote para ver uma descrição do que ele instala. Para também instalar o suporte para o recurso opcional de threads livres, consulte *Instalando binários com threads livres*.

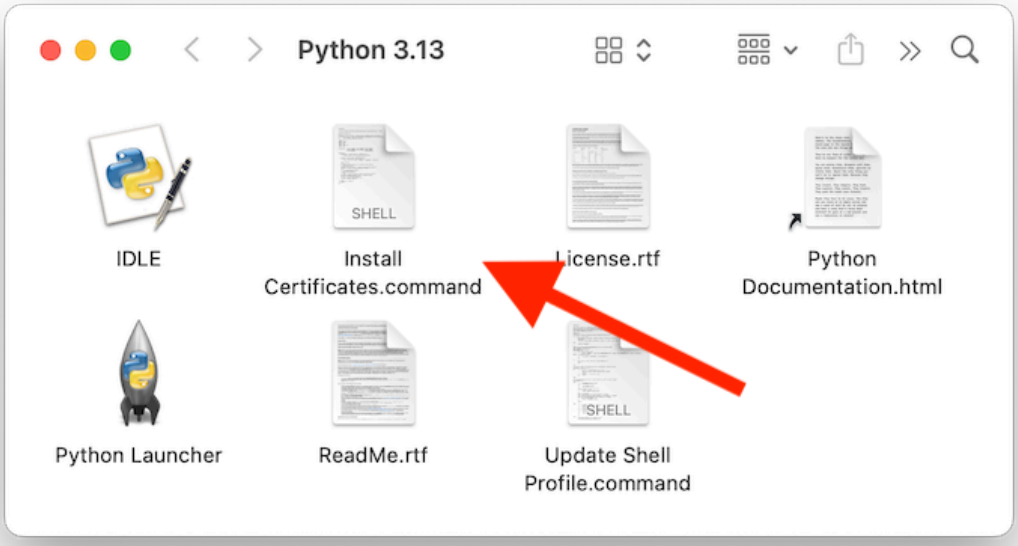


Em ambos os casos, clicar em **Instalar** iniciará o processo de instalação solicitando a permissão para instalar um novo software. É necessário um nome de usuário do macOS com privilégio de `Administrador`, pois o Python instalado estará disponível para todos os usuários do Mac.

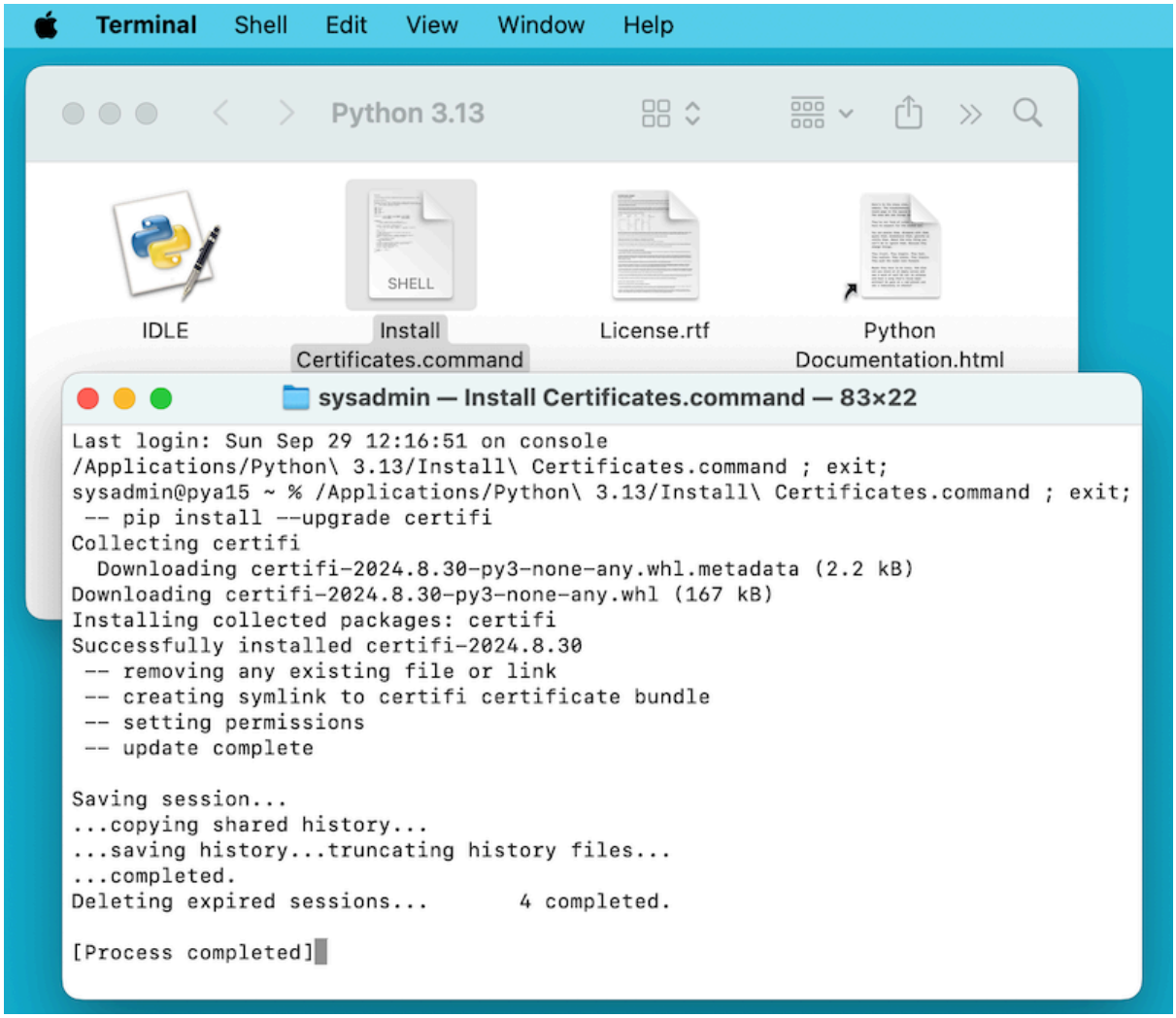
Quando a instalação for concluída, a janela **Resumo** será exibida.



Clique duas vezes no ícone ou arquivo **`Install Certificates.command`** na janela `/Applications/Python 3.14/` para concluir a instalação.



Isso abrirá uma janela temporária do console **`Terminal`** que usará o novo Python para baixar e instalar certificados raiz SSL para seu uso.



Se `Successfully installed certifi` e `update complete` aparecerem na janela do terminal, a instalação estará concluída. Feche essa janela do terminal e a janela do instalador.

Uma instalação do padrão incluirá:

- Uma pasta `Python 3.14` em sua pasta `Applications`. Aqui você encontra o **`IDLE`**, o ambiente de desenvolvimento que é parte das distribuições oficiais do Python; e o **`Python Launcher`**, que executa scripts Python quando são clicados duas vezes dentro do Finder .
- Uma framework `/Library/Frameworks/Python.framework`, que inclui o executável Python e suas bibliotecas. O instalador adiciona esse local ao seu *path* do console. Para desinstalar o Python, você pode remover esses três itens. É um link simbólico para o executável Python armazenado na pasta `/usr/local/bin/`.

Nota

As versões recentes do macOS incluem um comando **`python3`** em `/usr/bin/python3` que aponta para uma versão geralmente mais antiga e incompleta de Python fornecida por e para uso pelas ferramentas de desenvolvimento da Apple, **`Xcode`** ou o **`Command Line Tools for Xcode`**. Você nunca deve modificar ou tentar excluir essa instalação por ser controlada pela Apple e é usada por softwares da Apple ou de terceiros. Se você optar por instalar uma versão mais recente do Python a partir de `python.org`, você terá duas instalações diferentes, mas funcionais, de Python no computador que podem coexistir. As opções do instalador padrão devem garantir que o comando **`python3`** seja usado em vez do **`python3`** do sistema.

5.1.2 Como executar um script Python

Há duas maneiras de invocar o interpretador Python. Se você estiver familiarizado com o uso de um console Unix em terminal, você pode invocar `python3.14` ou `python3`, opcionalmente seguido por uma ou mais opções da linha de comando (descritas em *Linha de comando e ambiente*). O tutorial de Python também tem uma seção útil sobre o uso interativo do Python a partir de um console.

Você também pode invocar o interpretador por um ambiente de desenvolvimento integrado (IDE). O idle é um editor e ambiente de desenvolvimento básico incluído com a distribuição padrão de Python. **`IDLE`** inclui um menu Ajuda que te permite acessar a documentação de Python. Se você for completamente iniciante em Python, você pode ler o tutorial de introdução na documentação.

Há muitos outros editores e IDEs disponíveis. Consulte *Editores e IDEs* para mais informações.

Para executar um script Python na janela do terminal, você pode invocar o interpretador com o nome do arquivo de script:

```
python3.14 meuscript.py
```

Para executar seu script a partir do Finder, você pode:

- Arrastá-lo para o **`Python Launcher`**.
- Selecione **`Python Launcher`** como a aplicação padrão para abrir seu script (ou qualquer script `.py`) na janela de informações Finder Info, e clique duas vezes nele. O **`Python Launcher`** tem várias preferências para controlar como o script é iniciado. Você pode arrastar opções para alterá-las por uma invocação, ou usar o menu `Preferências` para alterar configurações globalmente.

Saiba que executar o script diretamente do Finder pode produzir resultados diferentes daqueles obtidos em uma janela de terminal, pois o script não será executado no ambiente normal do console, incluindo qualquer configuração de variáveis de ambiente nos perfis do console. E, como em qualquer outro script ou programa, certifique-se do que está prestes a executar.

5.2 Distribuições alternativas

Além do instalador padrão do `python.org` para macOS, existem distribuições de terceiros para macOS que podem incluir funcionalidades adicionais. Algumas distribuições populares e seus recursos-chave são:

ActivePython
: Instalador com compatibilidade multiplataforma, documentação

Anaconda
: Módulos científicos populares (como numpy, scipy e pandas) e o gerenciador de pacotes `conda`.

Homebrew
: Gerenciador de pacotes para macOS que inclui várias versões do Python e muitos pacotes Python de terceiros (incluindo numpy, scipy e pandas).

MacPorts
: Outro gerenciador de pacotes para macOS, incluindo várias versões de Python e muitos pacotes Python de terceiros. Pode incluir versões pré-construídas do Python e muitos pacotes para versões mais antigas do macOS.

Note que distribuições podem não incluir versões mais recentes de Python ou de outras bibliotecas, e não são mantidas ou providas pela equipe do Python.

5.3 Instalando pacotes adicionais ao python

Consulte o Guia de Usuário para Empacotamento de Python para mais informações.

5.4 Programação de GUI

Existem várias opções para criar aplicações GUI no Mac com Python.

O kit de ferramentas GUI padrão de Python é o módulo `tkinter`, baseado no kit de ferramentas multiplataforma Tk (https://www.tcl.tk). Uma versão nativa do Tk para macOS está incluída no instalador.

O PyObjC é uma ligação em Python para o Objective-C/Cocoa da Apple framework. Informações sobre o PyObjC estão disponíveis em pyobjc.

Há vários kits de ferramentas GUI alternativas disponíveis para o macOS, incluindo:

- PySide: Ligações oficiais em Python ao kit de ferramentas Qt GUI .
- PyQt: Ligações alternativas em Python ao Qt.
- Kivy: Um kit de ferramentas GUI multiplataforma que dá suporte a plataformas de desktop e dispositivos móveis.
- Toga: Parte do BeeWare Project; dá suporte a aplicativos para desktop, dispositivos móveis, Web e console.
- wxPython: Um kit de ferramentas multiplataforma que dá suporte a sistemas operacionais desktop.

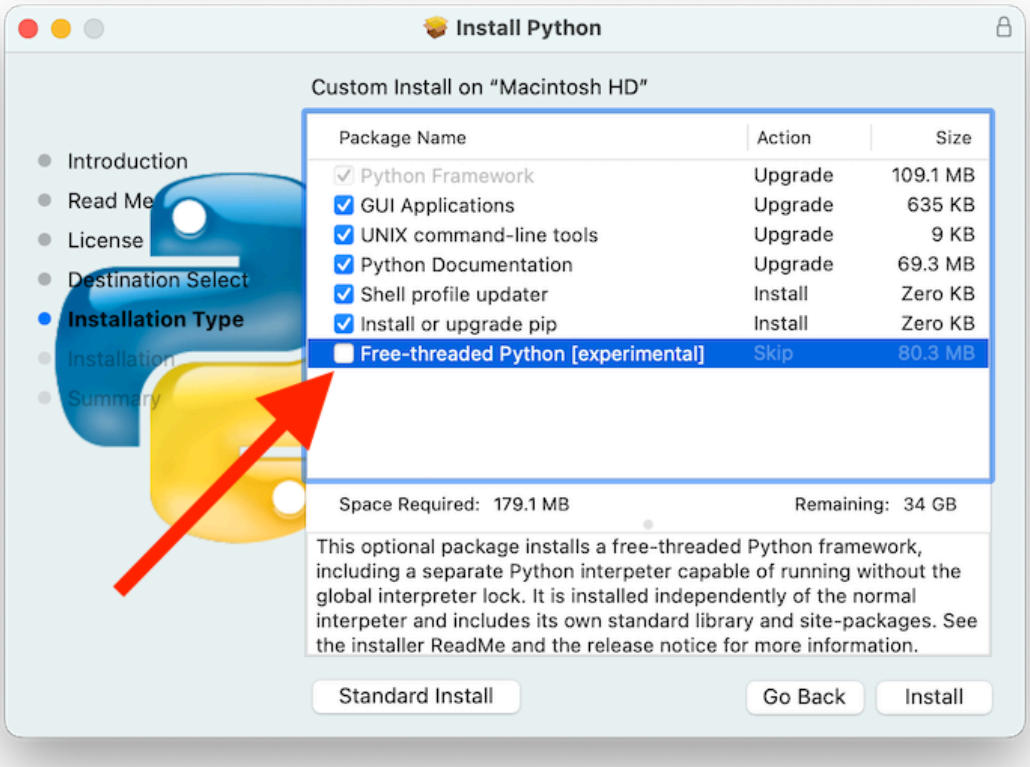
5.5 Tópicos Avançados

5.5.1 Instalando binários com threads livres

Adicionado na versão 3.13.

O pacote de instalação do *Python para macOS* em `python.org` pode, opcionalmente, instalar uma compilação adicional do Python 3.14 que dá suporte à **PEP 703**, o recurso de threads livres (executado com o *trava global do interpretador* desativado). Verifique a página de lançamento em `python.org` para obter informações atualizadas.

O modo com threads livres está funcionando e continua sendo aprimorado, mas há alguma sobrecarga adicional em cargas de trabalho de thread única em comparação com a construção regular. Além disso, pacotes de terceiros, em particular aqueles com um *módulo de extensão*, podem não estar prontos para uso em uma construção com threads livres e reativarão a *GIL*. Portanto, o suporte para threads livres não é instalado por padrão. Ele é fornecido como uma opção de instalação separada, disponível clicando no botão **Personalizar** na etapa **Tipo de Instalação** do instalador, conforme descrito acima.



Se a caixa ao lado do nome **Free-Threaded Python** for marcada, um arquivo `PythonT.framework` também será instalado junto ao arquivo `Python.framework` em `/Library/Frameworks`. Essa configuração permite que a compilação de Python 3.14 com threads livres coexista no seu sistema com a compilação de Python 3.14 tradicional (com GIL) com risco mínimo durante instalação ou testagem. Esse layout de instalação pode mudar em lançamentos futuros.

Precauções e limitações conhecidas:

- O pacote **Unix command-line tools**, que é selecionado por padrão, instalará links em `/usr/local/bin` para `python3.14t`, o interpretador com threads livres e `python3.14t-config`, um utilitário de configuração que pode ser útil para criadores de pacote. Como `/usr/local/bin` é normalmente incluído em seu `PATH` do console, geralmente não é necessário alterar as variáveis de ambiente do `PATH` para usar o `python3.14t`.
- Neste lançamento, o pacote **Shell profile updater** e o comando `Update Shell Profile.command` em `/Applications/Python 3.14/` não dão suporte ao pacote com threads livres.
- A construção com threads livres e a compilação tradicional têm caminhos de pesquisa separados e diretórios `site-packages` separados, portanto, por padrão, se você precisar de um pacote disponível em ambas construções, talvez seja necessário instalá-lo em ambos. O pacote com threads livres instalará uma instância separada do **`pip`** para uso com o `python3.14t`.
  - Para instalar um pacote usando **`pip`** sem um **`venv`**:

    ```
    python3.14t -m pip install <package_name>
    ```
- Ao trabalhar com vários ambientes Python, geralmente é mais seguro e mais fácil criar e usar ambientes virtuais. Isso pode tanto evitar possíveis conflitos de comandos, quanto evitar a confusão sobre qual Python está sendo usado:

  ```
  python3.14t -m venv <venv_name>
  ```

  e, em seguida, **`activate`**.
- Para executar uma versão do IDLE com threads livres:

```
python3.14t -m idlelib
```

- Os interpretadores em ambos as construções respondem às mesmas variáveis de ambiente *PYTHON*, o que pode gerar resultados inesperados, por exemplo, se você tiver `PYTHONPATH` definido em um perfil de console. Se necessário, há *opções de linha de comando* como `-E` para ignorar essas variáveis de ambiente.
- A construção com threads livres é ligada às bibliotecas compartilhadas de terceiros, como `OpenSSL` e `Tk`, instaladas no framework tradicional. Isso significa que ambas as compilações também compartilham um conjunto de certificados, como instalados pelo script **`Install Certificates.command`**, portanto, ele só precisa ser executado uma vez.
- Se você não puder depender do link em `/usr/local/bin` apontando para `python3.14t` com threads livres do `python.org` (por exemplo, se quiser instalar sua própria versão lá ou se alguma outra distribuição instalá-lo lá), você poderá definir explicitamente sua variável de ambiente de console `PATH` para incluir o diretório da framework `PythonT bin`:

  ```
  export PATH="/Library/Frameworks/PythonT.framework/Versions/3.14/bin":"$PATH"
  ```

  A instalação tradicional do framework por padrão faz algo semelhante, exceto pelo arquivo `Python.framework`. Observe que ter dois diretórios `bin` do framework em `PATH` pode gerar confusão se houver nomes duplicados, como `python3.14`, em ambos; qual deles é realmente usado depende da ordem em que aparecem em `PATH`. Os comandos `which python3.x` ou `which python3.xt` pode mostrar qual caminho está sendo usado. Usar ambientes virtuais pode ajudar a evitar essas ambiguidades. Outra opção pode ser criar um **`alias`** no console para o interpretador desejado, como:

  ```
  alias py3.14="/Library/Frameworks/Python.framework/Versions/3.14/bin/python3.
  ↪14"
  alias py3.14t="/Library/Frameworks/PythonT.framework/Versions/3.14/bin/python3.
  ↪14t"
  ```

5.5.2 Instalação usando a linha de comando

Se você quiser usar automação para instalar o pacote de instalação `python.org` (em vez de usar o aplicativo GUI **`Installer`** do macOS), o utilitário para linha de comando **`installer`** permite que você selecione opções que não sejam do padrão. Se você não estiver familiarizado com o **`installer`**, ele pode ser um tanto enigmático (consulte: command:*man installer* para obter mais informações). Como exemplo, o seguinte trecho de console mostra uma maneira de fazer isso, usando a versão `3.14.0b2` e selecionando a opção de interpretador com threads livres:

```
RELEASE="python-3.140b2-macos11.pkg"

# download installer pkg
curl -O https://www.python.org/ftp/python/3.14.0/${RELEASE}

# create installer choicechanges to customize the install:
#    enable the PythonTFramework-3.14 package
#    while accepting the other defaults (install all other packages)
cat > ./choicechanges.plist <<EOF
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/
↪PropertyList-1.0.dtd">
<plist version="1.0">
<array>
        <dict>
                <key>attributeSetting</key>
                <integer>1</integer>
                <key>choiceAttribute</key>
                <string>selected</string>
                <key>choiceIdentifier</key>
                <string>org.python.Python.PythonTFramework-3.14</string>
        </dict>
</array>
</plist>
```

```
EOF

sudo installer -pkg ./${RELEASE} -applyChoiceChangesXML ./choicechanges.plist␣
↪-target /
```

Você pode então testar que ambas as compilações de instaladores estão disponíveis com algo como:

```
$ # test that the free-threaded interpreter was installed if the Unix Command␣
↪Tools package was enabled
$ /usr/local/bin/python3.14t -VV
Python 3.14.0b2 free-threading build (v3.14.0b2:3a83b172af, Jun  5 2024, 12:57:31)␣
↪[Clang 15.0.0 (clang-1500.3.9.4)]
$ #    and the traditional interpreter
$ /usr/local/bin/python3.14 -VV
Python 3.14.0b2 (v3.14.0b2:3a83b172af, Jun  5 2024, 12:50:24) [Clang 15.0.0␣
↪(clang-1500.3.9.4)]
$ # test that they are also available without the prefix if /usr/local/bin is on
↪$PATH
$ python3.14t -VV
Python 3.14.0b2 free-threading build (v3.14.0b2:3a83b172af, Jun  5 2024, 12:57:31)␣
↪[Clang 15.0.0 (clang-1500.3.9.4)]
$ python3.14 -VV
Python 3.14.0b2 (v3.14.0b2:3a83b172af, Jun  5 2024, 12:50:24) [Clang 15.0.0␣
↪(clang-1500.3.9.4)]
```

Nota

Os instaladores atuais do `python.org` só instalam em locais fixos, como `/Library/Frameworks/`, `/Applications` e `/usr/local/bin`. Não é possível usar a opção `-domain` do **`installer`** para instalar em outros locais.

5.5.3 Distribuindo aplicações Python

Existe uma variedade de ferramentas para converter seu código Python em uma aplicação distribuível independente:

- py2app: Oferece suporte à criação de pacotes `.app` do macOS a partir de um projeto Python.
- Briefcase: Parte do BeeWare Project; uma ferramenta de empacotamento multiplataforma que permite criar *bundles* `.app` no macOS, além de gerenciar assinaturas digitais e a notarização.
- PyInstaller: Uma ferramenta de empacotamento multiplataforma que cria um único arquivo ou pasta como um artefato distribuível.

5.5.4 Conformidade com a App Store

As aplicações enviadas para distribuição pela macOS App Store devem passar pelo processo de revisão de aplicativos da Apple. Esse processo inclui um conjunto de regras de validação automatizadas que inspecionam o pacote de aplicativos enviado em busca de código problemático.

A biblioteca padrão do Python contém alguns códigos que violam essas regras automatizadas. Embora essas violações pareçam ser falsos positivos, as regras de revisão da Apple não podem ser contestadas. Portanto, é necessário modificar a biblioteca padrão do Python para que uma aplicação passe na revisão da App Store.

A árvore de fontes do Python contém um arquivo de patch que removerá todo o código que é conhecido por causar problemas no processo de revisão da App Store. Este patch é aplicado automaticamente quando o CPython é configurado com a opção `--with-app-store-compliance`.

Este patch normalmente não é necessário para usar o CPython em um Mac; nem é necessário se você estiver distribuindo uma aplicação *fora* da App Store do macOS. É necessário *apenas* se você estiver usando a App Store do macOS como canal de distribuição.

5.6 Outros recursos

A página de ajuda do python.org contém links para vários recursos úteis. A lista de discussão Pythonmac-SIG é outro recurso específico de suporte para usuários e desenvolvedores de Python no Mac.

CAPÍTULO 6

Utilizando Python no Android

Python no Android é diferente do Python em plataformas de desktop. Em uma plataforma desktop, o Python geralmente é instalado como um recurso do sistema que pode ser usado por qualquer usuário daquele computador. Os usuários então interagem com o Python executando um executável **`python`** e inserindo comandos em um prompt interativo ou executando um script Python.

No Android, não existe o conceito de instalação como recurso do sistema. A única unidade de distribuição de software é uma aplicação, ou "app". Também não há console onde você possa executar um executável **`python`** ou interagir com um REPL do Python.

Como resultado, a única maneira de usar Python no Andoid é no modo incorporado – ou seja, escrevendo uma aplicação Android nativo, incorporando um interpretador Python usando `libpython` e invocando o código Python usando API de incorporação do Python. O interpretador Python completo, a biblioteca padrão e todo o seu código Python são então empacotados para sua aplicação para seu próprio uso privado.

A biblioteca padrão do Python tem algumas omissões e restrições notáveis no Andoid. Consulte o guia de disponibilidade de API para Android para obter detalhes.

6.1 Adicionando Python a uma aplicação Android

A maioria dos desenvolvedores de aplicações deve usar uma das seguintes ferramentas, o que proporcionará uma experiência muito mais fácil:

- Briefcase, do projeto BeeWare
- Buildozer, do projeto Kivy
- Chaquopy
- pyqtdeploy
- Termux

Se você tem certeza de que quer fazer tudo isso manualmente, continue lendo. Você pode usar a aplicação testbed como um guia; cada passo abaixo contém um link para o arquivo relevante.

- Primeiro, obtenha uma construção do Python para Android:
  - A maneira mais fácil é baixar uma versão do Android em python.org. O diretório `prefix` mencionado abaixo está no nível raiz do pacote.
  - Ou, se quiser construí-la você mesmo, siga as instruções em Android/README.md. O diretório `prefix` será criado em `cross-build/`*`HOST`*.

- Adicione código ao seu arquivo build.gradle para copiar os seguintes itens para o seu projeto. Todos, exceto seu próprio código Python, podem ser copiados de `prefix/lib`:
  - Em suas bibliotecas JNI:
    * `libpython*.*.so`
    * `lib*_python.so` (bibliotecas externas, tal como OpenSSL)
  - Em seus ativos (*assets*):
    * `python*.*` (a biblioteca padrão do Python)
    * `python*.*/site-packages` (seu próprio código Python)
- Adicione código ao seu aplicativo para extrair os ativos para o sistema de arquivos.
- Adicione código a sua aplicação para iniciar o Python em modo incorporado. Isso precisará ser código C chamado via JNI.

6.2 Construindo um pacote Python para Android

Pacotes Python podem ser criados para Android como wheels e lançados no PyPI. A ferramenta recomendada para isso é cibuildwheel, que automatiza todos os detalhes da configuração de um ambiente de compilação cruzada, da construção da wheel e dos testes em um emulador.

CAPÍTULO 7

Usando Python no iOS

Autores
Russell Keith-Magee (2024-03)

Python no iOS é diferente do Python em plataformas de desktop. Em uma plataforma desktop, o Python geralmente é instalado como um recurso do sistema que pode ser usado por qualquer usuário daquele computador. Os usuários então interagem com o Python executando um executável **`python`** e inserindo comandos em um prompt interativo ou executando um script Python.

No iOS, não existe o conceito de instalação como recurso do sistema. A única unidade de distribuição de software é uma aplicação, ou "app". Também não há console onde você possa executar um executável **`python`** ou interagir com um REPL do Python.

Como resultado, a única maneira de usar Python no iOS é no modo incorporado – ou seja, escrevendo uma aplicação iOS nativo e incorporando um interpretador Python usando `libPython` e invocando o código Python usando API de incorporação do Python. O interpretador Python completo, a biblioteca padrão e todo o seu código Python são então empacotados como um pacote independente que pode ser distribuído pela iOS App Store.

Se você deseja experimentar pela primeira vez escrever uma aplicação iOS em Python, projetos como BeeWare e Kivy irão fornecer uma experiência de usuário muito mais acessível. Esses projetos gerenciam as complexidades associadas à execução de um projeto iOS, portanto, você só precisa lidar com o próprio código Python.

7.1 Python em tempo de execução no iOS

7.1.1 Compatibilidade com a versão do iOS

A versão mínima suportada do iOS é especificada em tempo de compilação, usando a opção `--host` do `configure`. Por padrão, quando compilado para iOS, Python será compilado com uma versão iOS mínima suportada de 13.0. Para usar uma versão mínima diferente do iOS, forneça o número da versão como parte do argumento `--host` - por exemplo, `--host=arm64-apple-ios15.4-simulator` compilaria um simulador ARM64 construído com uma meta de implantação de 15.4.

7.1.2 Identificação da plataforma

Ao executar no iOS, `sys.platform` reportará como `ios`. Este valor será retornado em um iPhone ou iPad, independentemente da aplicação estar executando no simulador ou em um dispositivo físico.

Informações sobre o ambiente de execução específico, incluindo a versão do iOS, modelo do dispositivo e se o dispositivo é um simulador, podem ser obtidas usando `platform.ios_ver()`. `platform.system()` reportará

`iOS` ou `iPadOS`, dependendo do dispositivo.

`os.uname()` reporta detalhes em nível de kernel; ele reportará o nome `Darwin`.

7.1.3 Disponibilidade da biblioteca padrão

A biblioteca padrão do Python tem algumas omissões e restrições notáveis no iOS. Consulte o guia de disponibilidade de API para iOS para obter detalhes.

7.1.4 Módulos de extensão binária

Uma diferença notável sobre o iOS como plataforma é que a distribuição da App Store impõe requisitos rígidos ao empacotamento de uma aplicação. Um desses requisitos rege como os módulos de extensão binária são distribuídos.

A iOS App Store exige que *todos* os módulos binários em uma aplicação iOS sejam bibliotecas dinâmicas, contidas em um framework com metadados apropriados, armazenados na pasta `Frameworks` da aplicação empacotada. Pode haver apenas um único binário por framework, e não pode haver nenhum material binário executável fora da pasta `Frameworks`.

Isto entra em conflito com a abordagem usual do Python para distribuição de binários, que permite que um módulo de extensão binária seja carregado de qualquer local em `sys.path`. Para garantir a conformidade com as políticas da App Store, um projeto iOS deve pós-processar quaisquer pacotes Python, convertendo módulos binários `.so` em estruturas independentes individuais com metadados e assinatura apropriados. Para obter detalhes sobre como realizar esse pós-processamento, consulte o guia para *adicionar Python ao seu projeto*.

Para ajudar o Python a descobrir binários em seu novo local, o arquivo `.so` original em `sys.path` é substituído por um arquivo `.fwork`. Este arquivo é um arquivo texto que contém a localização do binário do framework, relativo ao pacote de aplicações. Para permitir que o framework retorne ao local original, o framework deve conter um arquivo `.origin` que contém a localização do arquivo `.fwork`, relativo ao pacote da aplicação.

Por exemplo, considere o caso de uma importação `from foo.bar import _whiz`, onde `_whiz` é implementado com o módulo binário `sources/foo/bar/_whiz.abi3.so`, com `sources` sendo o local registrado em `sys.path`, relativo ao pacote da aplicação. Este módulo *deve* ser distribuído como `Frameworks/foo.bar._whiz.framework/foo.bar._whiz` (criando o nome do framework a partir do caminho de importação completo do módulo), com um arquivo `Info.plist` no diretório `.framework` identificando o binário como um framework. O módulo `foo.bar._whiz` seria representado no local original com um arquivo marcador `sources/foo/bar/_whiz.abi3.fwork`, contendo o caminho `Frameworks/foo.bar._whiz/foo.bar._whiz`. O framework também conteria `Frameworks/foo.bar._whiz.framework/foo.bar._whiz.origin`, contendo o caminho para o arquivo `.fwork`.

Ao executar no iOS, o interpretador Python instalará um `AppleFrameworkLoader` que é capaz de ler e importar arquivos `.fwork`. Uma vez importado, o atributo `__file__` do módulo binário reportará como a localização do arquivo `.fwork`. Entretanto, o `ModuleSpec` para o módulo carregado reportará a `origin` como a localização do binário na pasta do framework.

7.1.5 Binários stub do compilador

O Xcode não expõe compiladores explícitos para iOS; em vez disso, ele usa um script `xcrun` que resolve um caminho do compilador (por exemplo, `xcrun --sdk iphoneos clang` para obter o `clang` para um dispositivo iPhone). No entanto, usar este script apresenta dois problemas:

- A saída de `xcrun` inclui caminhos que são específicos da máquina, resultando em um módulo sysconfig que não pode ser compartilhado entre usuários; e
- Isso resulta em definições `CC`/`CPP`/`LD`/`AR` que incluem espaços. Existem muitas ferramentas do ecossistema C que pressupõem que você pode dividir uma linha de comando no primeiro espaço para obter o caminho para o executável do compilador; este não é o caso ao usar `xcrun`.

Para evitar esses problemas, o Python forneceu stubs para essas ferramentas. Esses stubs são wrappers de script de shell em torno das ferramentas subjacentes `xcrun`, distribuídos em uma pasta `bin` distribuída junto com a estrutura iOS compilada. Esses scripts são relocáveis e sempre serão resolvidos para os caminhos apropriados do sistema local. Ao incluir esses scripts na pasta bin que acompanha um framework, o conteúdo do módulo `sysconfig` se torna útil

para usuários finais compilarem seus próprios módulos. Ao compilar módulos Python de terceiros para iOS, você deve garantir que esses binários stub estejam no seu caminho.

7.2 Instalando Python no iOS

7.2.1 Ferramentas para construir aplicações de iOS

A construção para iOS requer o uso das ferramentas Xcode da Apple. É altamente recomendável que você use a versão estável mais recente do Xcode. Isso exigirá o uso da versão mais recente (ou segunda) do macOS lançada recentemente, já que a Apple não mantém o Xcode para versões mais antigas do macOS. As ferramentas de linha de comando do Xcode não são suficientes para o desenvolvimento iOS; você precisa de uma instalação *completa* do Xcode.

Se quiser executar seu código no simulador iOS, você também precisará instalar um iOS Simulator Platform. Você deverá ser solicitado a selecionar um iOS Simulator Platform ao executar o Xcode pela primeira vez. Alternativamente, você pode adicionar um iOS Simulator Platform selecionando na guia Platforms do painel Settings do Xcode.

7.2.2 Adicionando Python a um projeto iOS

Python pode ser adicionado a qualquer projeto iOS, usando Swift ou Objective C. Os exemplos a seguir usarão Objective C; se você estiver usando Swift, poderá achar uma biblioteca como PythonKit útil.

Para adicionar Python a um projeto Xcode de iOS:

1. Construa ou obtenha um `XCFramework` em Python. Veja as instruções em Apple/iOS/README.md (na distribuição fonte do CPython) para detalhes sobre como construir um `XCFramework` em Python. No mínimo, você precisará de uma construção que tenha suporte `arm64-apple-ios`, além de `arm64-apple-ios-simulator` ou `x86_64-apple-ios-simulator`.
2. Arraste o `XCframework` para o seu projeto iOS. Nas instruções a seguir, presumiremos que você colocou o `XCframework` na raiz do seu projeto; no entanto, você pode usar qualquer outro local desejado ajustando os caminhos conforme necessário.
3. Adicione o código da sua aplicação como uma pasta no seu projeto Xcode. Nas instruções a seguir, presumiremos que seu código de usuário está em uma pasta chamada `app` na raiz do seu projeto; você pode usar qualquer outro local ajustando os caminhos conforme necessário. Certifique-se de que esta pasta esteja associada ao destino da sua aplicação.
4. Selecione o destino da aplicação selecionando o nó raiz do seu projeto Xcode e, em seguida, o nome do destino na barra lateral que aparece.
5. Nas configurações de "General", em "Frameworks, Libraries and Embedded Content", adicione `Python.xcframework`, com "Embed & Sign" selecionado.
6. Na guia "Build Settings", modifique o seguinte:
   - Build Options
     - User Script Sandboxing: No
     - Enable Testability: Yes
   - Search Paths
     - Framework Search Paths: `$(PROJECT_DIR)`
     - Header Search Paths: `"$(BUILT_PRODUCTS_DIR)/Python.framework/Headers"`
   - Apple Clang - Warnings - All languages
     - Quoted Include In Framework Header: No
7. Adicione uma etapa de construção que processa a biblioteca padrão do Python e suas dependências de binários Python. Na aba "Build Phases", adicione uma nova etapa de construção "Run Script" *antes* da etapa "Embed

Frameworks", mas *depois* da etapa "Copy Bundle Resources". Nomeie a etapa como "Process Python libraries", desative a caixa de seleção "Based on dependency analysis" e defina o conteúdo do script como:

```
set -e
source $PROJECT_DIR/Python.xcframework/build/build_utils.sh
install_python Python.xcframework app
```

Se você colocou seu XCframework em algum lugar diferente da raiz do seu projeto, modifique o caminho para o primeiro argumento.

8. Adicione código Objective C para inicializar e usar um interpretador Python em modo embarcado. Você deve garantir que:
   - Modo UTF-8 (`PyPreConfig.utf8_mode`) esteja habilitado;
   - Stdio buffered (`PyConfig.buffered_stdio`) esteja desabilitado;
   - Escrita de bytecode (`PyConfig.write_bytecode`) esteja desabilitada;
   - Manipuladores de sinais (`PyConfig.install_signal_handlers`) estejam habilitados;
   - Registro de logs do sistema (`PyConfig.use_system_logger`) esteja desabilitado (opcional, mas fortemente recomendado; isso está habilitado por padrão);
   - *`PYTHONHOME`* para o interpretador esteja configurado para apontar para a subpasta `python` do pacote da sua aplicação; e
   - O *`PYTHONPATH`* para o interpretador inclui:
     - a subpasta `python/lib/python3.X` do pacote da sua aplicação,
     - a subpasta `python/lib/python3.X/lib-dynload` do pacote da sua aplicação, e
     - a subpasta `app` do pacote da sua aplicação

   O local do pacote da sua aplicação pode ser determinada usando `[[NSBundle mainBundle] resourcePath]`.

Os passos 7 e 8 destas instruções presumem que você tem uma única pasta de código de aplicação Python puro, chamada `app`. Se você tiver módulos binários de terceiros em sua aplicação, serão necessárias algumas etapas adicionais:

- Você precisa garantir que todas as pastas que contenham binários de terceiros sejam associadas ao destino da aplicação ou sejam explicitamente copiadas como parte da etapa 7. A etapa 7 também deve limpar quaisquer binários que não sejam apropriados para a plataforma que uma construção específica está direcionando (ou seja, exclua todos os binários do dispositivo se estiver criando uma aplicação direcionado ao simulador).
- If you're using a separate folder for third-party packages, ensure that folder is added to the end of the call to `install_python` in step 7, and as part of the *`PYTHONPATH`* configuration in step 8.
- Se alguma das pastas que contêm pacotes de terceiros contiver arquivos `.pth`, você deverá adicionar essa pasta como um *diretório de site* (usando `site.addsitedir()`), em vez de adicioná-la diretamente a *`PYTHONPATH`* ou `sys.path`.

7.2.3 Testando um pacote Python

The CPython source tree contains a testbed project that is used to run the CPython test suite on the iOS simulator. This testbed can also be used as a testbed project for running your Python library's test suite on iOS.

After building or obtaining an iOS XCFramework (see Apple/iOS/README.md for details), create a clone of the Python iOS testbed project. If you used the `Apple` build script to build the XCframework, you can run:

```
$ python cross-build/iOS/testbed clone --app <path/to/module1> --app <path/to/
↪module2> app-testbed
```

Or, if you've sourced your own XCframework, by running:

```
$ python Apple/testbed clone --platform iOS --framework <path/to/Python.
↪xcframework> --app <path/to/module1> --app <path/to/module2> app-testbed
```

Any folders specified with the `--app` flag will be copied into the cloned testbed project. The resulting testbed will be created in the `app-testbed` folder. In this example, the `module1` and `module2` would be importable modules at runtime. If your project has additional dependencies, they can be installed into the `app-testbed/Testbed/app_packages` folder (using `pip install --target app-testbed/Testbed/app_packages` or similar).

Você pode então usar a pasta `app-testbed` para executar o conjunto de testes para seu aplicativo. Por exemplo, se `módulo1.tests` fosse o ponto de entrada para seu conjunto de testes, você poderia executar:

```
$ python app-testbed run -- módulo1.tests
```

Isso é o equivalente a executar `python -m módulo1.tests` em uma compilação Python de desktop. Quaisquer argumentos após `--` serão passados para o banco de testes como se fossem argumentos para `python -m` em uma máquina de desktop.

Você também pode abrir o projeto de teste no Xcode executando:

```
$ open app-testbed/iOSTestbed.xcodeproj
```

Isso permitirá que você use o conjunto completo de ferramentas do Xcode para depuração.

Os argumentos usados para executar o conjunto de testes são definidos como parte do plano de teste. Para modificar o plano de teste, selecione o nó do plano de teste na árvore do projeto (deve ser o primeiro filho do nó raiz) e selecione a aba "Configurations". Modifique o valor "Arguments Passed On Launch" para alterar os argumentos de teste.

The test plan also disables parallel testing, and specifies the use of the `Testbed.lldbinit` file for providing configuration of the debugger. The default debugger configuration disables automatic breakpoints on the `SIGINT`, `SIGUSR1`, `SIGUSR2`, and `SIGXFSZ` signals.

7.3 Conformidade com a App Store

The only mechanism for distributing apps to third-party iOS devices is to submit the app to the iOS App Store; apps submitted for distribution must pass Apple's app review process. This process includes a set of automated validation rules that inspect the submitted application bundle for problematic code. There are some steps that must be taken to ensure that your app will be able to pass these validation steps.

7.3.1 Incompatible code in the standard library

A biblioteca padrão do Python contém alguns códigos que violam essas regras automatizadas. Embora essas violações pareçam ser falsos positivos, as regras de revisão da Apple não podem ser contestadas; portanto, é necessário modificar a biblioteca padrão do Python para que uma aplicação passe na revisão da App Store.

A árvore de fontes do Python contém um arquivo de patch que removerá todo o código que é conhecido por causar problemas no processo de revisão da App Store. Este patch é aplicado automaticamente quando o ao construir para o iOS.

7.3.2 Privacy manifests

In April 2025, Apple introduced a requirement for certain third-party libraries to provide a Privacy Manifest. As a result, if you have a binary module that uses one of the affected libraries, you must provide an `.xcprivacy` file for that library. OpenSSL is one library affected by this requirement, but there are others.

If you produce a binary module named `mymodule.so`, and use you the Xcode build script described in step 7 above, you can place a `mymodule.xcprivacy` file next to `mymodule.so`, and the privacy manifest will be installed into the required location when the binary module is converted into a framework.

CAPÍTULO 8

Editores e IDEs

Há um grande número de IDEs com suporte à linguagem de programação Python. Vários editores e IDEs provêm destaques coloridos de sintaxe, ferramentas de depuração, e checagem do código conforme a **PEP 8**.

8.1 IDLE — editor e console Python

IDLE é o Ambiente Integrado de Desenvolvimento e Aprendizagem do Python (em inglês Integrated Development and Learning Environment) e geralmente vem junto com as instalações do Python. Se você estiver no Linux e não tiver o IDLE instalado consulte *Instalando o IDLE no Linux*. Para obter mais informações consulte, consulte os documentos do IDLE.

8.2 Outros editores e IDEs

As wikis da comunidades Python têm informações enviadas pela comunidade sobre Editores e IDEs. Acesse Ferramentas de Desenvolvimento para obter uma lista abrangente enviada pela comunidade brasileira ou Python Editors e Integrated Development Environments para listas enviadas pela comunidade internacional (em inglês).

APÊNDICE A

Glossário

>>>
: O prompt padrão do console *interativo* do Python. Normalmente visto em exemplos de código que podem ser executados interativamente no interpretador.

...
: Pode se referir a:

  - O prompt padrão do console *interativo* do Python ao inserir o código para um bloco de código recuado, quando dentro de um par de delimitadores correspondentes esquerdo e direito (parênteses, colchetes, chaves ou aspas triplas) ou após especificar um decorador.
  - Os três pontos formam o objeto Ellipsis (reticências).

classe base abstrata
: Classes bases abstratas (ABCs, do inglês *abstract base class*) complementam a *tipagem pato*, fornecendo uma maneira de definir interfaces quando outras técnicas, como `hasattr()`, seriam desajeitadas ou sutilmente erradas (por exemplo, com métodos mágicos). ABCs introduzem subclasses virtuais, classes que não herdam de uma classe mas ainda são reconhecidas por `isinstance()` e `issubclass()`; veja a documentação do módulo `abc`. Python vem com muitas ABCs embutidas para estruturas de dados (no módulo `collections.abc`), números (no módulo `numbers`), fluxos (no módulo `io`), localizadores e carregadores de importação (no módulo `importlib.abc`). Você pode criar suas próprias ABCs com o módulo `abc`.

função de anotação
: Uma função que pode ser chamada para recuperar as *anotações* de um objeto. Esta função é acessível como o atributo `__annotate__` de funções, classes e módulos. Funções de anotação são um subconjunto de *funções de avaliação*.

anotação
: Um rótulo associado a uma variável, um atributo de classe ou um parâmetro de função ou valor de retorno, usado por convenção como *dica de tipo*.

  Anotações de variáveis locais não podem ser acessadas em tempo de execução, mas anotações de variáveis globais, atributos de classe e funções podem ser recuperadas chamando `annotationlib.get_annotations()` em módulos, classes e funções, respectivamente.

  Consulte *anotação de variável*, *anotação de função*, **PEP 484**, **PEP 526** e **PEP 649**, que descrevem essa funcionalidade. Consulte também annotations-howto para obter as melhores práticas sobre como trabalhar com anotações.

argumento
: Um valor passado para uma *função* (ou *método*) ao chamar a função. Existem dois tipos de argumento:

- *argumento nomeado*: um argumento precedido por um identificador (por exemplo, `name=`) na chamada de uma função ou passado como um valor em um dicionário precedido por `**`. Por exemplo, `3` e `5` são ambos argumentos nomeados nas chamadas da função `complex()` a seguir:

```
complex(real=3, imag=5)
complex(**{'real': 3, 'imag': 5})
```

- *argumento posicional*: um argumento que não é um argumento nomeado. Argumentos posicionais podem aparecer no início da lista de argumentos e/ou podem ser passados como elementos de um *iterável* precedidos por `*`. Por exemplo, `3` e `5` são ambos argumentos posicionais nas chamadas a seguir:

```
complex(3, 5)
complex(*(3, 5))
```

Argumentos são atribuídos às variáveis locais nomeadas no corpo da função. Veja a seção calls para as regras de atribuição. Sintaticamente, qualquer expressão pode ser usada para representar um argumento; após a expressão ser avaliada, o valor resultante é atribuído à variável local.

Veja também o termo *parâmetro* no glossário, a pergunta no FAQ sobre a diferença entre argumentos e parâmetros e **PEP 362**.

gerenciador de contexto assíncrono
Um objeto que controla o ambiente envolto numa instrução `async with` por meio da definição dos métodos `__aenter__()` e `__aexit__()`. Introduzido pela **PEP 492**.

gerador assíncrono
Uma função que retorna um *iterador gerador assíncrono*. É parecida com uma função de corrotina definida com `async def` exceto pelo fato de conter instruções `yield` para produzir uma série de valores que podem ser usados em um laço `async for`.

Normalmente se refere a uma função geradora assíncrona, mas pode se referir a um *iterador gerador assíncrono* em alguns contextos. Em casos em que o significado não esteja claro, usar o termo completo evita a ambiguidade.

Uma função geradora assíncrona pode conter expressões `await` e também as instruções `async for` e `async with`.

iterador gerador assíncrono
Um objeto criado por uma função *geradora assíncrona*.

Este é um *iterador assíncrono* que, quando chamado usando o método `__anext__()`, retorna um objeto aguardável que executará o corpo da função geradora assíncrona até a próxima expressão `yield`.

Cada `yield` suspende temporariamente o processamento, lembrando o estado de execução (incluindo variáveis locais e instruções `try` pendentes). Quando o *iterador gerador assíncrono* é efetivamente retomado com outro aguardável retornado por `__anext__()`, ele inicia de onde parou. Veja **PEP 492** e **PEP 525**.

iterável assíncrono
Um objeto que pode ser usado em uma instrução `async for`. Deve retornar um *iterador assíncrono* do seu método `__aiter__()`. Introduzido por **PEP 492**.

iterador assíncrono
Um objeto que implementa os métodos `__aiter__()` e `__anext__()`. `__anext__()` deve retornar um objeto *aguardável*. `async for` resolve os aguardáveis retornados por um método `__anext__()` do iterador assíncrono até que ele levante uma exceção `StopAsyncIteration`. Introduzido pela **PEP 492**.

estado de thread anexado
Um *estado de thread* que está ativo para a thread atual do sistema operacional.

Quando um *estado de thread* é anexado, a thread do sistema operacional tem acesso à API C completa do Python e pode invocar com segurança o interpretador de bytecode.

A menos que uma função indique explicitamente o contrário, tentar chamar a API C sem um estado de thread anexado resultará em um erro fatal ou comportamento indefinido. Um estado de thread pode ser anexado e

desanexado explicitamente pelo usuário por meio da API C, ou implicitamente pelo ambiente de execução, inclusive durante o bloqueio de chamadas C e pelo interpretador de bytecode entre as chamadas.

Na maioria das construções do Python, ter um estado de thread anexado implica que o chamador mantém a *GIL* para o interpretador atual, portanto, apenas uma thread do SO pode ter um estado de thread anexado em um determinado momento. Em construções do Python com *threads livres*, as threads podem manter simultaneamente um estado de thread anexado, permitindo um verdadeiro paralelismo do interpretador de bytecode.

atributo
Um valor associado a um objeto que é geralmente referenciado pelo nome separado por um ponto. Por exemplo, se um objeto *o* tem um atributo *a* esse seria referenciado como *o.a*.

É possível dar a um objeto um atributo cujo nome não seja um identificador conforme definido por identifiers, por exemplo usando `setattr()`, se o objeto permitir. Tal atributo não será acessível usando uma expressão pontilhada e, em vez disso, precisaria ser recuperado com `getattr()`.

aguardável
Um objeto que pode ser usado em uma expressão `await`. Pode ser uma *corrotina* ou um objeto com um método `__await__()`. Veja também a **PEP 492**.

BDFL
Abreviação da expressão da língua inglesa "Benevolent Dictator for Life" (em português, "Ditador Benevolente Vitalício"), referindo-se a Guido van Rossum, criador do Python.

arquivo binário
Um *objeto arquivo* capaz de ler e gravar em *objetos bytes ou similares*. Exemplos de arquivos binários são arquivos abertos no modo binário (`'rb'`, `'wb'` ou `'rb+'`), `sys.stdin.buffer`, `sys.stdout.buffer`, e instâncias de `io.BytesIO` e `gzip.GzipFile`.

Veja também *arquivo texto* para um objeto arquivo capaz de ler e gravar em objetos `str`.

referência emprestada
Na API C do Python, uma referência emprestada é uma referência a um objeto que não é dona da referência. Ela se torna um ponteiro solto se o objeto for destruído. Por exemplo, uma coleta de lixo pode remover a última *referência forte* para o objeto e assim destruí-lo.

Chamar `Py_INCREF()` na *referência emprestada* é recomendado para convertê-lo, internamente, em uma *referência forte*, exceto quando o objeto não pode ser destruído antes do último uso da referência emprestada. A função `Py_NewRef()` pode ser usada para criar uma nova *referência forte*.

objeto bytes ou similar
Um objeto com suporte ao bufferobjects e que pode exportar um buffer C *contíguo*. Isso inclui todos os objetos `bytes`, `bytearray` e `array.array`, além de muitos objetos `memoryview` comuns. Objetos bytes ou similares podem ser usados para várias operações que funcionam com dados binários; isso inclui compactação, salvamento em um arquivo binário e envio por um soquete.

Algumas operações precisam que os dados binários sejam mutáveis. A documentação geralmente se refere a eles como "objetos bytes ou similares para leitura e escrita". Exemplos de objetos de buffer mutável incluem `bytearray` e um `memoryview` de um `bytearray`. Outras operações exigem que os dados binários sejam armazenados em objetos imutáveis ("objetos bytes ou similares para somente leitura"); exemplos disso incluem `bytes` e a `memoryview` de um objeto `bytes`.

bytecode
O código-fonte Python é compilado para bytecode, a representação interna de um programa em Python no interpretador CPython. O bytecode também é mantido em cache em arquivos `.pyc` e `.pyo`, de forma que executar um mesmo arquivo é mais rápido na segunda vez (a recompilação dos fontes para bytecode não é necessária). Esta "linguagem intermediária" é adequada para execução em uma *máquina virtual*, que executa o código de máquina correspondente para cada bytecode. Tenha em mente que não se espera que bytecodes sejam executados entre máquinas virtuais Python diferentes, nem que se mantenham estáveis entre versões de Python.

Uma lista de instruções bytecode pode ser encontrada na documentação para o módulo dis.

chamável

Um chamável é um objeto que pode ser chamado, possivelmente com um conjunto de argumentos (veja *argumento*), com a seguinte sintaxe:

```
chamavel(argumento1, argumento2, argumentoN)
```

Uma *função*, e por extensão um *método*, é um chamável. Uma instância de uma classe que implementa o método `__call__()` também é um chamável.

função de retorno
Também conhecida como callback, é uma função sub-rotina que é passada como um argumento a ser executado em algum ponto no futuro.

classe
Um modelo para criação de objetos definidos pelo usuário. Definições de classe normalmente contém definições de métodos que operam sobre instâncias da classe.

variável de classe
Uma variável definida em uma classe e destinada a ser modificada apenas no nível da classe (ou seja, não em uma instância da classe).

variável de clausura
Uma *variável livre* referenciada de um *escopo aninhado* que é definida em um escopo externo em vez de ser resolvida em tempo de execução a partir dos espaços de nomes embutido ou globais. Pode ser explicitamente definida com a palavra reservada `nonlocal` para permitir acesso de gravação, ou implicitamente definida se a variável estiver sendo somente lida.

Por exemplo, na função `interna` no código a seguir, tanto `x` quanto `print` são *variáveis livres*, mas somente `x` é uma *variável de clausura*:

```
def externa():
    x = 0
    def interna():
        nonlocal x
        x += 1
        print(x)
    return interna
```

Devido ao atributo `codeobject.co_freevars` (que, apesar do nome, inclui apenas os nomes das variáveis de clausura em vez de listar todas as variáveis livres referenciadas), o termo mais geral *variável livre* às vezes é usado mesmo quando o significado pretendido é se referir especificamente às variáveis de clausura.

número complexo
Uma extensão ao familiar sistema de números reais em que todos os números são expressos como uma soma de uma parte real e uma parte imaginária. Números imaginários são múltiplos reais da unidade imaginária (a raiz quadrada de `-1`), normalmente escrita como `i` em matemática ou `j` em engenharia. O Python tem suporte nativo para números complexos, que são escritos com esta última notação; a parte imaginária escrita com um sufixo `j`, p.ex., `3+1j`. Para ter acesso aos equivalentes para números complexos do módulo `math`, utilize `cmath`. O uso de números complexos é uma funcionalidade matemática bastante avançada. Se você não sabe se irá precisar deles, é quase certo que você pode ignorá-los sem problemas.

contexto
Este termo tem diferentes significados dependendo de onde e como ele é usado. Alguns significados comuns:

- O estado ou ambiente temporário estabelecido por um *gerenciador de contexto* por meio de uma instrução `with`.
- A coleção de ligações de chave-valor associadas a um objeto `contextvars.Context` específico e acessadas por meio de objetos `ContextVar`. Veja também *variável de contexto*.
- Um objeto `contextvars.Context`. Veja também *contexto atual*.

protocolo de gerenciamento de contexto
Os métodos `__enter__()` e `__exit__()` chamados pela instrução `with`. Veja **PEP 343**.

gerenciador de contexto
Um objeto que implementa o *protocolo de gerenciamento de contexto* e controla o ambiente envolto em uma instrução `with`. Veja **PEP 343**.

variável de contexto
Uma variável cujo valor depende de qual contexto é o *contexto atual*. Os valores são acessados por meio de objetos `contextvars.ContextVar`. Variáveis de contexto são usadas principalmente para isolar o estado entre tarefas assíncronas simultâneas.

contíguo
Um buffer é considerado contíguo exatamente se for *contíguo C* ou *contíguo Fortran*. Os buffers de dimensão zero são contíguos C e Fortran. Em vetores unidimensionais, os itens devem ser dispostos na memória próximos um do outro, em ordem crescente de índices, começando do zero. Em vetores multidimensionais contíguos C, o último índice varia mais rapidamente ao visitar itens em ordem de endereço de memória. No entanto, nos vetores contíguos do Fortran, o primeiro índice varia mais rapidamente.

corrotina
Corrotinas são uma forma mais generalizada de sub-rotinas. Sub-rotinas tem a entrada iniciada em um ponto, e a saída em outro ponto. Corrotinas podem entrar, sair, e continuar em muitos pontos diferentes. Elas podem ser implementadas com a instrução `async def`. Veja também **PEP 492**.

função de corrotina
Uma função que retorna um objeto do tipo *corrotina*. Uma função de corrotina pode ser definida com a instrução `async def`, e pode conter as palavras chaves `await`, `async for`, e `async with`. Isso foi introduzido pela **PEP 492**.

CPython
A implementação canônica da linguagem de programação Python, como disponibilizada pelo python.org. O termo "CPython" é usado quando necessário distinguir esta implementação de outras como Jython ou IronPython.

contexto atual
O *contexto* (objeto `contextvars.Context`) que é usado atualmente pelos objetos `ContextVar` para acessar (obter ou definir) os valores de *variáveis de contexto*. Cada thread tem seu próprio contexto atual. Frameworks para executar tarefas assíncronas (veja `asyncio`) associam cada tarefa a um contexto que se torna o contexto atual sempre que a tarefa inicia ou retoma a execução.

isolado cíclico
Um subgrupo de um ou mais objetos que se referenciam em um ciclo de referência, mas não são referenciados por objetos de fora do grupo. O objetivo do *coletor de lixo cíclico* é identificar esses grupos e interromper os ciclos de referência para que a memória possa ser recuperada.

decorador
Uma função que retorna outra função, geralmente aplicada como uma transformação de função usando a sintaxe `@wrapper`. Exemplos comuns para decoradores são `classmethod()` e `staticmethod()`.

A sintaxe do decorador é meramente um açúcar sintático, as duas definições de funções a seguir são semanticamente equivalentes:

```
def f(arg):
    ...
f = staticmethod(f)

@staticmethod
def f(arg):
    ...
```

O mesmo conceito existe para as classes, mas não é comumente utilizado. Veja a documentação de definições de função e definições de classe para obter mais informações sobre decoradores.

descritor
Qualquer objeto que define os métodos `__get__()`, `__set__()` ou `__delete__()`. Quando um atributo de classe é um descritor, seu comportamento de associação especial é acionado no acesso a um atributo.

Normalmente, ao se utilizar *a.b* para se obter, definir ou excluir, um atributo dispara uma busca no objeto chamado *b* no dicionário de classe de *a*, mas se *b* for um descritor, o respectivo método descritor é chamado. Compreender descritores é a chave para um profundo entendimento de Python pois eles são a base de muitas funcionalidades incluindo funções, métodos, propriedades, métodos de classe, métodos estáticos e referências para superclasses.

Para obter mais informações sobre os métodos dos descritores, veja: descriptors ou o Guia de Descritores.

dicionário
Um vetor associativo em que chaves arbitrárias são mapeadas para valores. As chaves podem ser quaisquer objetos que possuam os métodos `__hash__()` e `__eq__()`. Isso é chamado de hash em Perl.

compreensão de dicionário
Uma maneira compacta de processar todos ou parte dos elementos de um iterável e retornar um dicionário com os resultados. `results = {n: n ** 2 for n in range(10)}` gera um dicionário contendo a chave `n` mapeada para o valor `n ** 2`. Veja comprehensions.

visão de dicionário
Os objetos retornados por `dict.keys()`, `dict.values()` e `dict.items()` são chamados de visões de dicionário. Eles fornecem uma visão dinâmica das entradas do dicionário, o que significa que quando o dicionário é alterado, a visão reflete essas alterações. Para forçar a visão de dicionário a se tornar uma lista completa use `list(dictview)`. Veja dict-views.

docstring
Abreviatura de "documentation string" (string de documentação). Uma string literal que aparece como primeira expressão numa classe, função ou módulo. Ainda que sejam ignoradas quando a suíte é executada, é reconhecida pelo compilador que a coloca no atributo `__doc__` da classe, função ou módulo que a encapsula. Como ficam disponíveis por meio de introspecção, docstrings são o lugar canônico para documentação do objeto.

tipagem pato
Também conhecida como *duck-typing*, é um estilo de programação que não verifica o tipo do objeto para determinar se ele possui a interface correta; em vez disso, o método ou atributo é simplesmente chamado ou utilizado ("Se se parece com um pato e grasna como um pato, então deve ser um pato.") Enfatizando interfaces ao invés de tipos específicos, o código bem desenvolvido aprimora sua flexibilidade por permitir substituição polimórfica. Tipagem pato evita necessidade de testes que usem `type()` ou `isinstance()`. (Note, porém, que a tipagem pato pode ser complementada com o uso de *classes base abstratas*.) Ao invés disso, são normalmente empregados testes `hasattr()` ou programação *EAFP*.

dunder
Uma abreviação informal para "double underscore" (em português, sublinhado duplo), usada ao se referir a um *método especial*. Por exemplo, `__init__` é frequentemente pronunciado como "dunder init".

EAFP
Iniciais da expressão em inglês "easier to ask for forgiveness than permission" que significa "é mais fácil pedir perdão que permissão". Este estilo de codificação comum no Python presume a existência de chaves ou atributos válidos e captura exceções caso essa premissa se prove falsa. Este estilo limpo e rápido se caracteriza pela presença de várias instruções `try` e `except`. A técnica diverge do estilo *LBYL*, comum em outras linguagens como C, por exemplo.

função de avaliação
Uma função que pode ser chamada para avaliar um atributo com avaliação preguiçosa de um objeto, como o valor de apelidos de tipo criados com a instrução `type`.

expressão
Uma parte da sintaxe que pode ser avaliada para algum valor. Em outras palavras, uma expressão é a acumulação de elementos de expressão como literais, nomes, atributos de acesso, operadores ou chamadas de funções, todos os quais retornam um valor. Em contraste com muitas outras linguagens, nem todas as construções de linguagem são expressões. Também existem *instruções*, as quais não podem ser usadas como expressões, como, por exemplo, `while`. Atribuições também são instruções, não expressões.

módulo de extensão
Um módulo escrito em C ou C++, usando a API C do Python para interagir tanto com código de usuário quanto do núcleo.

f-string
f-strings
Literais de string prefixados com `f` ou `F` são comumente chamados de "f-strings", abreviação em inglês de literais de strings formatadas. Veja também **PEP 498**.

objeto arquivo
Um objeto que expõe uma API orientada a arquivos (com métodos tais como `read()` ou `write()`) para um recurso subjacente. Dependendo da maneira como foi criado, um objeto arquivo pode mediar o acesso a um arquivo real no disco ou outro tipo de dispositivo de armazenamento ou de comunicação (por exemplo a entrada/saída padrão, buffers em memória, soquetes, pipes, etc.). Objetos arquivo também são chamados de *objetos arquivo ou similares* ou *fluxos*.

Atualmente há três categorias de objetos arquivo: *arquivos binários* brutos, *arquivos binários* em buffer e *arquivos textos*. Suas interfaces estão definidas no módulo `io`. A forma canônica para criar um objeto arquivo é usando a função `open()`.

objeto arquivo ou similar
Um sinônimo do termo *objeto arquivo*.

tratador de erros e codificação do sistema de arquivos
Tratador de erros e codificação usado pelo Python para decodificar bytes do sistema operacional e codificar Unicode para o sistema operacional.

A codificação do sistema de arquivos deve garantir a decodificação bem-sucedida de todos os bytes abaixo de 128. Se a codificação do sistema de arquivos falhar em fornecer essa garantia, as funções da API podem levantar `UnicodeError`.

As funções `sys.getfilesystemencoding()` e `sys.getfilesystemencodeerrors()` podem ser usadas para obter o tratador de erros e codificação do sistema de arquivos.

O *tratador de erros e codificação do sistema de arquivos* são configurados na inicialização do Python pela função `PyConfig_Read()`: veja os membros `filesystem_encoding` e `filesystem_errors` do `PyConfig`.

Veja também *codificação da localidade*.

localizador
Um objeto que tenta encontrar o *carregador* para um módulo que está sendo importado.

Existem dois tipos de localizador: *localizadores de metacaminho* para uso com `sys.meta_path`, e *localizadores de entrada de caminho* para uso com `sys.path_hooks`.

Veja finders-and-loaders e `importlib` para muito mais detalhes.

divisão pelo piso
Divisão matemática que arredonda para baixo para o inteiro mais próximo. O operador de divisão pelo piso é `//`. Por exemplo, a expressão `11 // 4` retorna o valor `2` ao invés de `2.75`, que seria retornado pela divisão de ponto flutuante. Note que `(-11) // 4` é `-3` porque é `-2.75` arredondado *para baixo*. Consulte a **PEP 238**.

threads livres
Um modelo de threads onde múltiplas threads podem simultaneamente executar bytecode Python no mesmo interpretador. Isso está em contraste com a *trava global do interpretador* que permite apenas uma thread por vez executar bytecode Python. Veja **PEP 703**.

variável livre
Formalmente, conforme definido no modelo de execução de linguagem, uma variável livre é qualquer variável usada em um espaço de nomes que não seja uma variável local naquele espaço de nomes. Veja *variável de clausura* para um exemplo. Pragmaticamente, devido ao nome do atributo `codeobject.co_freevars`, o termo também é usado algumas vezes como sinônimo de *variável de clausura*.

função
Uma série de instruções que retorna algum valor para um chamador. Também pode ser passado zero ou mais *argumentos* que podem ser usados na execução do corpo. Veja também *parâmetro*, *método* e a seção function.

anotação de função
Uma *anotação* de um parâmetro de função ou valor de retorno.

Anotações de função são comumente usados por *dicas de tipo*: por exemplo, essa função espera receber dois argumentos `int` e também é esperado que devolva um valor `int`:

```
def soma_dois_numeros(a: int, b: int) -> int:
   return a + b
```

A sintaxe de anotação de função é explicada na seção function.

Veja *anotação de variável* e **PEP 484**, que descrevem esta funcionalidade. Veja também annotations-howto para as melhores práticas sobre como trabalhar com anotações.

__future__
A instrução future, `from __future__ import <feature>`, direciona o compilador a compilar o módulo atual usando sintaxe ou semântica que será padrão em uma versão futura de Python. O módulo `__future__` documenta os possíveis valores de *feature*. Importando esse módulo e avaliando suas variáveis, você pode ver quando um novo recurso foi inicialmente adicionado à linguagem e quando será (ou se já é) o padrão:

```
>>> import __future__
>>> __future__.division
_Feature((2, 2, 0, 'alpha', 2), (3, 0, 0, 'alpha', 0), 8192)
```

coleta de lixo
Também conhecido como *garbage collection*, é o processo de liberar a memória quando ela não é mais utilizada. Python executa a liberação da memória através da contagem de referências e um coletor de lixo cíclico que é capaz de detectar e interromper referências cíclicas. O coletor de lixo pode ser controlado usando o módulo `gc`.

gerador
Uma função que retorna um *iterador gerador*. É parecida com uma função normal, exceto pelo fato de conter expressões `yield` para produzir uma série de valores que podem ser usados em um laço "for" ou que podem ser obtidos um de cada vez com a função `next()`.

Normalmente refere-se a uma função geradora, mas pode referir-se a um *iterador gerador* em alguns contextos. Em alguns casos onde o significado desejado não está claro, usar o termo completo evita ambiguidade.

iterador gerador
Um objeto criado por uma função *geradora*.

Cada `yield` suspende temporariamente o processamento, memorizando o estado da execução (incluindo variáveis locais e instruções try pendentes). Quando o *iterador gerador* retorna, ele se recupera do último ponto onde estava (em contrapartida as funções que iniciam uma nova execução a cada vez que são invocadas).

expressão geradora
Uma *expressão* que retorna um *iterador*. Parece uma expressão normal, seguido de uma cláusula `for` definindo uma variável de laço, um intervalo, e uma cláusula `if` opcional. A expressão combinada gera valores para uma função encapsuladora:

```
>>> sum(i*i for i in range(10))         # soma dos quadrados 0, 1, 4, ... 81
285
```

função genérica
Uma função composta por várias funções implementando a mesma operação para diferentes tipos. Qual implementação deverá ser usada durante a execução é determinada pelo algoritmo de despacho.

Veja também a entrada *despacho único* no glossário, o decorador `functools.singledispatch()`, e a **PEP 443**.

tipo genérico
Um *tipo* que pode ser parametrizado; tipicamente uma classe contêiner tal como `list` ou `dict`. Usado para *dicas de tipo* e *anotações*.

Para mais detalhes, veja tipo apelido genérico, **PEP 483**, **PEP 484**, **PEP 585**, e o módulo `typing`.

GIL
Veja *trava global do interpretador*.

trava global do interpretador
O mecanismo utilizado pelo interpretador *CPython* para garantir que apenas uma thread execute o *bytecode* Python por vez. Isto simplifica a implementação do CPython ao fazer com que o modelo de objetos (incluindo tipos embutidos críticos como o `dict`) ganhem segurança implícita contra acesso concorrente. Travar todo o interpretador facilita que o interpretador em si seja multithread, às custas de muito do paralelismo já provido por máquinas com multiprocessadores.

No entanto, alguns módulos de extensão, tanto da biblioteca padrão quanto de terceiros, são desenvolvidos de forma a liberar a GIL ao realizar tarefas computacionalmente muito intensas, como compactação ou cálculos de hash. Além disso, a GIL é sempre liberada nas operações de E/S.

A partir de Python 3.13, o GIL pode ser desabilitado usando a configuração de construção `--disable-gil`. Depois de construir Python com essa opção, o código deve ser executado com a opção `-X gil=0` ou a variável de ambiente `PYTHON_GIL=0` deve estar definida. Esse recurso provê um desempenho melhor para aplicações com múltiplas threads e torna mais fácil o uso eficiente de CPUs com múltiplos núcleos. Para mais detalhes, veja **PEP 703**.

Em versões anteriores da API C do Python, uma função podia declarar que precisava que a GIL fosse mantida para ser usada. Isso se refere a ter um *estado de thread anexado*.

pyc baseado em hash
Um arquivo de cache em bytecode que usa hash ao invés do tempo, no qual o arquivo de código-fonte foi modificado pela última vez, para determinar a sua validade. Veja pyc-invalidation.

hasheável
Um objeto é *hasheável* se tem um valor de hash que nunca muda durante seu ciclo de vida (precisa ter um método `__hash__()`) e pode ser comparado com outros objetos (precisa ter um método `__eq__()`). Objetos hasheáveis que são comparados como iguais devem ter o mesmo valor de hash.

A hasheabilidade faz com que um objeto possa ser usado como uma chave de dicionário e como um membro de conjunto, pois estas estruturas de dados utilizam os valores de hash internamente.

A maioria dos objetos embutidos imutáveis do Python são hasheáveis; contêineres mutáveis (tais como listas ou dicionários) não são; contêineres imutáveis (tais como tuplas e frozensets) são hasheáveis apenas se os seus elementos são hasheáveis. Objetos que são instâncias de classes definidas pelo usuário são hasheáveis por padrão. Todos eles comparam de forma desigual (exceto entre si mesmos), e o seu valor hash é derivado a partir do seu `id()`.

IDLE
Um ambiente de desenvolvimento e aprendizado integrado para Python. idle é um editor básico e um ambiente interpretador que vem junto com a distribuição padrão do Python.

imortal
Objetos imortais são um detalhe da implementação do CPython introduzida na **PEP 683**.

Se um objeto é imortal, sua *contagem de referências* nunca é modificada e, portanto, nunca é desalocado enquanto o interpretador está em execução. Por exemplo, `True` e `None` são imortais no CPython.

Objetos imortais podem ser identificados via `sys._is_immortal()` ou via `PyUnstable_IsImmortal()` na API C.

imutável
Um objeto que possui um valor fixo. Objetos imutáveis incluem números, strings e tuplas. Estes objetos não podem ser alterados. Um novo objeto deve ser criado se um valor diferente tiver de ser armazenado. Objetos imutáveis têm um papel importante em lugares onde um valor constante de hash seja necessário, como por exemplo uma chave em um dicionário.

caminho de importação
Uma lista de localizações (ou *entradas de caminho*) que são buscadas pelo *localizador baseado no caminho* por módulos para importar. Durante a importação, esta lista de localizações usualmente vem a partir de `sys.path`, mas para subpacotes ela também pode vir do atributo `__path__` de pacotes-pai.

importação
O processo pelo qual o código Python em um módulo é disponibilizado para o código Python em outro módulo.

importador
Um objeto que localiza e carrega um módulo; Tanto um *localizador* e o objeto *carregador*.

interativo
Python tem um interpretador interativo, o que significa que você pode digitar instruções e expressões no prompt do interpretador, executá-los imediatamente e ver seus resultados. Apenas execute `python` sem argumentos (possivelmente selecionando-o a partir do menu de aplicações de seu sistema operacional). O interpretador interativo é uma maneira poderosa de testar novas ideias ou aprender mais sobre módulos e pacotes (lembre-se do comando `help(x)`). Para saber mais sobre modo interativo, veja tut-interac.

interpretado
Python é uma linguagem interpretada, em oposição àquelas que são compiladas, embora esta distinção possa ser nebulosa devido à presença do compilador de bytecode. Isto significa que os arquivos-fontes podem ser executados diretamente sem necessidade explícita de se criar um arquivo executável. Linguagens interpretadas normalmente têm um ciclo de desenvolvimento/depuração mais curto que as linguagens compiladas, apesar de seus programas geralmente serem executados mais lentamente. Veja também *interativo*.

desligamento do interpretador
Quando solicitado para desligar, o interpretador Python entra em uma fase especial, onde ele gradualmente libera todos os recursos alocados, tais como módulos e várias estruturas internas críticas. Ele também faz diversas chamadas para o *coletor de lixo*. Isto pode disparar a execução de código em destrutores definidos pelo usuário ou função de retorno de referência fraca. Código executado durante a fase de desligamento pode encontrar diversas exceções, pois os recursos que ele depende podem não funcionar mais (exemplos comuns são os módulos de bibliotecas, ou os mecanismos de avisos).

A principal razão para o interpretador desligar, é que o módulo `__main__` ou o script sendo executado terminou sua execução.

iterável
Um objeto capaz de retornar seus membros um de cada vez. Exemplos de iteráveis incluem todos os tipos de sequência (tais como `list`, `str` e `tuple`) e alguns tipos de não-sequência, como o `dict` e *objetos arquivo*, além dos objetos de quaisquer classes que você definir com um método `__iter__()` ou `__getitem__()` que implementam a semântica de *sequência*.

Iteráveis podem ser usados em um laço `for` e em vários outros lugares em que uma sequência é necessária (`zip()`, `map()`, ...). Quando um objeto iterável é passado como argumento para a função embutida `iter()`, ela retorna um iterador para o objeto. Este iterador é adequado para se varrer todo o conjunto de valores. Ao usar iteráveis, normalmente não é necessário chamar `iter()` ou lidar com os objetos iteradores em si. A instrução `for` faz isso automaticamente para você, criando uma variável temporária para armazenar o iterador durante a execução do laço. Veja também *iterador*, *sequência*, e *gerador*.

iterador
Um objeto que representa um fluxo de dados. Repetidas chamadas ao método `__next__()` de um iterador (ou passando o objeto para a função embutida `next()`) vão retornar itens sucessivos do fluxo. Quando não houver mais dados disponíveis uma exceção `StopIteration` será levantada. Neste ponto, o objeto iterador se esgotou e quaisquer chamadas subsequentes a seu método `__next__()` vão apenas levantar a exceção `StopIteration` novamente. Iteradores precisam ter um método `__iter__()` que retorne o objeto iterador em si, de forma que todo iterador também é iterável e pode ser usado na maioria dos lugares em que um iterável é requerido. Uma notável exceção é código que tenta realizar passagens em múltiplas iterações. Um objeto contêiner (como uma `list`) produz um novo iterador a cada vez que você passá-lo para a função `iter()` ou utilizá-lo em um laço `for`. Tentar isso com o mesmo iterador apenas iria retornar o mesmo objeto iterador esgotado já utilizado na iteração anterior, como se fosse um contêiner vazio.

Mais informações podem ser encontradas em typeiter.

O CPython não aplica consistentemente o requisito de que um iterador defina `__iter__()`. E também observe que o CPython com threads livres não garante a segurança do thread das operações do iterador.

função chave
Uma função chave ou função colação é um chamável que retorna um valor usado para ordenação ou classifi-

cação. Por exemplo, `locale.strxfrm()` é usada para produzir uma chave de ordenação que leva o locale em consideração para fins de ordenação.

Uma porção de ferramentas no Python aceitam funções chave para controlar como os elementos são ordenados ou agrupados. Algumas delas incluem `min()`, `max()`, `sorted()`, `list.sort()`, `heapq.merge()`, `heapq.nsmallest()`, `heapq.nlargest()` e `itertools.groupby()`.

Há várias maneiras de se criar funções chave. Por exemplo, o método `str.lower()` pode servir como uma função chave para ordenações insensíveis à caixa. Alternativamente, uma função chave ad-hoc pode ser construída a partir de uma expressão `lambda`, como `lambda r: (r[0], r[2])`. Além disso, `operator.attrgetter()`, `operator.itemgetter()` e `operator.methodcaller()` são três construtores de função chave. Consulte o guia de Ordenação para ver exemplos de como criar e utilizar funções chave.

argumento nomeado
Veja *argumento*.

lambda
Uma função de linha anônima consistindo de uma única *expressão*, que é avaliada quando a função é chamada. A sintaxe para criar uma função lambda é `lambda [parameters]: expression`

LBYL
Iniciais da expressão em inglês "look before you leap", que significa algo como "olhe antes de pisar". Este estilo de codificação testa as pré-condições explicitamente antes de fazer chamadas ou buscas. Este estilo contrasta com a abordagem *EAFP* e é caracterizada pela presença de muitas instruções `if`.

Em um ambiente multithread, a abordagem LBYL pode arriscar a introdução de uma condição de corrida entre "o olhar" e "o pisar". Por exemplo, o código `if key in mapping: return mapping[key]` pode falhar se outra thread remover *key* do *mapping* após o teste, mas antes da olhada. Esse problema pode ser resolvido com travas ou usando a abordagem EAFP.

analisador léxico
Nome formal para o *tokenizador*; veja *token*.

lista
Uma *sequência* embutida no Python. Apesar do seu nome, é mais próximo de um vetor em outras linguagens do que uma lista encadeada, como o acesso aos elementos é da ordem $O(1)$.

compreensão de lista
Uma maneira compacta de processar todos ou parte dos elementos de uma sequência e retornar os resultados em uma lista. `result = ['{:#04x}'.format(x) for x in range(256) if x % 2 == 0]` gera uma lista de strings contendo números hexadecimais (0x..) no intervalo de 0 a 255. A cláusula `if` é opcional. Se omitida, todos os elementos no `range(256)` serão processados.

carregador
Um objeto que carrega um módulo. Ele deve definir os métodos `exec_module()` e `create_module()` para implementar a interface `Loader`. Um carregador é normalmente retornado por um *localizador*. Veja também:

- finders-and-loaders
- `importlib.abc.Loader`
- **PEP 302**

codificação da localidade
No Unix, é a codificação da localidade do LC_CTYPE, que pode ser definida com `locale.setlocale(locale.LC_CTYPE, new_locale)`.

No Windows, é a página de código ANSI (ex: `"cp1252"`).

No Android e no VxWorks, o Python usa `"utf-8"` como a codificação da localidade.

`locale.getencoding()` pode ser usado para obter a codificação da localidade.

Veja também *tratador de erros e codificação do sistema de arquivos*.

método mágico
Um sinônimo informal para um *método especial*.

mapeamento
Um objeto contêiner com suporte a pesquisas de chave arbitrária e que implementa os métodos especificados nas classes base abstratas `collections.abc.Mapping` ou `collections.abc.MutableMapping`. Exemplos incluem `dict`, `collections.defaultdict`, `collections.OrderedDict` e `collections.Counter`.

localizador de metacaminho
Um *localizador* retornado por uma busca de `sys.meta_path`. Localizadores de metacaminho são relacionados a, mas diferentes de, *localizadores de entrada de caminho*.

Veja `importlib.abc.MetaPathFinder` para os métodos que localizadores de metacaminho implementam.

metaclasse
A classe de uma classe. Definições de classe criam um nome de classe, um dicionário de classe e uma lista de classes base. A metaclasse é responsável por receber estes três argumentos e criar a classe. A maioria das linguagens de programação orientadas a objetos provê uma implementação default. O que torna o Python especial é o fato de ser possível criar metaclasses personalizadas. A maioria dos usuários nunca vai precisar deste recurso, mas quando houver necessidade, metaclasses possibilitam soluções poderosas e elegantes. Metaclasses têm sido utilizadas para gerar registros de acesso a atributos, para incluir proteção contra acesso concorrente, rastrear a criação de objetos, implementar singletons, dentre muitas outras tarefas.

Mais informações podem ser encontradas em metaclasses.

método
Uma função que é definida dentro do corpo de uma classe. Se chamada como um atributo de uma instância daquela classe, o método receberá a instância do objeto como seu primeiro *argumento* (que comumente é chamado de `self`). Veja *função* e *escopo aninhado*.

ordem de resolução de métodos
Ordem de resolução de métodos é a ordem em que os membros de uma classe base são buscados durante a pesquisa. Veja python_2.3_mro para detalhes do algoritmo usado pelo interpretador do Python desde a versão 2.3.

módulo
Um objeto que serve como uma unidade organizacional de código Python. Os módulos têm um espaço de nomes contendo objetos Python arbitrários. Os módulos são carregados pelo Python através do processo de *importação*.

Veja também *pacote*.

spec de módulo
Um espaço de nomes que contém as informações relacionadas à importação usadas para carregar um módulo. Uma instância de `importlib.machinery.ModuleSpec`.

Veja também module-specs.

MRO
Veja *ordem de resolução de métodos*.

mutável
Objeto mutável é aquele que pode modificar seus valor mas manter seu `id()`. Veja também *imutável*.

tupla nomeada
O termo "tupla nomeada" é aplicado a qualquer tipo ou classe que herda de tupla e cujos elementos indexáveis também são acessíveis usando atributos nomeados. O tipo ou classe pode ter outras funcionalidades também.

Diversos tipos embutidos são tuplas nomeadas, incluindo os valores retornados por `time.localtime()` e `os.stat()`. Outro exemplo é `sys.float_info`:

```
>>> sys.float_info[1]                    # acesso indexado
1024
>>> sys.float_info.max_exp               # acesso a campo nomeado
1024
>>> isinstance(sys.float_info, tuple)    # tipo de tupla
True
```

Algumas tuplas nomeadas são tipos embutidos (tal como os exemplos acima). Alternativamente, uma tupla nomeada pode ser criada a partir de uma definição de classe regular, que herde de `tuple` e que defina campos nomeados. Tal classe pode ser escrita a mão, ou ela pode ser criada herdando `typing.NamedTuple` ou com uma função fábrica `collections.namedtuple()`. As duas últimas técnicas também adicionam alguns métodos extras, que podem não ser encontrados quando foi escrita manualmente, ou em tuplas nomeadas embutidas.

espaço de nomes
O lugar em que uma variável é armazenada. Espaços de nomes são implementados como dicionários. Existem os espaços de nomes local, global e nativo, bem como espaços de nomes aninhados em objetos (em métodos). Espaços de nomes suportam modularidade ao prevenir conflitos de nomes. Por exemplo, as funções `__builtin__.open()` e `os.open()` são diferenciadas por seus espaços de nomes. Espaços de nomes também auxiliam na legibilidade e na manutenibilidade ao torar mais claro quais módulos implementam uma função. Escrever `random.seed()` ou `itertools.izip()`, por exemplo, deixa claro que estas funções são implementadas pelos módulos `random` e `itertools` respectivamente.

pacote de espaço de nomes
Um *pacote* que serve apenas como contêiner para subpacotes. Pacotes de espaços de nomes podem não ter representação física, e especificamente não são como um *pacote regular* porque eles não tem um arquivo `__init__.py`.

Pacotes de espaço de nomes permitem que vários pacotes instaláveis individualmente tenham um pacote pai comum. Caso contrário, é recomendado usar um *pacote regular*.

Para mais informações, veja **PEP 420** e reference-namespace-package.

Veja também *módulo*.

escopo aninhado
A habilidade de referir-se a uma variável em uma definição de fechamento. Por exemplo, uma função definida dentro de outra pode referenciar variáveis da função externa. Perceba que escopos aninhados por padrão funcionam apenas por referência e não por atribuição. Variáveis locais podem ler e escrever no escopo mais interno. De forma similar, variáveis globais podem ler e escrever para o espaço de nomes global. O `nonlocal` permite escrita para escopos externos.

classe estilo novo
Antigo nome para o tipo de classes agora usado para todos os objetos de classes. Em versões anteriores do Python, apenas classes estilo novo podiam usar recursos novos e versáteis do Python, tais como `__slots__`, descritores, propriedades, `__getattribute__()`, métodos de classe e métodos estáticos.

objeto
Qualquer dado que tenha estado (atributos ou valores) e comportamento definidos (métodos). Também a última classe base de qualquer *classe estilo novo*.

escopo otimizado
Um escopo no qual os nomes das variáveis locais de destino são conhecidos de forma confiável pelo compilador quando o código é compilado, permitindo a otimização do acesso de leitura e gravação a esses nomes. Os espaços de nomes locais para funções, geradores, corrotinas, compreensões e expressões geradoras são otimizados desta forma. Nota: a maioria das otimizações de interpretador são aplicadas a todos os escopos, apenas aquelas que dependem de um conjunto conhecido de nomes de variáveis locais e não locais são restritas a escopos otimizados.

pacote
Um *módulo* Python é capaz de conter submódulos ou recursivamente, subpacotes. Tecnicamente, um pacote é um módulo Python com um atributo `__path__`.

Veja também *pacote regular* e *pacote de espaço de nomes*.

parâmetro
Uma entidade nomeada na definição de uma *função* (ou método) que específica um *argumento* (ou em alguns casos, argumentos) que a função pode receber. Existem cinco tipos de parâmetros:

- *posicional-ou-nomeado*: especifica um argumento que pode ser tanto *posicional* quanto *nomeado*. Esse é o tipo padrão de parâmetro, por exemplo *foo* e *bar* a seguir:

```
def func(foo, bar=None): ...
```

- *somente-posicional*: especifica um argumento que pode ser fornecido apenas por posição. Parâmetros somente-posicionais podem ser definidos incluindo o caractere `/` na lista de parâmetros da definição da função após eles, por exemplo *somentepos1* e *somentepos2* a seguir:

```
def func(somentepos1, somentepos2, /, posicional_ou_nomeado): ...
```

- *somente-nomeado*: especifica um argumento que pode ser passado para a função somente por nome. Parâmetros somente-nomeados podem ser definidos com um simples parâmetro var-posicional ou um `*` antes deles na lista de parâmetros na definição da função, por exemplo *somente_nom1* and *somente_nom2* a seguir:

```
def func(arg, *, somente_nom1, somente_nom2): ...
```

- *var-posicional*: especifica que uma sequência arbitrária de argumentos posicionais pode ser fornecida (em adição a qualquer argumento posicional já aceito por outros parâmetros). Tal parâmetro pode ser definido colocando um `*` antes do nome do parâmetro, por exemplo *args* a seguir:

```
def func(*args, **kwargs): ...
```

- *var-nomeado*: especifica que, arbitrariamente, muitos argumentos nomeados podem ser fornecidos (em adição a qualquer argumento nomeado já aceito por outros parâmetros). Tal parâmetro pode ser definido colocando-se `**` antes do nome, por exemplo, *kwargs* no exemplo acima.

Parâmetros podem especificar tanto argumentos opcionais quanto obrigatórios, assim como valores padrão para alguns argumentos opcionais.

Veja também o termo *argumento* no glossário, a pergunta do FAQ sobre a diferença entre argumentos e parâmetros, a classe `inspect.Parameter`, a seção function e a **PEP 362**.

entrada de caminho
Um local único no *caminho de importação* que o *localizador baseado no caminho* consulta para encontrar módulos a serem importados.

localizador de entrada de caminho
Um *localizador* retornado por um chamável em `sys.path_hooks` (ou seja, um *gancho de entrada de caminho*) que sabe como localizar os módulos *entrada de caminho*.

Veja `importlib.abc.PathEntryFinder` para os métodos que localizadores de entrada de caminho implementam.

gancho de entrada de caminho
Um chamável na lista `sys.path_hooks` que retorna um *localizador de entrada de caminho* caso saiba como localizar módulos em uma *entrada de caminho* específica.

localizador baseado no caminho
Um dos *localizadores de metacaminho* padrão que procura por um *caminho de importação* de módulos.

objeto caminho ou similar
Um objeto representando um caminho de sistema de arquivos. Um objeto caminho ou similar é ou um objeto `str` ou `bytes` representando um caminho, ou um objeto implementando o protocolo `os.PathLike`. Um objeto que suporta o protocolo `os.PathLike` pode ser convertido para um arquivo de caminho do sistema `str` ou `bytes`, através da chamada da função `os.fspath()`; `os.fsdecode()` e `os.fsencode()` podem ser usadas para garantir um `str` ou `bytes` como resultado, respectivamente. Introduzido na **PEP 519**.

PEP
Proposta de melhoria do Python. Uma PEP é um documento de design que fornece informação para a comunidade Python, ou descreve uma nova funcionalidade para o Python ou seus predecessores ou ambientes. PEPs devem prover uma especificação técnica concisa e um racional para funcionalidades propostas.

PEPs têm a intenção de ser os mecanismos primários para propor novas funcionalidades significativas, para coletar opiniões da comunidade sobre um problema, e para documentar as decisões de design que foram

adicionadas ao Python. O autor da PEP é responsável por construir um consenso dentro da comunidade e documentar opiniões dissidentes.

Veja **PEP 1**.

porção
Um conjunto de arquivos em um único diretório (possivelmente armazenado em um arquivo zip) que contribuem para um pacote de espaço de nomes, conforme definido em **PEP 420**.

argumento posicional
Veja *argumento*.

API provisória
Uma API provisória é uma API que foi deliberadamente excluída das bibliotecas padrões com compatibilidade retroativa garantida. Enquanto mudanças maiores para tais interfaces não são esperadas, contanto que elas sejam marcadas como provisórias, mudanças retroativas incompatíveis (até e incluindo a remoção da interface) podem ocorrer se consideradas necessárias pelos desenvolvedores principais. Tais mudanças não serão feitas gratuitamente – elas irão ocorrer apenas se sérias falhas fundamentais forem descobertas, que foram esquecidas anteriormente a inclusão da API.

Mesmo para APIs provisórias, mudanças retroativas incompatíveis são vistas como uma "solução em último caso" - cada tentativa ainda será feita para encontrar uma resolução retroativa compatível para quaisquer problemas encontrados.

Esse processo permite que a biblioteca padrão continue a evoluir com o passar do tempo, sem se prender em erros de design problemáticos por períodos de tempo prolongados. Veja **PEP 411** para mais detalhes.

pacote provisório
Veja *API provisória*.

Python 3000
Apelido para a linha de lançamento da versão do Python 3.x (cunhada há muito tempo, quando o lançamento da versão 3 era algo em um futuro muito distante.) Esse termo possui a seguinte abreviação: "Py3k".

Pythônico
Uma ideia ou um pedaço de código que segue de perto as formas de escritas mais comuns da linguagem Python, ao invés de implementar códigos usando conceitos comuns a outras linguagens. Por exemplo, um formato comum em Python é fazer um laço sobre todos os elementos de uma iterável usando a instrução `for`. Muitas outras linguagens não têm esse tipo de construção, então as pessoas que não estão familiarizadas com o Python usam um contador numérico:

```
for i in range(len(comida)):
    print(comida[i])
```

Ao contrário do método mais limpo, Pythônico:

```
for parte in comida:
    print(parte)
```

nome qualificado
Um nome pontilhado (quando 2 termos são ligados por um ponto) que mostra o "path" do escopo global de um módulo para uma classe, função ou método definido num determinado módulo, conforme definido pela **PEP 3155**. Para funções e classes de nível superior, o nome qualificado é o mesmo que o nome do objeto:

```
>>> class C:
...     class D:
...         def metodo(self):
...             pass
...
>>> C.__qualname__
'C'
>>> C.D.__qualname__
```

(continua na próxima página)

(continuação da página anterior)

```
'C.D'
>>> C.D.metodo.__qualname__
'C.D.metodo'
```

Quando usado para se referir a módulos, o *nome totalmente qualificado* significa todo o caminho pontilhado para o módulo, incluindo quaisquer pacotes pai, por exemplo: `email.mime.text`:

```
>>> import email.mime.text
>>> email.mime.text.__name__
'email.mime.text'
```

contagem de referências
O número de referências a um objeto. Quando a contagem de referências de um objeto cai para zero, ele é desalocado. Alguns objetos são *imortais* e têm contagens de referências que nunca são modificadas e, portanto, os objetos nunca são desalocados. A contagem de referências geralmente não é visível para o código Python, mas é um elemento-chave da implementação do *CPython*. Os programadores podem chamar a função `sys.getrefcount()` para retornar a contagem de referências para um objeto específico.

Em *CPython*, as contagens de referências não são consideradas valores estáveis ou bem definidos; o número de referências a um objeto e como esse número é afetado pelo código Python pode ser diferente entre as versões.

pacote regular
Um *pacote* tradicional, como um diretório contendo um arquivo `__init__.py`.

Veja também *pacote de espaço de nomes*.

REPL
Um acrônimo para "read–eval–print loop", outro nome para o console *interativo* do interpretador.

__slots__
Uma declaração dentro de uma classe que economiza memória pré-declarando espaço para atributos de instâncias, e eliminando dicionários de instâncias. Apesar de popular, a técnica é um tanto quanto complicada de acertar, e é melhor se for reservada para casos raros, onde existe uma grande quantidade de instâncias em uma aplicação onde a memória é crítica.

sequência
Um *iterável* com suporte para acesso eficiente a seus elementos através de índices inteiros via método especial `__getitem__()` e que define o método `__len__()` que devolve o tamanho da sequência. Alguns tipos de sequência embutidos são: `list`, `str`, `tuple`, e `bytes`. Note que `dict` também tem suporte para `__getitem__()` e `__len__()`, mas é considerado um mapeamento e não uma sequência porque a busca usa uma chave *hasheável* arbitrária em vez de inteiros.

A classe base abstrata `collections.abc.Sequence` define uma interface mais rica que vai além de apenas `__getitem__()` e `__len__()`, adicionando `count()`, `index()`, `__contains__()`, e `__reversed__()`. Tipos que implementam essa interface podem ser explicitamente registrados usando `register()`. Para mais documentação sobre métodos de sequências em geral, veja Operações comuns de sequências.

compreensão de conjunto
Uma maneira compacta de processar todos ou parte dos elementos em iterável e retornar um conjunto com os resultados. `results = {c for c in 'abracadabra' if c not in 'abc'}` gera um conjunto de strings `{'r', 'd'}`. Veja comprehensions.

despacho único
Uma forma de despacho de *função genérica* onde a implementação é escolhida com base no tipo de um único argumento.

fatia
Um objeto geralmente contendo uma parte de uma *sequência*. Uma fatia é criada usando a notação de subscrito `[]` pode conter também até dois pontos entre números, como em `variable_name[1:3:5]`. A notação de suporte (subscrito) utiliza objetos `slice` internamente.

suavemente descontinuado
Uma API suavemente descontinuada não deve ser usada em código novo, mas é seguro para código já existente usá-la. A API continua documentada e testada, mas não será melhorada.

A descontinuação suave, diferentemente da descontinuação normal, não planeja remover a API e não emitirá avisos.

Veja PEP 387: Descontinuação suave.

método especial
Um método que é chamado implicitamente pelo Python para executar uma certa operação em um tipo, como uma adição por exemplo. Tais métodos tem nomes iniciando e terminando com dois underscores. Métodos especiais estão documentados em specialnames.

biblioteca padrão
A coleção de *pacotes*, *módulos* e *módulos de extensão* distribuída como parte do pacote do interpretador Python oficial. A relação exata de elementos da coleção pode variar de acordo com a plataforma, bibliotecas de sistema disponíveis ou outros critérios. A documentação pode ser encontrada em library-index.

Veja também `sys.stdlib_module_names` para obter uma lista de todos os possíveis nomes de módulos da biblioteca padrão.

instrução
Uma instrução é parte de uma suíte (um "bloco" de código). Uma instrução é ou uma *expressão* ou uma de várias construções com uma palavra reservada, tal como `if`, `while` ou `for`.

verificador de tipo estático
Uma ferramenta externa que lê código Python e o analisa, procurando por problemas como tipos incorretos. Consulte também *dicas de tipo* e o módulo `typing`.

stdlib
Uma abreviação para o termo *biblioteca padrão* em inglês (standard library).

referência forte
Na API C do Python, uma referência forte é uma referência a um objeto que pertence ao código que contém a referência. A referência forte é obtida chamando `Py_INCREF()` quando a referência é criada e liberada com `Py_DECREF()` quando a referência é excluída.

A função `Py_NewRef()` pode ser usada para criar uma referência forte para um objeto. Normalmente, a função `Py_DECREF()` deve ser chamada na referência forte antes de sair do escopo da referência forte, para evitar o vazamento de uma referência.

Veja também *referência emprestada*.

t-string
t-strings
Literais de string prefixados com `t` ou `T` são comumente chamados de "t-strings", que é uma abreviação de literais de strings templates.

codificador de texto
Uma string em Python é uma sequência de pontos de código Unicode (no intervalo `U+0000–U+10FFFF`). Para armazenar ou transferir uma string, ela precisa ser serializada como uma sequência de bytes.

A serialização de uma string em uma sequência de bytes é conhecida como "codificação" e a recriação da string a partir de uma sequência de bytes é conhecida como "decodificação".

Há uma variedade de diferentes serializações de texto codecs, que são coletivamente chamadas de "codificações de texto".

arquivo texto
Um *objeto arquivo* apto a ler e escrever objetos `str`. Geralmente, um arquivo texto, na verdade, acessa um fluxo de dados de bytes e captura o *codificador de texto* automaticamente. Exemplos de arquivos texto são: arquivos abertos em modo texto (`'r'` or `'w'`), `sys.stdin`, `sys.stdout`, e instâncias de `io.StringIO`.

Veja também *arquivo binário* para um objeto arquivo apto a ler e escrever *objetos bytes ou similares*.

estado de thread
: As informações usadas pelo ambiente de execução do *CPython* para executar em uma thread do sistema operacional. Por exemplo, isso inclui a exceção atual, se houver, e o estado do interpretador de bytecode.

  Cada estado de thread está vinculado a uma única thread do SO, mas as threads podem ter muitos estados de thread disponíveis. No máximo, um deles pode ser *anexado* por vez.

  Um *estado de thread anexado* é necessário para chamar a maioria da API C do Python, a menos que uma função documente explicitamente o contrário. O interpretador de bytecode só é executado em um estado de thread anexado.

  Cada estado de thread pertence a um único interpretador, mas cada interpretador pode ter vários estados de thread, incluindo vários para a mesma thread do sistema operacional. Estados de thread de vários interpretadores podem ser vinculados à mesma thread, mas apenas um pode ser *anexado* nessa thread a qualquer momento.

  Veja Estado de thread e a trava global do interpretador para mais informações.

token
: Uma pequena unidade de código-fonte, gerada pelo analisador léxico (também chamado de *tokenizador*). Nomes, números, strings, operadores, quebras de linha e similares são representados por tokens.

  O módulo `tokenize` expõe o analisador léxico do Python. O módulo `token` contém informações sobre os vários tipos de tokens.

aspas triplas
: Uma string que está definida com três ocorrências de aspas duplas (") ou apóstrofos ('). Enquanto elas não fornecem nenhuma funcionalidade não disponível com strings de aspas simples, elas são úteis para inúmeras razões. Elas permitem que você inclua aspas simples e duplas não escapadas dentro de uma string, e elas podem utilizar múltiplas linhas sem o uso de caractere de continuação, fazendo-as especialmente úteis quando escrevemos documentação em docstrings.

tipo
: O tipo de um objeto Python determina qual classe de objeto ele é; cada objeto tem um tipo. Um tipo de objeto é acessível pelo atributo `__class__` ou pode ser recuperado com `type(obj)`.

apelido de tipo
: Um sinônimo para um tipo, criado através da atribuição do tipo para um identificador.

  Apelidos de tipo são úteis para simplificar *dicas de tipo*. Por exemplo:

```
def remove_tons_de_cinza(
        cores: list[tuple[int, int, int]]) -> list[tuple[int, int, int]]:
    pass
```

  pode tornar-se mais legível desta forma:

```
Cor = tuple[int, int, int]

def remove_tons_de_cinza(cores: list[Cor]) -> list[Cor]:
    pass
```

  Veja `typing` e **PEP 484**, a qual descreve esta funcionalidade.

dica de tipo
: Uma *anotação* que especifica o tipo esperado para uma variável, um atributo de classe, ou um parâmetro de função ou um valor de retorno.

  Dicas de tipo são opcionais e não são forçadas pelo Python, mas elas são úteis para *verificadores de tipo estático*. Eles também ajudam IDEs a completar e refatorar código.

  Dicas de tipos de variáveis globais, atributos de classes, e funções, mas não de variáveis locais, podem ser acessadas usando `typing.get_type_hints()`.

  Veja `typing` e **PEP 484**, a qual descreve esta funcionalidade.

novas linhas universais
: Uma maneira de interpretar fluxos de textos, na qual todos estes são reconhecidos como caracteres de fim de linha: a convenção para fim de linha no Unix `'\n'`, a convenção no Windows `'\r\n'`, e a antiga convenção no Macintosh `'\r'`. Veja **PEP 278** e **PEP 3116**, bem como `bytes.splitlines()` para uso adicional.

anotação de variável
: Uma *anotação* de uma variável ou um atributo de classe.

  Ao fazer uma anotação de uma variável ou um atributo de classe, a atribuição é opcional:

```
class C:
    campo: 'anotação'
```

  Anotações de variáveis são normalmente usadas para *dicas de tipo*: por exemplo, espera-se que esta variável receba valores do tipo `int`:

```
contagem: int = 0
```

  A sintaxe de anotação de variável é explicada na seção annassign.

  Veja *anotação de função*, **PEP 484** e **PEP 526**, que descrevem esta funcionalidade. Veja também annotations-howto para as melhores práticas sobre como trabalhar com anotações.

ambiente virtual
: Um ambiente de execução isolado que permite usuários Python e aplicações instalarem e atualizarem pacotes Python sem interferir no comportamento de outras aplicações Python em execução no mesmo sistema.

  Veja também `venv`.

máquina virtual
: Um computador definido inteiramente em software. A máquina virtual de Python executa o *bytecode* emitido pelo compilador de bytecode.

operador morsa
: Uma maneira bem-humorada de se referir ao operador da expressão de atribuição `:=` porque ele se parece um pouco com uma morsa se você virar a cabeça.

Zen do Python
: Lista de princípios de projeto e filosofias do Python que são úteis para a compreensão e uso da linguagem. A lista é exibida quando se digita "`import this`" no console interativo.

APÊNDICE B

Sobre esta documentação

A documentação do Python é gerada a partir de fontes reStructuredText usando Sphinx, um gerador de documentação criado originalmente para Python e agora mantido como um projeto independente.

O desenvolvimento da documentação e de suas ferramentas é um esforço totalmente voluntário, como Python em si. Se você quer contribuir, por favor dê uma olhada na página reporting-bugs para informações sobre como fazer. Novos voluntários são sempre bem-vindos!

Agradecimentos especiais para:

- Fred L. Drake, Jr., o criador do primeiro conjunto de ferramentas para documentar Python e autor de boa parte do conteúdo;
- O projeto Docutils por criar reStructuredText e o pacote Docutils;
- Fredrik Lundh, pelo seu projeto de referência alternativa em Python, do qual Sphinx pegou muitas boas ideias.

B.1 Contribuidores da documentação do Python

Muitas pessoas tem contribuído para a linguagem Python, sua biblioteca padrão e sua documentação. Veja Misc/ACKS na distribuição do código do Python para ver uma lista parcial de contribuidores.

Tudo isso só foi possível com o esforço e a contribuição da comunidade Python, por isso temos essa maravilhosa documentação – Obrigado a todos!

APÊNDICE C

História e Licença

C.1 História do software

Python foi criado no início dos anos 1990 por Guido van Rossum no Stichting Mathematisch Centrum (CWI, veja https://www.cwi.nl) na Holanda como sucessor de uma linguagem chamada ABC. Guido continua sendo o principal autor do Python, embora inclua muitas contribuições de outros.

Em 1995, Guido continuou seu trabalho em Python na Corporation for National Research Initiatives (CNRI, veja https://www.cnri.reston.va.us) em Reston, Virgínia, onde lançou várias versões do software.

Em maio de 2000, Guido e a equipe de desenvolvimento do núcleo Python mudaram-se para BeOpen.com para formar a equipe BeOpen PythonLabs. Em outubro do mesmo ano, a equipe PythonLabs mudou-se para a Digital Creations, que se tornou Zope Corporation. Em 2001, a Python Software Foundation (PSF, veja https://www.python.org/psf/) foi formada, uma organização sem fins lucrativos criada especificamente para possuir Propriedade Intelectual relacionada ao Python. A Zope Corporation era um membro patrocinador da PSF.

Todas as versões do Python são de código aberto (consulte https://opensource.org para a definição de código aberto). Historicamente, a maioria, mas não todas, versões do Python também são compatíveis com GPL; a tabela abaixo resume os vários lançamentos.

Versão	Derivada de	Ano	Proprietário	Compatível com a GPL? (1)
0.9.0 a 1.2	n/a	1991-1995	CWI	sim
1.3 a 1.5.2	1.2	1995-1999	CNRI	sim
1.6	1.5.2	2000	CNRI	não
2.0	1.6	2000	BeOpen.com	não
1.6.1	1.6	2001	CNRI	sim (2)
2.1	2.0+1.6.1	2001	PSF	não
2.0.1	2.0+1.6.1	2001	PSF	sim
2.1.1	2.1+2.0.1	2001	PSF	sim
2.1.2	2.1.1	2002	PSF	sim
2.1.3	2.1.2	2002	PSF	sim
2.2 e acima	2.1.1	2001-agora	PSF	sim

Nota

(1) Compatível com a GPL não significa que estamos distribuindo Python sob a GPL. Todas as licenças do Python, ao contrário da GPL, permitem distribuir uma versão modificada sem fazer alterações em código aberto. As licenças compatíveis com a GPL possibilitam combinar o Python com outro software lançado sob a GPL; os outros não.

(2) De acordo com Richard Stallman, 1.6.1 não é compatível com GPL, porque sua licença tem uma cláusula de escolha de lei. De acordo com a CNRI, no entanto, o advogado de Stallman disse ao advogado da CNRI que 1.6.1 "não é incompatível" com a GPL.

Graças aos muitos voluntários externos que trabalharam sob a direção de Guido para tornar esses lançamentos possíveis.

C.2 Termos e condições para acessar ou usar Python

O software e a documentação do Python são licenciados sob a Python Software Foundation License Versão 2.

A partir do Python 3.8.6, exemplos, receitas e outros códigos na documentação são licenciados duplamente sob o Licença PSF versão 2 e a *Licença BSD de Zero Cláusula*.

Alguns softwares incorporados ao Python estão sob licenças diferentes. As licenças são listadas com o código abrangido por essa licença. Veja *Licenças e Reconhecimentos para Software Incorporado* para uma lista incompleta dessas licenças.

C.2.1 PYTHON SOFTWARE FOUNDATION LICENSE VERSION 2

```
1. This LICENSE AGREEMENT is between the Python Software Foundation ("PSF"), and
   the Individual or Organization ("Licensee") accessing and otherwise using this
   software ("Python") in source or binary form and its associated documentation.

2. Subject to the terms and conditions of this License Agreement, PSF hereby
   grants Licensee a nonexclusive, royalty-free, world-wide license to reproduce,
   analyze, test, perform and/or display publicly, prepare derivative works,
   distribute, and otherwise use Python alone or in any derivative
   version, provided, however, that PSF's License Agreement and PSF's notice of
   copyright, i.e., "Copyright © 2001 Python Software Foundation; All Rights
   Reserved" are retained in Python alone or in any derivative version
   prepared by Licensee.

3. In the event Licensee prepares a derivative work that is based on or
   incorporates Python or any part thereof, and wants to make the
   derivative work available to others as provided herein, then Licensee hereby
   agrees to include in any such work a brief summary of the changes made to␣
↪Python.

4. PSF is making Python available to Licensee on an "AS IS" basis.
   PSF MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED.  BY WAY OF
   EXAMPLE, BUT NOT LIMITATION, PSF MAKES NO AND DISCLAIMS ANY REPRESENTATION OR
   WARRANTY OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR THAT THE
   USE OF PYTHON WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.

5. PSF SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF PYTHON
   FOR ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS A RESULT OF
   MODIFYING, DISTRIBUTING, OR OTHERWISE USING PYTHON, OR ANY DERIVATIVE
   THEREOF, EVEN IF ADVISED OF THE POSSIBILITY THEREOF.

6. This License Agreement will automatically terminate upon a material breach of
   its terms and conditions.
```

(continua na próxima página)

(continuação da página anterior)

```
7. Nothing in this License Agreement shall be deemed to create any relationship
   of agency, partnership, or joint venture between PSF and Licensee.  This License
   Agreement does not grant permission to use PSF trademarks or trade name in a
   trademark sense to endorse or promote products or services of Licensee, or any
   third party.

8. By copying, installing or otherwise using Python, Licensee agrees
   to be bound by the terms and conditions of this License Agreement.
```

C.2.2 ACORDO DE LICENCIAMENTO DA BEOPEN.COM PARA PYTHON 2.0

ACORDO DE LICENCIAMENTO DA BEOPEN DE FONTE ABERTA DO PYTHON VERSÃO 1

```
1. This LICENSE AGREEMENT is between BeOpen.com ("BeOpen"), having an office at
   160 Saratoga Avenue, Santa Clara, CA 95051, and the Individual or Organization
   ("Licensee") accessing and otherwise using this software in source or binary
   form and its associated documentation ("the Software").

2. Subject to the terms and conditions of this BeOpen Python License Agreement,
   BeOpen hereby grants Licensee a non-exclusive, royalty-free, world-wide license
   to reproduce, analyze, test, perform and/or display publicly, prepare derivative
   works, distribute, and otherwise use the Software alone or in any derivative
   version, provided, however, that the BeOpen Python License is retained in the
   Software, alone or in any derivative version prepared by Licensee.

3. BeOpen is making the Software available to Licensee on an "AS IS" basis.
   BEOPEN MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED.  BY WAY OF
   EXAMPLE, BUT NOT LIMITATION, BEOPEN MAKES NO AND DISCLAIMS ANY REPRESENTATION OR
   WARRANTY OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR THAT THE
   USE OF THE SOFTWARE WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.

4. BEOPEN SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF THE SOFTWARE FOR
   ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS A RESULT OF USING,
   MODIFYING OR DISTRIBUTING THE SOFTWARE, OR ANY DERIVATIVE THEREOF, EVEN IF
   ADVISED OF THE POSSIBILITY THEREOF.

5. This License Agreement will automatically terminate upon a material breach of
   its terms and conditions.

6. This License Agreement shall be governed by and interpreted in all respects
   by the law of the State of California, excluding conflict of law provisions.
   Nothing in this License Agreement shall be deemed to create any relationship of
   agency, partnership, or joint venture between BeOpen and Licensee.  This License
   Agreement does not grant permission to use BeOpen trademarks or trade names in a
   trademark sense to endorse or promote products or services of Licensee, or any
   third party.  As an exception, the "BeOpen Python" logos available at
   http://www.pythonlabs.com/logos.html may be used according to the permissions
   granted on that web page.

7. By copying, installing or otherwise using the software, Licensee agrees to be
   bound by the terms and conditions of this License Agreement.
```

C.2.3 CONTRATO DE LICENÇA DA CNRI PARA O PYTHON 1.6.1

```
1. This LICENSE AGREEMENT is between the Corporation for National Research
   Initiatives, having an office at 1895 Preston White Drive, Reston, VA 20191
   ("CNRI"), and the Individual or Organization ("Licensee") accessing and
   otherwise using Python 1.6.1 software in source or binary form and its
   associated documentation.

2. Subject to the terms and conditions of this License Agreement, CNRI hereby
   grants Licensee a nonexclusive, royalty-free, world-wide license to reproduce,
   analyze, test, perform and/or display publicly, prepare derivative works,
   distribute, and otherwise use Python 1.6.1 alone or in any derivative version,
   provided, however, that CNRI's License Agreement and CNRI's notice of copyright,
   i.e., "Copyright © 1995-2001 Corporation for National Research Initiatives; All
   Rights Reserved" are retained in Python 1.6.1 alone or in any derivative version
   prepared by Licensee.  Alternately, in lieu of CNRI's License Agreement,
   Licensee may substitute the following text (omitting the quotes): "Python 1.6.1
   is made available subject to the terms and conditions in CNRI's License
   Agreement.  This Agreement together with Python 1.6.1 may be located on the
   internet using the following unique, persistent identifier (known as a handle):
   1895.22/1013.  This Agreement may also be obtained from a proxy server on the
   internet using the following URL: http://hdl.handle.net/1895.22/1013".

3. In the event Licensee prepares a derivative work that is based on or
   incorporates Python 1.6.1 or any part thereof, and wants to make the derivative
   work available to others as provided herein, then Licensee hereby agrees to
   include in any such work a brief summary of the changes made to Python 1.6.1.

4. CNRI is making Python 1.6.1 available to Licensee on an "AS IS" basis.  CNRI
   MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED.  BY WAY OF EXAMPLE,
   BUT NOT LIMITATION, CNRI MAKES NO AND DISCLAIMS ANY REPRESENTATION OR WARRANTY
   OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR THAT THE USE OF
   PYTHON 1.6.1 WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.

5. CNRI SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF PYTHON 1.6.1 FOR
   ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS A RESULT OF
   MODIFYING, DISTRIBUTING, OR OTHERWISE USING PYTHON 1.6.1, OR ANY DERIVATIVE
   THEREOF, EVEN IF ADVISED OF THE POSSIBILITY THEREOF.

6. This License Agreement will automatically terminate upon a material breach of
   its terms and conditions.

7. This License Agreement shall be governed by the federal intellectual property
   law of the United States, including without limitation the federal copyright
   law, and, to the extent such U.S. federal law does not apply, by the law of the
   Commonwealth of Virginia, excluding Virginia's conflict of law provisions.
   Notwithstanding the foregoing, with regard to derivative works based on Python
   1.6.1 that incorporate non-separable material that was previously distributed
   under the GNU General Public License (GPL), the law of the Commonwealth of
   Virginia shall govern this License Agreement only as to issues arising under or
   with respect to Paragraphs 4, 5, and 7 of this License Agreement.  Nothing in
   this License Agreement shall be deemed to create any relationship of agency,
   partnership, or joint venture between CNRI and Licensee.  This License Agreement
   does not grant permission to use CNRI trademarks or trade name in a trademark
   sense to endorse or promote products or services of Licensee, or any third
   party.
```

(continua na próxima página)

(continuação da página anterior)

```
8. By clicking on the "ACCEPT" button where indicated, or by copying, installing
   or otherwise using Python 1.6.1, Licensee agrees to be bound by the terms and
   conditions of this License Agreement.
```

C.2.4 ACORDO DE LICENÇA DA CWI PARA PYTHON 0.9.0 A 1.2

```
Copyright © 1991 - 1995, Stichting Mathematisch Centrum Amsterdam, The
Netherlands.  All rights reserved.

Permission to use, copy, modify, and distribute this software and its
documentation for any purpose and without fee is hereby granted, provided that
the above copyright notice appear in all copies and that both that copyright
notice and this permission notice appear in supporting documentation, and that
the name of Stichting Mathematisch Centrum or CWI not be used in advertising or
publicity pertaining to distribution of the software without specific, written
prior permission.

STICHTING MATHEMATISCH CENTRUM DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS
SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS, IN NO
EVENT SHALL STICHTING MATHEMATISCH CENTRUM BE LIABLE FOR ANY SPECIAL, INDIRECT
OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE,
DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS
ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS
SOFTWARE.
```

C.2.5 ZERO-CLAUSE BSD LICENSE FOR CODE IN THE PYTHON DOCUMENTATION

```
Permission to use, copy, modify, and/or distribute this software for any
purpose with or without fee is hereby granted.

THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES WITH
REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY
AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY SPECIAL, DIRECT,
INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM
LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR
OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR
PERFORMANCE OF THIS SOFTWARE.
```

C.3 Licenças e Reconhecimentos para Software Incorporado

Esta seção é uma lista incompleta, mas crescente, de licenças e reconhecimentos para softwares de terceiros incorporados na distribuição do Python.

C.3.1 Mersenne Twister

A extensão C `_random` subjacente ao módulo `random` inclui código baseado em um download de http://www.math.sci.hiroshima-u.ac.jp/~m-mat/MT/MT2002/emt19937ar.html. A seguir estão os comentários literais do código original:

```
A C-program for MT19937, with initialization improved 2002/1/26.
Coded by Takuji Nishimura and Makoto Matsumoto.

Before using, initialize the state by using init_genrand(seed)
```

(continua na próxima página)

(continuação da página anterior)

```
or init_by_array(init_key, key_length).

Copyright (C) 1997 - 2002, Makoto Matsumoto and Takuji Nishimura,
All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:

 1. Redistributions of source code must retain the above copyright
    notice, this list of conditions and the following disclaimer.

 2. Redistributions in binary form must reproduce the above copyright
    notice, this list of conditions and the following disclaimer in the
    documentation and/or other materials provided with the distribution.

 3. The names of its contributors may not be used to endorse or promote
    products derived from this software without specific prior written
    permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
A PARTICULAR PURPOSE ARE DISCLAIMED.  IN NO EVENT SHALL THE COPYRIGHT OWNER OR
CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL,
EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO,
PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR
PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF
LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.


Any feedback is very welcome.
http://www.math.sci.hiroshima-u.ac.jp/~m-mat/MT/emt.html
email: m-mat @ math.sci.hiroshima-u.ac.jp (remove space)
```

C.3.2 Soquetes

O módulo `socket` usa as funções `getaddrinfo()` e `getnameinfo()`, que são codificadas em arquivos de origem separados do Projeto WIDE, https://www.wide.ad.jp/.

```
Copyright (C) 1995, 1996, 1997, and 1998 WIDE Project.
All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:
1. Redistributions of source code must retain the above copyright
   notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright
   notice, this list of conditions and the following disclaimer in the
   documentation and/or other materials provided with the distribution.
3. Neither the name of the project nor the names of its contributors
   may be used to endorse or promote products derived from this software
   without specific prior written permission.
```

(continua na próxima página)

(continuação da página anterior)

```
THIS SOFTWARE IS PROVIDED BY THE PROJECT AND CONTRIBUTORS "AS IS" AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
ARE DISCLAIMED.  IN NO EVENT SHALL THE PROJECT OR CONTRIBUTORS BE LIABLE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
SUCH DAMAGE.
```

C.3.3 Serviços de soquete assíncrono

Os módulos `test.support.asynchat` e `test.support.asyncore` contêm o seguinte aviso:

```
Copyright 1996 by Sam Rushing

                        All Rights Reserved

Permission to use, copy, modify, and distribute this software and
its documentation for any purpose and without fee is hereby
granted, provided that the above copyright notice appear in all
copies and that both that copyright notice and this permission
notice appear in supporting documentation, and that the name of Sam
Rushing not be used in advertising or publicity pertaining to
distribution of the software without specific, written prior
permission.

SAM RUSHING DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS SOFTWARE,
INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS, IN
NO EVENT SHALL SAM RUSHING BE LIABLE FOR ANY SPECIAL, INDIRECT OR
CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS
OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT,
NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF OR IN
CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.
```

C.3.4 Gerenciamento de cookies

O módulo `http.cookies` contém o seguinte aviso:

```
Copyright 2000 by Timothy O'Malley <timo@alum.mit.edu>

               All Rights Reserved

Permission to use, copy, modify, and distribute this software
and its documentation for any purpose and without fee is hereby
granted, provided that the above copyright notice appear in all
copies and that both that copyright notice and this permission
notice appear in supporting documentation, and that the name of
Timothy O'Malley  not be used in advertising or publicity
pertaining to distribution of the software without specific, written
prior permission.

Timothy O'Malley DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS
```

(continua na próxima página)

(continuação da página anterior)

```
SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY
AND FITNESS, IN NO EVENT SHALL Timothy O'Malley BE LIABLE FOR
ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES
WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS,
WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS
ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR
PERFORMANCE OF THIS SOFTWARE.
```

C.3.5 Rastreamento de execução

O módulo `trace` contém o seguinte aviso:

```
portions copyright 2001, Autonomous Zones Industries, Inc., all rights...
err...  reserved and offered to the public under the terms of the
Python 2.2 license.
Author: Zooko O'Whielacronx
http://zooko.com/
mailto:zooko@zooko.com

Copyright 2000, Mojam Media, Inc., all rights reserved.
Author: Skip Montanaro

Copyright 1999, Bioreason, Inc., all rights reserved.
Author: Andrew Dalke

Copyright 1995-1997, Automatrix, Inc., all rights reserved.
Author: Skip Montanaro

Copyright 1991-1995, Stichting Mathematisch Centrum, all rights reserved.


Permission to use, copy, modify, and distribute this Python software and
its associated documentation for any purpose without fee is hereby
granted, provided that the above copyright notice appears in all copies,
and that both that copyright notice and this permission notice appear in
supporting documentation, and that the name of neither Automatrix,
Bioreason or Mojam Media be used in advertising or publicity pertaining to
distribution of the software without specific, written prior permission.
```

C.3.6 Funções UUencode e UUdecode

O codec `uu` contém o seguinte aviso:

```
Copyright 1994 by Lance Ellinghouse
Cathedral City, California Republic, United States of America.
                       All Rights Reserved
Permission to use, copy, modify, and distribute this software and its
documentation for any purpose and without fee is hereby granted,
provided that the above copyright notice appear in all copies and that
both that copyright notice and this permission notice appear in
supporting documentation, and that the name of Lance Ellinghouse
not be used in advertising or publicity pertaining to distribution
of the software without specific, written prior permission.
LANCE ELLINGHOUSE DISCLAIMS ALL WARRANTIES WITH REGARD TO
THIS SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS, IN NO EVENT SHALL LANCE ELLINGHOUSE CENTRUM BE LIABLE
```

(continua na próxima página)

(continuação da página anterior)

```
FOR ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES
WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN
ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT
OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.

Modified by Jack Jansen, CWI, July 1995:
- Use binascii module to do the actual line-by-line conversion
  between ascii and binary. This results in a 1000-fold speedup. The C
  version is still 5 times faster, though.
- Arguments more compliant with Python standard
```

C.3.7 Chamadas de procedimento remoto XML

O módulo `xmlrpc.client` contém o seguinte aviso:

```
    The XML-RPC client interface is

Copyright (c) 1999-2002 by Secret Labs AB
Copyright (c) 1999-2002 by Fredrik Lundh

By obtaining, using, and/or copying this software and/or its
associated documentation, you agree that you have read, understood,
and will comply with the following terms and conditions:

Permission to use, copy, modify, and distribute this software and
its associated documentation for any purpose and without fee is
hereby granted, provided that the above copyright notice appears in
all copies, and that both that copyright notice and this permission
notice appear in supporting documentation, and that the name of
Secret Labs AB or the author not be used in advertising or publicity
pertaining to distribution of the software without specific, written
prior permission.

SECRET LABS AB AND THE AUTHOR DISCLAIMS ALL WARRANTIES WITH REGARD
TO THIS SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANT-
ABILITY AND FITNESS.  IN NO EVENT SHALL SECRET LABS AB OR THE AUTHOR
BE LIABLE FOR ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY
DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS,
WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS
ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE
OF THIS SOFTWARE.
```

C.3.8 test_epoll

O módulo `test.test_epoll` contém o seguinte aviso:

```
Copyright (c) 2001-2006 Twisted Matrix Laboratories.

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:
```

(continua na próxima página)

(continuação da página anterior)

```
The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE
LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION
OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

C.3.9 kqueue de seleção

O módulo `select` contém o seguinte aviso para a interface do kqueue:

```
Copyright (c) 2000 Doug White, 2006 James Knight, 2007 Christian Heimes
All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:
1. Redistributions of source code must retain the above copyright
   notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright
   notice, this list of conditions and the following disclaimer in the
   documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS "AS IS" AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
ARE DISCLAIMED.  IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
SUCH DAMAGE.
```

C.3.10 SipHash24

O arquivo `Python/pyhash.c` contém a implementação de Marek Majkowski do algoritmo SipHash24 de Dan Bernstein. Contém a seguinte nota:

```
<MIT License>
Copyright (c) 2013  Marek Majkowski <marek@popcount.org>

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.
```

(continua na próxima página)

(continuação da página anterior)

```
</MIT License>

Original location:
   https://github.com/majek/csiphash/

Solution inspired by code from:
   Samuel Neves (supercop/crypto_auth/siphash24/little)
   djb (supercop/crypto_auth/siphash24/little2)
   Jean-Philippe Aumasson (https://131002.net/siphash/siphash24.c)
```

C.3.11 strtod e dtoa

O arquivo `Python/dtoa.c`, que fornece as funções C dtoa e strtod para conversão de duplas de C para e de strings, é derivado do arquivo com o mesmo nome de David M. Gay, atualmente disponível em https://web.archive.org/web/20220517033456/http://www.netlib.org/fp/dtoa.c. O arquivo original, conforme recuperado em 16 de março de 2009, contém os seguintes avisos de direitos autorais e de licenciamento:

```
/****************************************************************
 *
 * The author of this software is David M. Gay.
 *
 * Copyright (c) 1991, 2000, 2001 by Lucent Technologies.
 *
 * Permission to use, copy, modify, and distribute this software for any
 * purpose without fee is hereby granted, provided that this entire notice
 * is included in all copies of any software which is or includes a copy
 * or modification of this software and in all copies of the supporting
 * documentation for such software.
 *
 * THIS SOFTWARE IS BEING PROVIDED "AS IS", WITHOUT ANY EXPRESS OR IMPLIED
 * WARRANTY.  IN PARTICULAR, NEITHER THE AUTHOR NOR LUCENT MAKES ANY
 * REPRESENTATION OR WARRANTY OF ANY KIND CONCERNING THE MERCHANTABILITY
 * OF THIS SOFTWARE OR ITS FITNESS FOR ANY PARTICULAR PURPOSE.
 *
 ***************************************************************/
```

C.3.12 OpenSSL

Os módulos `hashlib`, `posix` e `ssl` usam a biblioteca OpenSSL para desempenho adicional se forem disponibilizados pelo sistema operacional. Além disso, os instaladores do Windows e do Mac OS X para Python podem incluir uma cópia das bibliotecas do OpenSSL, portanto incluímos uma cópia da licença do OpenSSL aqui: Para o lançamento do OpenSSL 3.0, e lançamentos posteriores derivados deste, se aplica a Apache License v2:

```
                                 Apache License
                           Version 2.0, January 2004
                        https://www.apache.org/licenses/

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

   "License" shall mean the terms and conditions for use, reproduction,
   and distribution as defined by Sections 1 through 9 of this document.

   "Licensor" shall mean the copyright owner or entity authorized by
   the copyright owner that is granting the License.
```

(continua na próxima página)

(continuação da página anterior)

```
   "Legal Entity" shall mean the union of the acting entity and all
   other entities that control, are controlled by, or are under common
   control with that entity. For the purposes of this definition,
   "control" means (i) the power, direct or indirect, to cause the
   direction or management of such entity, whether by contract or
   otherwise, or (ii) ownership of fifty percent (50%) or more of the
   outstanding shares, or (iii) beneficial ownership of such entity.

   "You" (or "Your") shall mean an individual or Legal Entity
   exercising permissions granted by this License.

   "Source" form shall mean the preferred form for making modifications,
   including but not limited to software source code, documentation
   source, and configuration files.

   "Object" form shall mean any form resulting from mechanical
   transformation or translation of a Source form, including but
   not limited to compiled object code, generated documentation,
   and conversions to other media types.

   "Work" shall mean the work of authorship, whether in Source or
   Object form, made available under the License, as indicated by a
   copyright notice that is included in or attached to the work
   (an example is provided in the Appendix below).

   "Derivative Works" shall mean any work, whether in Source or Object
   form, that is based on (or derived from) the Work and for which the
   editorial revisions, annotations, elaborations, or other modifications
   represent, as a whole, an original work of authorship. For the purposes
   of this License, Derivative Works shall not include works that remain
   separable from, or merely link (or bind by name) to the interfaces of,
   the Work and Derivative Works thereof.

   "Contribution" shall mean any work of authorship, including
   the original version of the Work and any modifications or additions
   to that Work or Derivative Works thereof, that is intentionally
   submitted to Licensor for inclusion in the Work by the copyright owner
   or by an individual or Legal Entity authorized to submit on behalf of
   the copyright owner. For the purposes of this definition, "submitted"
   means any form of electronic, verbal, or written communication sent
   to the Licensor or its representatives, including but not limited to
   communication on electronic mailing lists, source code control systems,
   and issue tracking systems that are managed by, or on behalf of, the
   Licensor for the purpose of discussing and improving the Work, but
   excluding communication that is conspicuously marked or otherwise
   designated in writing by the copyright owner as "Not a Contribution."

   "Contributor" shall mean Licensor and any individual or Legal Entity
   on behalf of whom a Contribution has been received by Licensor and
   subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of
   this License, each Contributor hereby grants to You a perpetual,
   worldwide, non-exclusive, no-charge, royalty-free, irrevocable
   copyright license to reproduce, prepare Derivative Works of,
```

(continua na próxima página)

(continuação da página anterior)

```
   publicly display, publicly perform, sublicense, and distribute the
   Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of
   this License, each Contributor hereby grants to You a perpetual,
   worldwide, non-exclusive, no-charge, royalty-free, irrevocable
   (except as stated in this section) patent license to make, have made,
   use, offer to sell, sell, import, and otherwise transfer the Work,
   where such license applies only to those patent claims licensable
   by such Contributor that are necessarily infringed by their
   Contribution(s) alone or by combination of their Contribution(s)
   with the Work to which such Contribution(s) was submitted. If You
   institute patent litigation against any entity (including a
   cross-claim or counterclaim in a lawsuit) alleging that the Work
   or a Contribution incorporated within the Work constitutes direct
   or contributory patent infringement, then any patent licenses
   granted to You under this License for that Work shall terminate
   as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the
   Work or Derivative Works thereof in any medium, with or without
   modifications, and in Source or Object form, provided that You
   meet the following conditions:

   (a) You must give any other recipients of the Work or
       Derivative Works a copy of this License; and

   (b) You must cause any modified files to carry prominent notices
       stating that You changed the files; and

   (c) You must retain, in the Source form of any Derivative Works
       that You distribute, all copyright, patent, trademark, and
       attribution notices from the Source form of the Work,
       excluding those notices that do not pertain to any part of
       the Derivative Works; and

   (d) If the Work includes a "NOTICE" text file as part of its
       distribution, then any Derivative Works that You distribute must
       include a readable copy of the attribution notices contained
       within such NOTICE file, excluding those notices that do not
       pertain to any part of the Derivative Works, in at least one
       of the following places: within a NOTICE text file distributed
       as part of the Derivative Works; within the Source form or
       documentation, if provided along with the Derivative Works; or,
       within a display generated by the Derivative Works, if and
       wherever such third-party notices normally appear. The contents
       of the NOTICE file are for informational purposes only and
       do not modify the License. You may add Your own attribution
       notices within Derivative Works that You distribute, alongside
       or as an addendum to the NOTICE text from the Work, provided
       that such additional attribution notices cannot be construed
       as modifying the License.

   You may add Your own copyright statement to Your modifications and
   may provide additional or different license terms and conditions
   for use, reproduction, or distribution of Your modifications, or
```

(continua na próxima página)

(continuação da página anterior)

```
   for any such Derivative Works as a whole, provided Your use,
   reproduction, and distribution of the Work otherwise complies with
   the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise,
   any Contribution intentionally submitted for inclusion in the Work
   by You to the Licensor shall be under the terms and conditions of
   this License, without any additional terms or conditions.
   Notwithstanding the above, nothing herein shall supersede or modify
   the terms of any separate license agreement you may have executed
   with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade
   names, trademarks, service marks, or product names of the Licensor,
   except as required for reasonable and customary use in describing the
   origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or
   agreed to in writing, Licensor provides the Work (and each
   Contributor provides its Contributions) on an "AS IS" BASIS,
   WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or
   implied, including, without limitation, any warranties or conditions
   of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A
   PARTICULAR PURPOSE. You are solely responsible for determining the
   appropriateness of using or redistributing the Work and assume any
   risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory,
   whether in tort (including negligence), contract, or otherwise,
   unless required by applicable law (such as deliberate and grossly
   negligent acts) or agreed to in writing, shall any Contributor be
   liable to You for damages, including any direct, indirect, special,
   incidental, or consequential damages of any character arising as a
   result of this License or out of the use or inability to use the
   Work (including but not limited to damages for loss of goodwill,
   work stoppage, computer failure or malfunction, or any and all
   other commercial damages or losses), even if such Contributor
   has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing
   the Work or Derivative Works thereof, You may choose to offer,
   and charge a fee for, acceptance of support, warranty, indemnity,
   or other liability obligations and/or rights consistent with this
   License. However, in accepting such obligations, You may act only
   on Your own behalf and on Your sole responsibility, not on behalf
   of any other Contributor, and only if You agree to indemnify,
   defend, and hold each Contributor harmless for any liability
   incurred by, or claims asserted against, such Contributor by reason
   of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS
```

C.3.13 expat

A extensão `pyexpat` é construída usando uma cópia incluída das fontes de expatriadas, a menos que a compilação esteja configurada `--with-system-expat`:

```
Copyright (c) 1998, 1999, 2000 Thai Open Source Software Center Ltd
                               and Clark Cooper

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be included
in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

C.3.14 libffi

A extensão C `_ctypes` subjacente ao módulo `ctypes` é construída usando uma cópia incluída das fontes do libffi, a menos que a construção esteja configurada com `--with-system-libffi`:

```
Copyright (c) 1996-2008  Red Hat, Inc and others.

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be included
in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT.  IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT
HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN THE SOFTWARE.
```

C.3.15 zlib

A extensão `zlib` é construída usando uma cópia incluída das fontes zlib se a versão do zlib encontrada no sistema for muito antiga para ser usada na construção:

```
Copyright (C) 1995-2011 Jean-loup Gailly and Mark Adler

This software is provided 'as-is', without any express or implied
warranty.  In no event will the authors be held liable for any damages
arising from the use of this software.

Permission is granted to anyone to use this software for any purpose,
including commercial applications, and to alter it and redistribute it
freely, subject to the following restrictions:

1. The origin of this software must not be misrepresented; you must not
   claim that you wrote the original software. If you use this software
   in a product, an acknowledgment in the product documentation would be
   appreciated but is not required.

2. Altered source versions must be plainly marked as such, and must not be
   misrepresented as being the original software.

3. This notice may not be removed or altered from any source distribution.

Jean-loup Gailly        Mark Adler
jloup@gzip.org          madler@alumni.caltech.edu
```

C.3.16 cfuhash

A implementação da tabela de hash usada pelo `tracemalloc` é baseada no projeto cfuhash:

```
Copyright (c) 2005 Don Owens
All rights reserved.

This code is released under the BSD license:

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:

  * Redistributions of source code must retain the above copyright
    notice, this list of conditions and the following disclaimer.

  * Redistributions in binary form must reproduce the above
    copyright notice, this list of conditions and the following
    disclaimer in the documentation and/or other materials provided
    with the distribution.

  * Neither the name of the author nor the names of its
    contributors may be used to endorse or promote products derived
    from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS
FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE
COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT,
```

(continua na próxima página)

(continuação da página anterior)

```
INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED
OF THE POSSIBILITY OF SUCH DAMAGE.
```

C.3.17 libmpdec

A extensão C `_decimal` subjacente ao módulo `decimal` é construída usando uma cópia incluída da biblioteca libmpdec, a menos que a construção esteja configurada com `--with-system-libmpdec`:

```
Copyright (c) 2008-2020 Stefan Krah. All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:

1. Redistributions of source code must retain the above copyright
   notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright
   notice, this list of conditions and the following disclaimer in the
   documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS "AS IS" AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
ARE DISCLAIMED.  IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
SUCH DAMAGE.
```

C.3.18 Conjunto de testes C14N do W3C

O conjunto de testes C14N 2.0 no pacote `test` (`Lib/test/xmltestdata/c14n-20/`) foi recuperado do site do W3C em https://www.w3.org/TR/xml-c14n2-testcases/ e é distribuído sob a licença BSD de 3 cláusulas:

```
Copyright (c) 2013 W3C(R) (MIT, ERCIM, Keio, Beihang),
All Rights Reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:

* Redistributions of works must retain the original copyright notice,
  this list of conditions and the following disclaimer.
* Redistributions in binary form must reproduce the original copyright
  notice, this list of conditions and the following disclaimer in the
  documentation and/or other materials provided with the distribution.
* Neither the name of the W3C nor the names of its contributors may be
```

(continua na próxima página)

(continuação da página anterior)

```
  used to endorse or promote products derived from this work without
  specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
```

C.3.19 mimalloc

Licença MIT:

```
Copyright (c) 2018-2021 Microsoft Corporation, Daan Leijen

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

C.3.20 asyncio

Partes do módulo `asyncio` são incorporadas do uvloop 0.16, que é distribuído sob a licença MIT:

```
Copyright (c) 2015-2021 MagicStack Inc.  http://magic.io

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.
```

(continua na próxima página)

(continuação da página anterior)

```
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE
LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION
OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

C.3.21 Global Unbounded Sequences (GUS)

O arquivo `Python/qsbr.c` é adaptado do esquema de recuperação de memória segura "Global Unbounded Sequences" do FreeBSD em subr_smr.c. O arquivo é distribuído sob a licença BSD de 2 cláusulas:

```
Copyright (c) 2019,2020 Jeffrey Roberson <jeff@FreeBSD.org>

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:
1. Redistributions of source code must retain the above copyright
   notice unmodified, this list of conditions, and the following
   disclaimer.
2. Redistributions in binary form must reproduce the above copyright
   notice, this list of conditions and the following disclaimer in the
   documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR "AS IS" AND ANY EXPRESS OR
IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES
OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED.
IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT, INDIRECT,
INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT
NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF
THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
```

C.3.22 Ligações Zstandard

As ligações de Zstandard em `Modules/_zstd` e `Lib/compression/zstd` são baseadas no código da biblioteca pyzstd, com direitos autorais de Ma Lin e colaboradores. O código do pyzstd é distribuído sob a Licença BSD de 3 Cláusulas:

```
Copyright (c) 2020-present, Ma Lin and contributors.
All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this
   list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice,
   this list of conditions and the following disclaimer in the documentation
   and/or other materials provided with the distribution.

3. Neither the name of the copyright holder nor the names of its
```

(continua na próxima página)

(continuação da página anterior)

```
   contributors may be used to endorse or promote products derived from
   this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS"
AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER
CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY,
OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
```

APÊNDICE D

Direitos autorais

Python e essa documentação é:

Copyright © 2001 Python Software Foundation. Todos os direitos reservados.

Copyright © 2000 BeOpen.com. Todos os direitos reservados.

Copyright © 1995-2000 Corporation for National Research Initiatives. Todos os direitos reservados.

Copyright © 1991-1995 Stichting Mathematisch Centrum. Todos os direitos reservados.

Veja: *História e Licença* para informações completas de licença e permissões.

Índice

Não alfabético

... , **99**
    literal de reticências, 99
-?
    opção de linha de comando, 5
>>>, **99**
__future__, **106**
__slots__, **114**

A

aguardável, **101**
ambiente virtual, **117**
analisador léxico, **109**
anotação, **99**
anotação de função, **105**
anotação de variável, **117**
apelido de tipo, **116**
API provisória, **113**
argumento, **99**
argumento nomeado, **109**
argumento posicional, **113**
arquivo binário, **101**
arquivo texto, **115**
aspas triplas, **116**
atributo, **101**

B

-B
    opção de linha de comando, 6
-b
    opção de linha de comando, 6
BDFL, **101**
biblioteca padrão, **115**
BOLT_APPLY_FLAGS
    opção de linha de comando, 33
BOLT_INSTRUMENT_FLAGS
    opção de linha de comando, 33
--build
    opção de linha de comando, 39
bytecode, **101**
BZIP2_CFLAGS
    opção de linha de comando, 30
BZIP2_LIBS
    opção de linha de comando, 30

C

-c
    opção de linha de comando, 4
caminho de importação, **107**
carregador, **109**
CC
    opção de linha de comando, 29
CFLAGS, 32, 43, 44
    opção de linha de comando, 29
CFLAGS_NODIST, 43, 44
chamável, **101**
--check-hash-based-pycs
    opção de linha de comando, 6
classe, **102**
classe base abstrata, **99**
classe estilo novo, **111**
codificação da localidade, **109**
codificador de texto, **115**
coleta de lixo, **106**
compreensão de conjunto, **114**
compreensão de dicionário, **104**
compreensão de lista, **109**
CONFIG_SITE
    opção de linha de comando, 39
contagem de referências, **114**
contexto, **102**
contexto atual, **103**
contíguo, **103**
contíguo C, 103
contíguo Fortran, 103
corrotina, **103**
CPP
    opção de linha de comando, 29
CPPFLAGS, 42, 44
    opção de linha de comando, 29
CPython, **103**
CURSES_CFLAGS
    opção de linha de comando, 30
CURSES_LIBS
    opção de linha de comando, 30

D

-d

opção de linha de comando, 6
decorador, 103
descritor, 103
desligamento do interpretador, 108
despacho único, 114
dica de tipo, 116
dicionário, 104
--disable-gil
opção de linha de comando, 29
--disable-ipv6
opção de linha de comando, 26
--disable-safety
opção de linha de comando, 37
--disable-test-modules
opção de linha de comando, 32
divisão pelo piso, 105
docstring, 104
dunder, 104

E

-E
opção de linha de comando, 6
EAFP, 104
--enable-big-digits
opção de linha de comando, 26
--enable-bolt
opção de linha de comando, 33
--enable-experimental-jit
opção de linha de comando, 29
--enable-framework
opção de linha de comando, 38, 39
--enable-loadable-sqlite-extensions
opção de linha de comando, 26
--enable-optimizations
opção de linha de comando, 32
--enable-profiling
opção de linha de comando, 34
--enable-pystats
opção de linha de comando, 27
--enable-shared
opção de linha de comando, 36
--enable-slower-safety
opção de linha de comando, 38
--enable-universalsdk
opção de linha de comando, 38
--enable-wasm-dynamic-linking
opção de linha de comando, 31
--enable-wasm-pthreads
opção de linha de comando, 31
entrada de caminho, 112
escopo aninhado, 111
escopo otimizado, 111
espaço de nomes, 111
especial
método, 115
estado de thread, 116
estado de thread anexado, 100
--exec-prefix
opção de linha de comando, 32
expressão, 104
expressão geradora, 106

F

f-string, 105
f-strings, 105
fatia, 114
função, 105
função chave, 108
função de anotação, 99
função de avaliação, 104
função de corrotina, 103
função de retorno, 102
função genérica, 106

G

gancho de entrada de caminho, 112
GDBM_CFLAGS
opção de linha de comando, 30
GDBM_LIBS
opção de linha de comando, 30
gerador, 106
gerador assíncrono, 100
gerenciador de contexto, 103
gerenciador de contexto assíncrono, 100
GIL, 107

H

-h
opção de linha de comando, 5
hasheável, 107
--help
opção de linha de comando, 5
--help-all
opção de linha de comando, 5
--help-env
opção de linha de comando, 5
--help-xoptions
opção de linha de comando, 5
--host
opção de linha de comando, 39
HOSTRUNNER
opção de linha de comando, 39

I

-I
opção de linha de comando, 7
-i
opção de linha de comando, 6
IDLE, 107
imortal, 107
importação, 108
importador, 108
imutável, 107
instrução, 115
interativo, 108
interpretado, 108

isolado cíclico, **103**
iterador, **108**
iterador assíncrono, **100**
iterador gerador, **106**
iterador gerador assíncrono, **100**
iterável, **108**
iterável assíncrono, **100**

L

lambda, **109**
LBYL, **109**
LDFLAGS, 42, 44
    opção de linha de comando, 30
LDFLAGS_NODIST, 44
LIBB2_CFLAGS
    opção de linha de comando, 30
LIBB2_LIBS
    opção de linha de comando, 30
LIBEDIT_CFLAGS
    opção de linha de comando, 30
LIBEDIT_LIBS
    opção de linha de comando, 30
LIBFFI_CFLAGS
    opção de linha de comando, 30
LIBFFI_LIBS
    opção de linha de comando, 30
LIBLZMA_CFLAGS
    opção de linha de comando, 31
LIBLZMA_LIBS
    opção de linha de comando, 31
LIBMPDEC_CFLAGS
    opção de linha de comando, 30
LIBMPDEC_LIBS
    opção de linha de comando, 30
LIBREADLINE_CFLAGS
    opção de linha de comando, 31
LIBREADLINE_LIBS
    opção de linha de comando, 31
LIBS
    opção de linha de comando, 30
LIBSQLITE3_CFLAGS
    opção de linha de comando, 31
LIBSQLITE3_LIBS
    opção de linha de comando, 31
LIBUUID_CFLAGS
    opção de linha de comando, 31
LIBUUID_LIBS
    opção de linha de comando, 31
LIBZSTD_CFLAGS
    opção de linha de comando, 31
LIBZSTD_LIBS
    opção de linha de comando, 31
lista, **109**
localizador, **105**
localizador baseado no caminho, **112**
localizador de entrada de caminho, **112**
localizador de metacaminho, **110**

M

-m
    opção de linha de comando, 4
MACHDEP
    opção de linha de comando, 30
mágico
    método, 109
mapeamento, **110**
máquina virtual, **117**
metaclasse, **110**
método, **110**
    especial, 115
    mágico, 109
método especial, **115**
método mágico, **109**
módulo, **110**
módulo de extensão, **104**
MRO, **110**
mutável, **110**

N

nome qualificado, **113**
novas linhas universais, **117**
número complexo, **102**

O

-O
    opção de linha de comando, 7
objeto, **111**
objeto arquivo, **105**
objeto arquivo ou similar, **105**
objeto bytes ou similar, **101**
objeto caminho ou similar, **112**
-OO
    opção de linha de comando, 7
opção de linha de comando
    -?, 5
    -B, 6
    -b, 6
    BOLT_APPLY_FLAGS, 33
    BOLT_INSTRUMENT_FLAGS, 33
    --build, 39
    BZIP2_CFLAGS, 30
    BZIP2_LIBS, 30
    -c, 4
    CC, 29
    CFLAGS, 29
    --check-hash-based-pycs, 6
    CONFIG_SITE, 39
    CPP, 29
    CPPFLAGS, 29
    CURSES_CFLAGS, 30
    CURSES_LIBS, 30
    -d, 6
    --disable-gil, 29
    --disable-ipv6, 26
    --disable-safety, 37
    --disable-test-modules, 32

-E, 6
--enable-big-digits, 26
--enable-bolt, 33
--enable-experimental-jit, 29
--enable-framework, 38, 39
--enable-loadable-sqlite-extensions, 26
--enable-optimizations, 32
--enable-profiling, 34
--enable-pystats, 27
--enable-shared, 36
--enable-slower-safety, 38
--enable-universalsdk, 38
--enable-wasm-dynamic-linking, 31
--enable-wasm-pthreads, 31
--exec-prefix, 32
GDBM_CFLAGS, 30
GDBM_LIBS, 30
-h, 5
--help, 5
--help-all, 5
--help-env, 5
--help-xoptions, 5
--host, 39
HOSTRUNNER, 39
-I, 7
-i, 6
LDFLAGS, 30
LIBB2_CFLAGS, 30
LIBB2_LIBS, 30
LIBEDIT_CFLAGS, 30
LIBEDIT_LIBS, 30
LIBFFI_CFLAGS, 30
LIBFFI_LIBS, 30
LIBLZMA_CFLAGS, 31
LIBLZMA_LIBS, 31
LIBMPDEC_CFLAGS, 30
LIBMPDEC_LIBS, 30
LIBREADLINE_CFLAGS, 31
LIBREADLINE_LIBS, 31
LIBS, 30
LIBSQLITE3_CFLAGS, 31
LIBSQLITE3_LIBS, 31
LIBUUID_CFLAGS, 31
LIBUUID_LIBS, 31
LIBZSTD_CFLAGS, 31
LIBZSTD_LIBS, 31
-m, 4
MACHDEP, 30
-O, 7
-OO, 7
-P, 7
PANEL_CFLAGS, 31
PANEL_LIBS, 31
PKG_CONFIG, 29
PKG_CONFIG_LIBDIR, 29
PKG_CONFIG_PATH, 29
--prefix, 32
-q, 7
-R, 7
-S, 8
-s, 7
TCLTK_CFLAGS, 31
TCLTK_LIBS, 31
-u, 8
-V, 6
-v, 8
--version, 6
-W, 8
--with-address-sanitizer, 35
--with-app-store-compliance, 39
--with-assertions, 35
--with-build-python, 39
--with-builtin-hashlib-hashes, 37
--with-computed-gotos, 33
--with-dbmliborder, 27
--with-dtrace, 35
--with-ensurepip, 32
--with-framework-name, 39
--with-hash-algorithm, 37
--with-libc, 36
--with-libm, 36
--with-libs, 36
--with-lto, 33
--with-memory-sanitizer, 35
--with-openssl, 37
--with-openssl-rpath, 37
--without-c-locale-coercion, 27
--without-decimal-contextvar, 27
--without-doc-strings, 33
--without-mimalloc, 33
--without-pymalloc, 33
--without-readline, 36
--without-remote-debug, 34
--without-static-libpython, 36
--with-pkg-config, 27
--with-platlibdir, 27
--with-pydebug, 35
--with-readline, 36
--with-ssl-default-suites, 37
--with-strict-overflow, 34
--with-suffix, 26
--with-system-expat, 36
--with-system-libmpdec, 36
--with-tail-call-interp, 33
--with-thread-sanitizer, 36
--with-trace-refs, 35
--with-tzpath, 26
--with-undefined-behavior-sanitizer, 35
--with-universal-archs, 38
--with-valgrind, 35
--with-wheel-pkg-dir, 27
-X, 9
-x, 9
ZLIB_CFLAGS, 31

ZLIB_LIBS, 31
operador morsa, **117**
OPT, 35
ordem de resolução de métodos, **110**

P

-P
opção de linha de comando, 7
pacote, **111**
pacote de espaço de nomes, **111**
pacote provisório, **113**
pacote regular, **114**
PANEL_CFLAGS
opção de linha de comando, 31
PANEL_LIBS
opção de linha de comando, 31
parâmetro, **111**
PATH, 12, 23, 48, 53, 54, 56, 57, 64, 66, 69, 71
PATHEXT, 66
PEP, **112**
PKG_CONFIG
opção de linha de comando, 29
PKG_CONFIG_LIBDIR
opção de linha de comando, 29
PKG_CONFIG_PATH
opção de linha de comando, 29
porção, **113**
--prefix
opção de linha de comando, 32
PROFILE_TASK, 32
Propostas de Melhorias do Python
PEP 1, 113
PEP 7, 25
PEP 8, 97
PEP 11, 25, 60
PEP 238, 105
PEP 278, 117
PEP 302, 109
PEP 338, 4
PEP 343, 102, 103
PEP 362, 100, 112
PEP 370, 8, 14
PEP 397, 69
PEP 411, 113
PEP 420, 111, 113
PEP 443, 106
PEP 483, 106
PEP 484, 99, 106, 116, 117
PEP 488, 7
PEP 492, 100, 101, 103
PEP 498, 105
PEP 514, 69
PEP 519, 112
PEP 525, 100
PEP 526, 99, 117
PEP 528, 61
PEP 529, 15, 61
PEP 538, 16, 27
PEP 585, 106
PEP 649, 99
PEP 683, 107
PEP 703, 68, 84, 105, 107
PEP 768, 10, 17, 34
PEP 3116, 117
PEP 3155, 113
protocolo de gerenciamento de contexto, **102**
Py_REMOTE_DEBUG (*C macro*), 34
pyc baseado em hash, **107**
Python 3000, **113**
PYTHON_COLORS, 11
PYTHON_CONTEXT_AWARE_WARNINGS, 11
PYTHON_CPU_COUNT, 10
PYTHON_DISABLE_REMOTE_DEBUG, 10
PYTHON_FROZEN_MODULES, 10
PYTHON_GIL, 11, 107
PYTHON_JIT, 29
PYTHON_MANAGER_DEFAULT, 48, 57
PYTHON_PERF_JIT_SUPPORT, 10
PYTHON_PRESITE, 11
PYTHON_THREAD_INHERIT_CONTEXT, 11
PYTHON_TLBC, 11
PYTHONCOERCECLOCALE, 27
PYTHONDEBUG, 6, 34
PYTHONDEVMODE, 9
PYTHONDONTWRITEBYTECODE, 6
PYTHONDUMPREFS, 35
PYTHONFAULTHANDLER, 9
PYTHONHASHSEED, 7, 13
PYTHONHOME, 6, 12, 61, 62, 94
Pythônico, **113**
PYTHONINSPECT, 7
PYTHONINTMAXSTRDIGITS, 9
PYTHONIOENCODING, 16
PYTHONLEGACYWINDOWSSTDIO, 14
PYTHONMALLOC, 15, 33
PYTHONNODEBUGRANGES, 10
PYTHONNOUSERSITE, 8
PYTHONOPTIMIZE, 7
PYTHONPATH, 6, 12, 61, 62, 94
PYTHONPERFSUPPORT, 10
PYTHONPROFILEIMPORTTIME, 9
PYTHONPYCACHEPREFIX, 10
PYTHONSAFEPATH, 7
PYTHONSTARTUP, 7, 13
PYTHONTRACEMALLOC, 9
PYTHONUNBUFFERED, 8
PYTHONUTF8, 10, 16, 60
PYTHONVERBOSE, 8
PYTHONWARNDEFAULTENCODING, 10
PYTHONWARNINGS, 9

Q

-q
opção de linha de comando, 7

R

-R
    opção de linha de comando, 7
referência emprestada, **101**
referência forte, **115**
REPL, **114**

S

-S
    opção de linha de comando, 8
-s
    opção de linha de comando, 7
sequência, **114**
spec de módulo, **110**
stdlib, **115**
suavemente descontinuado, **115**

T

t-string, **115**
t-strings, **115**
TCLTK_CFLAGS
    opção de linha de comando, 31
TCLTK_LIBS
    opção de linha de comando, 31
threads livres, **105**
tipagem pato, **104**
tipo, **116**
tipo genérico, **106**
token, **116**
tratador de erros e codificação do sistema de arquivos, **105**
trava global do interpretador, **107**
tupla nomeada, **110**

U

-u
    opção de linha de comando, 8

V

-V
    opção de linha de comando, 6
-v
    opção de linha de comando, 8
variável de ambiente
    BASECFLAGS, 43
    BASECPPFLAGS, 42
    BLDSHARED, 45
    CC, 42
    CCSHARED, 43
    CFLAGS, 32, 4244
    CFLAGS_ALIASING, 43
    CFLAGS_NODIST, 43, 44
    CFLAGSFORSHARED, 43
    COMPILEALL_OPTS, 43
    CONFIGURE_CFLAGS, 43
    CONFIGURE_CFLAGS_NODIST, 43
    CONFIGURE_CPPFLAGS, 42
    CONFIGURE_LDFLAGS, 44
    CONFIGURE_LDFLAGS_NODIST, 44
    CPPFLAGS, 42, 44
    CXX, 42
    EXTRA_CFLAGS, 43
    LDFLAGS, 42, 44
    LDFLAGS_NODIST, 44
    LDSHARED, 44
    LIBS, 44
    LINKCC, 44
    OPT, 35, 43
    PATH, 12, 23, 48, 53, 54, 56, 57, 64, 66, 69, 71
    PATHEXT, 66
    PROFILE_TASK, 32
    PURIFY, 44
    PY_BUILTIN_MODULE_CFLAGS, 44
    PY_CFLAGS, 43
    PY_CFLAGS_NODIST, 43
    PY_CORE_CFLAGS, 44
    PY_CORE_LDFLAGS, 45
    PY_CPPFLAGS, 42
    PY_LDFLAGS, 45
    PY_LDFLAGS_NODIST, 45
    PY_STDMODULE_CFLAGS, 43
    PYTHON_BASIC_REPL, 18
    PYTHON_COLORS, 11, 17
    PYTHON_CONTEXT_AWARE_WARNINGS, 11, 18
    PYTHON_CPU_COUNT, 10, 17
    PYTHON_DISABLE_REMOTE_DEBUG, 10, 17
    PYTHON_FROZEN_MODULES, 10, 17
    PYTHON_GIL, 11, 18, 107
    PYTHON_HISTORY, 18
    PYTHON_JIT, 18, 29
    PYTHON_MANAGER_DEFAULT, 48, 52, 57
    PYTHON_PERF_JIT_SUPPORT, 10, 17
    PYTHON_PRESITE, 11, 19
    PYTHON_THREAD_INHERIT_CONTEXT, 11, 18
    PYTHON_TLBC, 11, 18
    PYTHONASYNCIODEBUG, 15
    PYTHONBREAKPOINT, 12
    PYTHONCASEOK, 13
    PYTHONCOERCECLOCALE, 16, 27
    PYTHONDEBUG, 6, 12, 34
    PYTHONDEVMODE, 9, 16
    PYTHONDONTWRITEBYTECODE, 6, 13
    PYTHONDUMPREFS, 18, 35
    PYTHONDUMPREFSFILE, 18
    PYTHONEXECUTABLE, 14
    PYTHONFAULTHANDLER, 9, 14
    PYTHONHASHSEED, 7, 13
    PYTHONHOME, 6, 11, 12, 61, 62, 94
    PYTHONINSPECT, 7, 12
    PYTHONINTMAXSTRDIGITS, 9, 13
    PYTHONIOENCODING, 13, 16
    PYTHONLEGACYWINDOWSFSENCODING, 15
    PYTHONLEGACYWINDOWSSTDIO, 14, 15
    PYTHONMALLOC, 15, 33
    PYTHONMALLOCSTATS, 15

PYTHONNODEBUGRANGES, 10, 17
PYTHONNOUSERSITE, 8, 14
PYTHONOPTIMIZE, 7, 12
PYTHONPATH, 6, 12, 61, 62, 94
PYTHONPERFSUPPORT, 10, 17
PYTHONPLATLIBDIR, 12
PYTHONPROFILEIMPORTTIME, 9, 14
PYTHONPYCACHEPREFIX, 10, 13
PYTHONSAFEPATH, 7, 12
PYTHONSTARTUP, 7, 12, 13
PYTHONTRACEMALLOC, 9, 14
PYTHONUNBUFFERED, 8, 13
PYTHONUSERBASE, 14
PYTHONUTF8, 10, 16, 60
PYTHONVERBOSE, 8, 13
PYTHONWARNDEFAULTENCODING, 10, 17
PYTHONWARNINGS, 9, 14
variável de classe, **102**
variável de clausura, **102**
variável de contexto, **103**
variável livre, **105**
verificador de tipo estático, **115**
--version
opção de linha de comando, 6
visão de dicionário, **104**

W

-W
opção de linha de comando, 8
--with-address-sanitizer
opção de linha de comando, 35
--with-app-store-compliance
opção de linha de comando, 39
--with-assertions
opção de linha de comando, 35
--with-build-python
opção de linha de comando, 39
--with-builtin-hashlib-hashes
opção de linha de comando, 37
--with-computed-gotos
opção de linha de comando, 33
--with-dbmliborder
opção de linha de comando, 27
--with-dtrace
opção de linha de comando, 35
--with-ensurepip
opção de linha de comando, 32
--with-framework-name
opção de linha de comando, 39
--with-hash-algorithm
opção de linha de comando, 37
--with-libc
opção de linha de comando, 36
--with-libm
opção de linha de comando, 36
--with-libs
opção de linha de comando, 36
--with-lto
opção de linha de comando, 33
--with-memory-sanitizer
opção de linha de comando, 35
--with-openssl
opção de linha de comando, 37
--with-openssl-rpath
opção de linha de comando, 37
--without-c-locale-coercion
opção de linha de comando, 27
--without-decimal-contextvar
opção de linha de comando, 27
--without-doc-strings
opção de linha de comando, 33
--without-mimalloc
opção de linha de comando, 33
--without-pymalloc
opção de linha de comando, 33
--without-readline
opção de linha de comando, 36
--without-remote-debug
opção de linha de comando, 34
--without-static-libpython
opção de linha de comando, 36
--with-pkg-config
opção de linha de comando, 27
--with-platlibdir
opção de linha de comando, 27
--with-pydebug
opção de linha de comando, 35
--with-readline
opção de linha de comando, 36
--with-ssl-default-suites
opção de linha de comando, 37
--with-strict-overflow
opção de linha de comando, 34
--with-suffix
opção de linha de comando, 26
--with-system-expat
opção de linha de comando, 36
--with-system-libmpdec
opção de linha de comando, 36
--with-tail-call-interp
opção de linha de comando, 33
--with-thread-sanitizer
opção de linha de comando, 36
--with-trace-refs
opção de linha de comando, 35
--with-tzpath
opção de linha de comando, 26
--with-undefined-behavior-sanitizer
opção de linha de comando, 35
--with-universal-archs
opção de linha de comando, 38
--with-valgrind
opção de linha de comando, 35
--with-wheel-pkg-dir
opção de linha de comando, 27

X

-X
    opção de linha de comando, 9
-x
    opção de linha de comando, 9

Z

Zen do Python, 117
ZLIB_CFLAGS
    opção de linha de comando, 31
ZLIB_LIBS
    opção de linha de comando, 31